

The logo for FAVSY, featuring the word "FAVSY" in a bold, orange, sans-serif font. A thin orange horizontal line is positioned directly beneath the text. To the left of the text, a thin orange vertical line extends upwards from the baseline of the text.

PROYECTO FINAL DE CICLO

Cristian Guerrero Terciado
5-6-2026

1. Indice

1. Indice	2
2. Abstract.....	6
3. Justificación del Proyecto	6
4. Motivación del Proyecto.....	7
5. Introducción.....	7
6. Objetivos.....	8
6.1. Objetivo General	8
6.2. Objetivos Específicos.....	8
7. Estado del arte	9
8. Diseño	9
8.1. Diagrama de Casos de Uso	9
8.2. Diseño de la Base de Datos	10
8.2.1. Elección del Motor de Base de Datos	10
8.2.2. Esquema Conceptual: Modelo Entidad-Relación.....	11
8.2.3. Esquema lógico: Modelo Relacional	12
8.3. Diseño de la Interfaz Gráfica	17
8.3.1. Aplicación de Escritorio (.NET MAUI).....	17
8.3.2. PaginaLogin	18
8.3.3. PaginaRegistro.....	19
8.3.4. PaginaInicio	20
8.3.5. PaginaFavoresDisponibles.....	21
8.3.6. PaginaCrearFavor	22
8.3.7. PaginaMisGrupos y Mis Favores)	23
8.3.8. PaginaDetalleGrupo	24
8.3.9. PaginaDetalleChatGrupo y PaginaChatPrivado.....	25
8.3.10. PaginaNotificaciones.....	26
8.3.11. PaginaPerfil	27
8.3.12. PaginaInformes	27
8.3.13. PaginaAdministracion	28
8.3.14. Mini Aplicación Móvil (Kotlin / Android).....	30
8.3.15. MainActivity + FragmentoLogin	30
8.3.16. FragmentoRegistro.....	31

8.3.17.	ActividadInicio + BottomNavigationView	32
8.3.18.	FragmentoFavoresDisponibles	33
8.3.19.	FragmentoCrearFavor	34
8.3.20.	FragmentoMisGrupos	36
8.3.21.	FragmentoExplorarGrupos.....	37
8.3.22.	FragmentoCrearGrupo	38
8.3.23.	FragmentoDetalleGrupo	38
8.3.24.	FragmentoMisChats	39
8.3.25.	FragmentoChatDetalle	40
8.3.26.	FragmentoPerfil.....	42
9.	Desarrollo	44
9.1.	Arquitectura del Sistema.....	44
9.2.	Estructura del Backend	45
9.3.	Endpoints Principales de la API.....	46
9.3.1.	Autenticación y Usuarios	46
9.3.2.	Favores	47
9.3.3.	Chats.....	47
9.3.4.	Grupos.....	48
9.4.	Estructura del Cliente de Escritorio (.NET MAUI)	48
9.4.1.	Modelos del cliente escritorio	50
9.4.2.	Métodos de los Servicios HTTP	51
9.5.	Estructura del Cliente de movil (Kotlin/Android).....	54
9.5.1.	Modelos del cliente móvil.....	56
9.5.2.	Repositorios y métodos de acceso a datos.....	57
9.5.3.	ClienteRetrofit y detección de emulador	58
9.5.4.	Utilidades	58
9.6.	Gestión de Concurrencia.....	60
9.7.	Sistema de Roles y Perfiles de Usuario	60
9.8.	Ciclo de Vida de un Favor	61
9.9.	Sistema de Notificaciones	61
10.	Desarrollo Empresarial.....	62
10.1.	PRESENTACIÓN DEL PROYECTO Y DEL EMPRENDEDOR	62
10.1.1.	a) Presentación del proyecto	62

10.1.2.	b) El emprendedor	63
10.2.	EL MERCADO Y LOS CLIENTES	71
10.2.1.	a) Tipo de mercado	71
10.2.2.	b) El segmento de mercado	71
10.2.3.	c) Estudio de mercado: los clientes.....	72
10.3.	ENTORNO Y LA COMPETENCIA	75
10.3.1.	a) El entorno general (macroentorno)	75
10.3.2.	b) El entorno específico (microentorno).....	77
10.3.3.	c) La competencia	79
10.3.4.	d) Análisis DAFO	81
10.3.5.	e) La localización de la empresa	86
10.3.6.	f) Misión, visión y valores.....	87
10.3.7.	g) Responsabilidad Social Corporativa	88
10.4.	EL MARKETING	90
10.4.1.	a) Estrategias de promoción o posicionamiento	90
10.4.2.	b) Estrategias de producto	91
10.4.3.	c) Estrategias de precio	93
10.4.4.	d) Estrategias de distribución.....	94
10.4.5.	e) Atención al cliente	95
10.5.	RECURSOS HUMANOS.....	97
10.5.1.	a) Organigrama de la empresa	97
10.5.2.	b) Diseño de puestos y selección de personal	97
10.5.3.	c) Plan de motivación laboral.....	99
10.5.4.	d) Contratación y coste de los trabajadores	99
10.5.5.	e) Organización de la prevención.....	100
10.6.	FORMA JURÍDICA.	101
10.6.1.	a) Forma jurídica y motivos por los que se ha elegido.	101
10.7.	ANÁLISIS ECONÓMICO-FINANCIERO.....	102
10.7.1.	a) Análisis de Costes.....	102
10.7.2.	b) Inversiones y gastos iniciales	103
10.7.3.	c) Plan de financiación	104
10.7.4.	d) Plan de tesorería	107
10.7.5.	e) Previsión de resultados y balance de situación.	109

10.8.	PLAN DE PUESTA EN MARCHA	116
10.8.1.	a) Trámites y plazos para la puesta en marcha.	116
10.8.2.	b) Documentación administrativa-contable.	118
11.	Metodología de Programación	120
11.1.	Tipo de Metodología.....	120
11.2.	Fases de Trabajo y Diagrama de Gantt.....	120
11.3.	Organización del Código	120
11.4.	Pruebas Realizadas.....	120
12.	Conclusiones	121
13.	Trabajo Futuro y Posibles Mejoras.....	122
14.	Glosario de Términos	124
15.	Bibliografía	125
16.	Enlace al Repositorio.....	125
17.	ANEXO I CURRÍCULUM VITAE EUROPASS.....	126
18.	ANEXO II NOMINA.....	127

2. Abstract

Este proyecto consiste en el desarrollo de una plataforma colaborativa de intercambio de favores entre usuarios de una misma comunidad o entorno cercano. La aplicación está compuesta por una aplicación de escritorio desarrollada en .NET MAUI, una mini aplicación móvil complementaria y un backend REST API construido con FastAPI y Python. La información se almacena en una base de datos documental NoSQL (MongoDB), elegida por su flexibilidad y alto rendimiento en escrituras concurrentes. El sistema permite a los usuarios registrarse, publicar favores, aceptarlos, comunicarse mediante chat privado y grupal, gestionar grupos temáticos y recibir valoraciones tras completar cada tarea. Se implementa un sistema de roles (usuario básico, administrador de grupo y administrador global) con validación en el servidor mediante JWT.

This project consists in the development of a collaborative platform for exchanging favours between users within the same community or close environment. The application is composed of a desktop application developed in .NET MAUI, a complementary mobile mini-application, and a REST API backend built with FastAPI and Python. Data is stored in a NoSQL document database (MongoDB), chosen for its flexibility and high performance in concurrent writes. The system allows users to register, publish favours, accept them, communicate via private and group chat, manage thematic groups, and receive ratings after completing each task. A role-based system (basic user, group administrator and global administrator) is implemented with server-side validation using JWT.

3. Justificación del Proyecto

En el entorno cotidiano de vecinos, estudiantes y asociaciones locales existe una necesidad no resuelta de coordinación eficiente para pequeñas tareas: recoger paquetes, ayudar con mudanzas, acompañar a personas mayores o compartir habilidades. Estas interacciones se gestionan hoy de forma informal (grupos de WhatsApp, avisos en papel) sin ningún mecanismo que garantice seguimiento, valoración de calidad ni comunicación estructurada.

La justificación de este proyecto radica en la ausencia de una herramienta que combine, en un mismo ecosistema, la gestión de favores con ciclo de vida completo, mensajería en tiempo real, grupos temáticos administrables y un sistema de reputación transparente. Además, la mayoría de soluciones existentes sólo ofrecen cliente web o móvil, mientras que este proyecto incorpora también una aplicación de escritorio, lo que lo hace especialmente adecuado para entornos domiciliarios o de oficina.

4. Motivación del Proyecto

La motivación principal surge de la observación directa de cómo los vecinos y compañeros de estudios gestionan sus necesidades de ayuda mutua: de forma caótica, sin trazabilidad y con el riesgo de que las peticiones se pierdan en grupos de mensajería saturados. Existe una oportunidad real de digitalizar y estructurar este proceso aplicándole tecnologías modernas aprendidas durante el ciclo formativo.

Desde un punto de vista técnico, el proyecto representa un reto completo que integra base de datos NoSQL, API REST con autenticación JWT, cliente de escritorio multiplataforma (.NET MAUI), aplicación móvil y comunicación en tiempo real, cubriendo así las competencias de varios módulos del ciclo.

5. Introducción

La economía colaborativa ha experimentado un crecimiento notable en la última década. Plataformas como Nextdoor, VolunteersNow o las secciones de grupos de Facebook han demostrado que existe demanda de herramientas digitales para la ayuda mutua. Sin embargo, estas soluciones presentan limitaciones importantes: falta de seguimiento estructurado del favor, ausencia de chat en tiempo real integrado, carencia de grupos con administración de roles y, sobre todo, no existen versiones de escritorio pensadas para este tipo de uso comunitario.

Este proyecto propone una arquitectura cliente-servidor moderna: un backend construido con FastAPI (Python) que expone una API REST con autenticación basada en tokens JWT y persiste los datos en MongoDB, una base de datos documental NoSQL. MongoDB es especialmente adecuada para este dominio porque los documentos (favores, chats, grupos) tienen estructuras variables y anidadas que en una base de datos relacional exigirían múltiples tablas y JOINS costosos. Además, MongoDB ofrece alta velocidad de inserción, indexación flexible y escalabilidad horizontal, características críticas cuando el número de mensajes de chat puede crecer rápidamente.

El cliente principal es una aplicación de escritorio desarrollada con .NET MAUI, framework multiplataforma de Microsoft que permite compilar para Windows, macOS, Android e iOS

desde un único código base. La mini aplicación móvil comparte el mismo backend y ofrece una interfaz simplificada pensada para el uso en movilidad.

6. Objetivos

6.1. Objetivo General

Desarrollar una plataforma colaborativa digital, compuesta por una aplicación de escritorio, una aplicación móvil y un backend REST API, que permita a los usuarios gestionar el ciclo de vida completo de un intercambio de favores, incluyendo publicación, aceptación, comunicación, finalización y valoración.

6.2. Objetivos Específicos

- Implementar un sistema de autenticación seguro basado en JWT que controle el acceso a todos los endpoints de la API.
- Diseñar e implementar un módulo de gestión de favores con estados (disponible, aceptado, finalizado, cancelado) y lógica de transición entre ellos.
- Desarrollar un sistema de mensajería privada (chat entre dos usuarios) y mensajería de grupo, con indicadores de lectura.
- Crear un sistema de grupos temáticos con roles diferenciados (administrador y miembro), incluyendo flujo de solicitudes de unión e invitaciones.
- Implementar un sistema de valoración de usuarios (1-5 estrellas) que calcule automáticamente la calificación media.
- Construir el cliente de escritorio con .NET MAUI siguiendo el patrón MVVM, con navegación por Shell y enlace de datos (data binding).
- Desarrollar la mini aplicación móvil con .NET MAUI, reutilizando los servicios HTTP del cliente de escritorio.
- Garantizar la seguridad y la integridad de los datos mediante validaciones en el backend antes de ejecutar cualquier operación sensible.

7. Estado del arte

En los últimos años han surgido múltiples plataformas colaborativas que permiten a personas de una misma comunidad ayudarse mutuamente. A continuación se analizan las principales soluciones existentes y sus limitaciones:

Plataforma	Limitaciones detectadas
Nextdoor	Tabón de anuncios sin seguimiento del favor de inicio a fin; sin valoraciones obligatorias.
VolunteersNow	Orientada a voluntariado formal; no permite favores económicos ni chat integrado.
Facebook Groups	No estructura el flujo favor; sin estados ni historial trazable.
TaskRabbit	Solo profesionales que cobran; no permite intercambios gratuitos o de trueque.
Wallapop	Plataforma de compraventa, no de intercambio de servicios o favores.

El sistema propuesto mejora estas limitaciones integrando: gestión de favores con ciclo de vida completo, chat en tiempo real, grupos con roles administrables, valoraciones obligatorias, soporte para favores gratuitos/económicos/trueque y cliente de escritorio (poco común en apps sociales).

8. Diseño

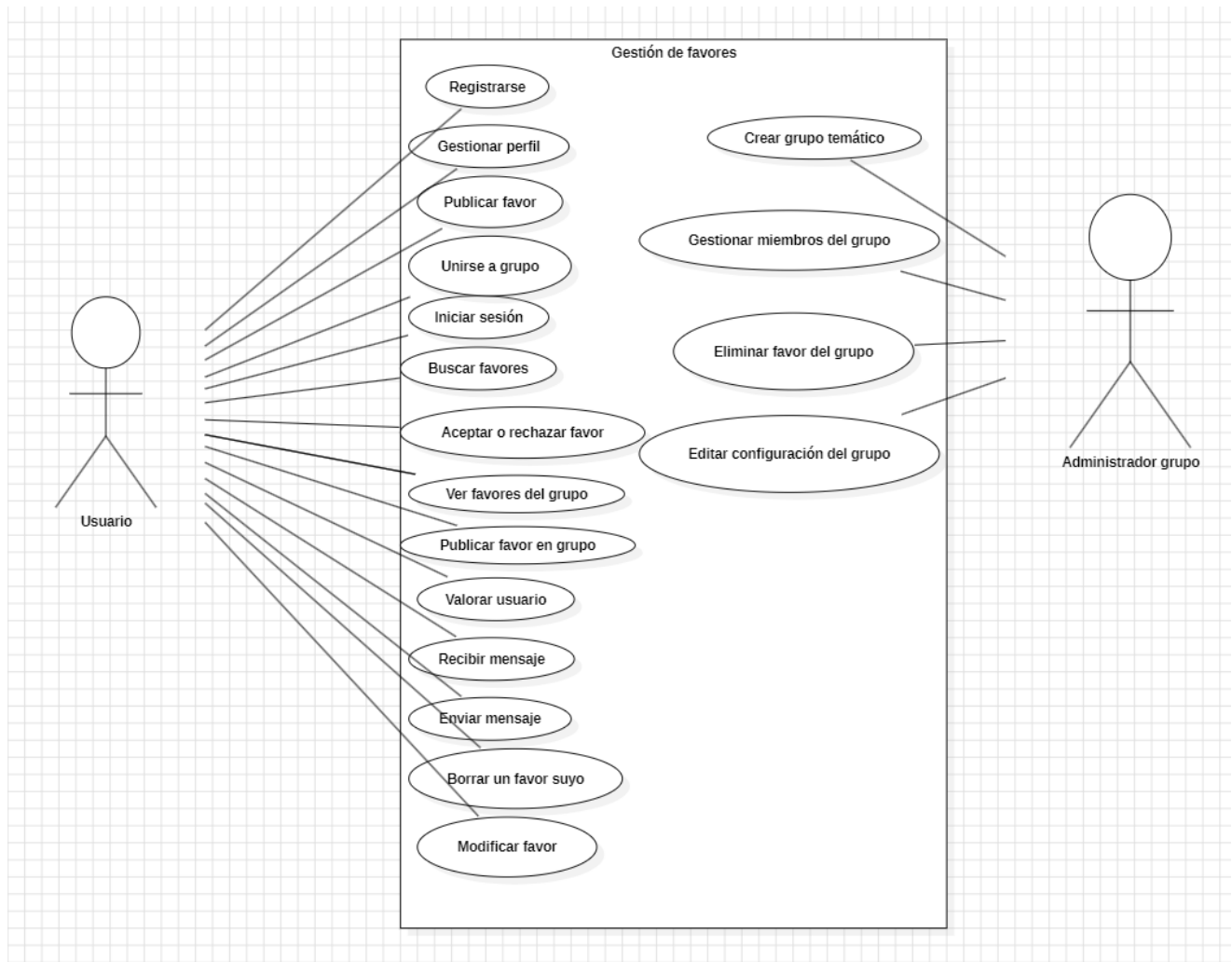
8.1. Diagrama de Casos de Uso

El siguiente diagrama de casos de uso recoge las interacciones principales entre los tres actores del sistema (Usuario Básico, Administrador de Grupo y Administrador Global) y las funcionalidades de la plataforma. Los casos de uso se agrupan en cinco áreas funcionales: autenticación, favores, chats, grupos y notificaciones.

Actor Usuario Básico: puede registrarse, iniciar sesión, crear/aceptar/cancelar/finalizar favores, enviar mensajes, unirse a grupos, pedir invitación y valorar a otros usuarios.

Actor Administrador de Grupo: hereda todas las acciones del usuario básico y además puede aceptar/rechazar solicitudes de unión, expulsar miembros, cambiar roles, editar y eliminar el grupo.

Actor Administrador Global: supervisa la actividad general de la plataforma, puede bloquear usuarios y eliminar grupos.



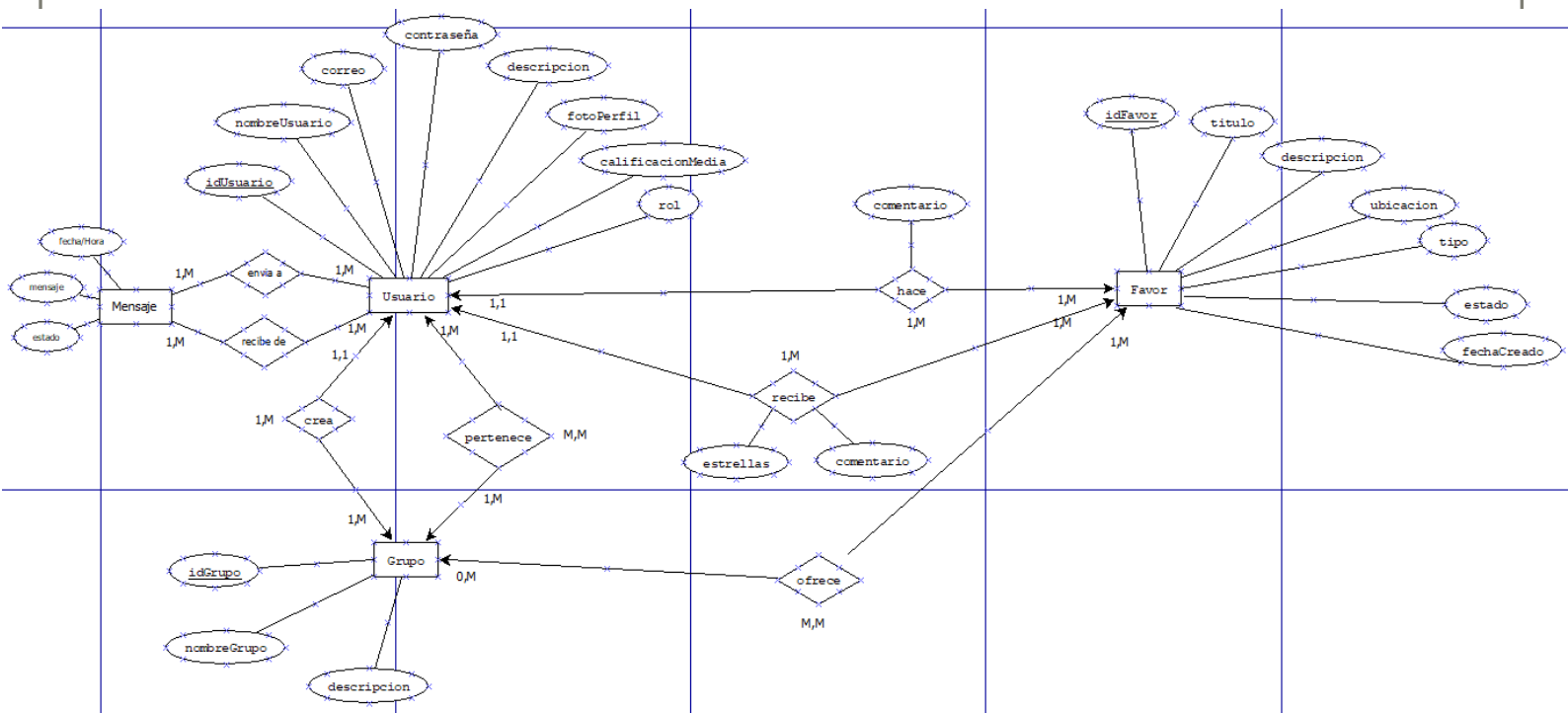
8.2. Diseño de la Base de Datos

8.2.1. Elección del Motor de Base de Datos

Para este proyecto se ha elegido MongoDB como motor de base de datos por los siguientes motivos:

- **Modelo documental (JSON/BSON):** los documentos de la aplicación (favores, chats, grupos) tienen estructuras anidadas y variables que se mapean de forma natural a documentos JSON, evitando el problema de impedancia objeto-relacional.
- **Alto rendimiento en escrituras:** el chat genera un volumen elevado de inserciones. MongoDB con operaciones como `$push` permite añadir mensajes al array de un documento en una sola operación atómica, sin necesidad de transacciones complejas.
- **Flexibilidad de esquema:** durante el desarrollo, los modelos evolucionan. MongoDB permite añadir campos sin migraciones costosas.
- **Escalabilidad horizontal:** el sharding nativo de MongoDB permite distribuir la carga cuando el número de usuarios crece.
- **Integración con Python (Motor):** la librería Motor proporciona acceso asíncrono a MongoDB, compatible con el modelo `async/await` de FastAPI.

8.2.2. Esquema Conceptual: Modelo Entidad-Relación



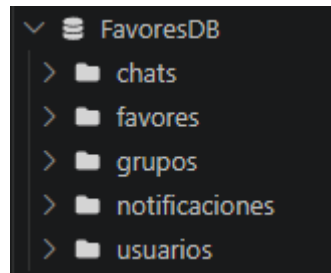
El modelo entidad-relación de la aplicación está compuesto por cuatro entidades principales. A continuación se describe cada entidad y sus relaciones:

Entidad	Atributos principales
Usuario	id, correo (PK), nombreUsuario, descripcion, fotoPerfil, calificacionMedia, rol, historialNotas
Favor	id, titulo, descripcion, modalidad, ubicacion, tipoCompensacion, cantidadDinero, recompensaFavor, estado, fechaCreacion, idUsuarioOfrece, idUsuarioSolicita
Grupo	id, nombreGrupo, descripcion, fotoGrupo, miembros[], solicitudes[], fechaCreacion
Chat	id (correo1_correo2 o idGrupo), participantes[], mensajes[], ultimaActividad
Mensaje	emisor, contenido, fechaEnvio, leido
Notificacion	usuarioDestino, mensaje, tipo, leida, idReferencia, fecha

Relaciones principales:

- Un Usuario publica 0..N Favores (como ofertante) y puede aceptar 0..N Favores (como solicitante).
- Un Usuario pertenece a 0..N Grupos; un Grupo tiene 1..N Usuarios (relación muchos a muchos representada mediante el array miembros[] dentro del documento Grupo).
- Un Usuario recibe 0..N Notificaciones.

Aquí se ve una imagen de las colecciones



Colección MongoDB	Equivalente relacional
usuarios	USUARIO
favores	FAVOR
grupos	GRUPO + USUARIO_GRUPO (miembros embebidos)
chats	CHAT + MENSAJE (mensajes embebidos)
notificaciones	NOTIFICACION

8.2.3. Esquema lógico: Modelo Relacional

USUARIO

Campo	Tipo
idUsuario	INT (PK)
nombreUsuario	VARCHAR
contraseñaHash	VARCHAR
correo	VARCHAR
descripcion	TEXT

Campo	Tipo
fotoPerfil	VARCHAR
calificacionMedia	DECIMAL
rol	VARCHAR

FAVOR

Campo	Tipo
idFavor	INT (PK)
titulo	VARCHAR
descripcion	TEXT
ubicacion	VARCHAR
modalidad	VARCHAR
tipoCompensacion	VARCHAR
cantidadDinero	DECIMAL
recompensaFavor	TEXT
estado	VARCHAR
fechaCreacion	DATETIME
idUsuarioOfrece	INT (FK)
idUsuarioSolicita	INT (FK)

GRUPO

Campo	Tipo
idGrupo	INT (PK)
nombreGrupo	VARCHAR
descripcion	TEXT
fotoGrupo	VARCHAR
fechaCreacion	DATETIME
idUsuarioCreador	INT (FK)

MENSAJE

Campo	Tipo
idMensaje	INT (PK)
idEmisor	INT (FK)
idReceptor	INT (FK)
mensaje	TEXT
fechaHora	DATETIME
estado	VARCHAR

USUARIO_GRUPO

Campo	Tipo
idUsuario	INT (PK, FK)
idGrupo	INT (PK, FK)
rol	VARCHAR

Campo	Tipo
fechaUnion	DATETIME

GRUPO_FAVOR

Campo	Tipo
idGrupo	INT (PK, FK)
idFavor	INT (PK, FK)

VALORACION

Campo	Tipo
idValoracion	INT (PK)
idFavor	INT (FK)
idUsuarioValora	INT (FK)
idUsuarioValorado	INT (FK)
estrellas	DECIMAL
comentario	TEXT
fechaValoracion	DATETIME

CUMPLIMIENTO_FAVOR

Campo	Tipo
idCumplimiento	INT (PK)
idFavor	INT (FK)
idUsuario	INT (FK)
comentario	TEXT

Campo	Tipo
fechaCumplimiento	DATETIME

SOLICITUD_GRUPO

Campo	Tipo
idSolicitud	INT (PK)
idUsuario	INT (FK)
idGrupo	INT (FK)
comentario	TEXT
fechaSolicitud	DATETIME
estado	VARCHAR

NOTIFICACION

Campo	Tipo
idNotificacion	INT (PK)
idUsuario	INT (FK)
mensaje	TEXT
tipo	VARCHAR
leida	BOOLEAN
idReferencia	VARCHAR
fecha	DATETIME

8.3.Diseño de la Interfaz Gráfica

8.3.1. Aplicación de Escritorio (.NET MAUI)

La aplicación de escritorio sigue el patrón de diseño MVVM (Model-View-ViewModel), que separa la lógica de negocio de la interfaz gráfica. La navegación principal se implementa mediante AppShell, que define la jerarquía de páginas y las rutas de navegación.

A continuación se describe el flujo de pantallas:

- Página de Login → El usuario introduce correo y contraseña. Si las credenciales son correctas, el token JWT se almacena en SecureStorage y se navega al Dashboard principal.
- Dashboard (Página principal) → Muestra el listado de favores disponibles. Desde aquí el usuario puede crear un nuevo favor, acceder al chat o navegar a grupos.
- Detalle de Favor → Muestra toda la información del favor seleccionado. Si el usuario no es el propietario y el favor está disponible, aparece el botón Aceptar.
- Lista de Chats → Muestra todas las conversaciones del usuario, ordenadas por última actividad.
- Detalle de Chat → Interfaz de mensajería con historial de mensajes, campo de texto y botón de envío. Los mensajes del usuario aparecen alineados a la derecha; los del interlocutor a la izquierda.
- Lista de Grupos → Muestra los grupos del usuario y permite buscar nuevos grupos.
- Detalle de Grupo → Lista de miembros y, si el usuario es administrador, opciones de gestión (expulsar, cambiar rol, aceptar/rechazar solicitudes, editar grupo).

La navegación principal se gestiona mediante AppShell, que define rutas absolutas (con prefijo //) para las páginas del menú Flyout y rutas relativas para páginas de detalle accesibles por navegación anidada (GoToAsync).

8.3.2. PaginaLogin

Ruta Shell: //PaginaLogin. Accesible sin sesión iniciada, dentro del TabBar de autenticación.

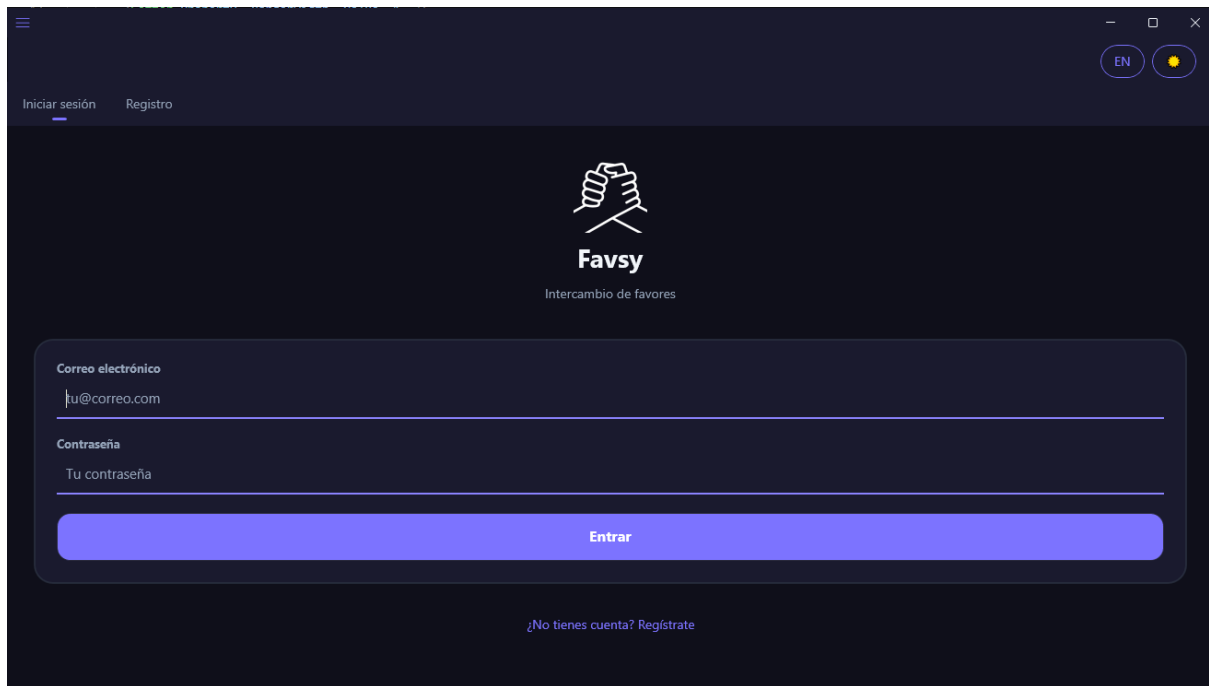
Estructura de layout

Contenedor raíz: ScrollView.

- VerticalStackLayout (Padding=32,24,32,32) — columna principal:
- VerticalStackLayout (centrado, Margin=0,12,0,40): Image del logo (LogoImagen), Label con App_Nombre (TituloPrimario), Label con App_Subtitulo (TextoSecundario).
- Border (StrokeShape RoundedRectangle 20, fondo ColorSuperficie, Padding=24) — tarjeta del formulario:
 - Campo correo: Label + EntryLimpio + BoxView subrayado.
 - Campo contraseña: Label + EntryLimpio (IsPassword=True) + BoxView subrayado.
 - Label MensajeError — visible solo cuando HayError.
 - Button Conectar (ComandoLogin, EstiloBotonPrimario, desactivado mientras EstaOcupado).
 - ActivityIndicator — visible mientras EstaOcupado.
- Button de registro (ComandoIrRegistro, EstiloBotonLink).

Lógica destacada (LoginVistaModelo)

- EjecutarLoginAsync(): valida que correo y contraseña no estén vacíos → llama ServicioAutenticacion.LoginAsync() → si éxito, llama AppShell.MostrarMenuTrasLogin() que activa el Flyout y navega a //PaginaInicio → si error HTTP, muestra el campo detail del JSON de FastAPI en MensajeError.
- El token JWT recibido se persiste en SecureStorage (DPAPI en Windows) mediante GuardarSesionAsync().
- Si ya existe un token al arrancar, AppShell.CreateWindow() lo recupera y aplica automáticamente en todos los servicios HTTP sin volver a pedir credenciales.



8.3.3. PaginaRegistro

Ruta Shell: //PaginaRegistro. Segunda pestaña del TabBar de autenticación.

Estructura de layout

Contenedor raíz: ScrollView.

- VerticalStackLayout (Padding=32,24,32,32):
- Cabecera: Label Registro_Titulo + Label Registro_Subtitulo.
- Border (tarjeta formulario, mismo estilo que Login):
 - 4 campos con Label + EntryLimpio + BoxView: Nombre de usuario, Correo, Contraseña, Confirmar contraseña.
 - Label MensajeError.
 - Button Registrarme (ComandoRegistrar, EstiloBotonPrimario).
 - ActivityIndicator.
- Button vuelta al login (ComandoVolverLogin, EstiloBotonLink).

Validaciones antes de registrar (RegistroVistaModelo)

- Todos los campos obligatorios deben estar rellenos.
- El nombre de usuario debe tener al menos 3 caracteres.
- El correo debe contener @.
- La contraseña debe tener al menos 6 caracteres.
- Las contraseñas deben coincidir.
- Si el registro tiene éxito, muestra un DisplayAlert de confirmación y navega a //PaginaLogin.

Sign in Sign up

Create account

Join the community

Username
Your name

Email
you@email.com

Password
At least 6 characters

Confirm password
Repeat your password

Sign up

Already have an account? Sign in

8.3.4. PaginaInicio

Ruta Shell: //PaginaInicio. Primera pantalla tras el login.

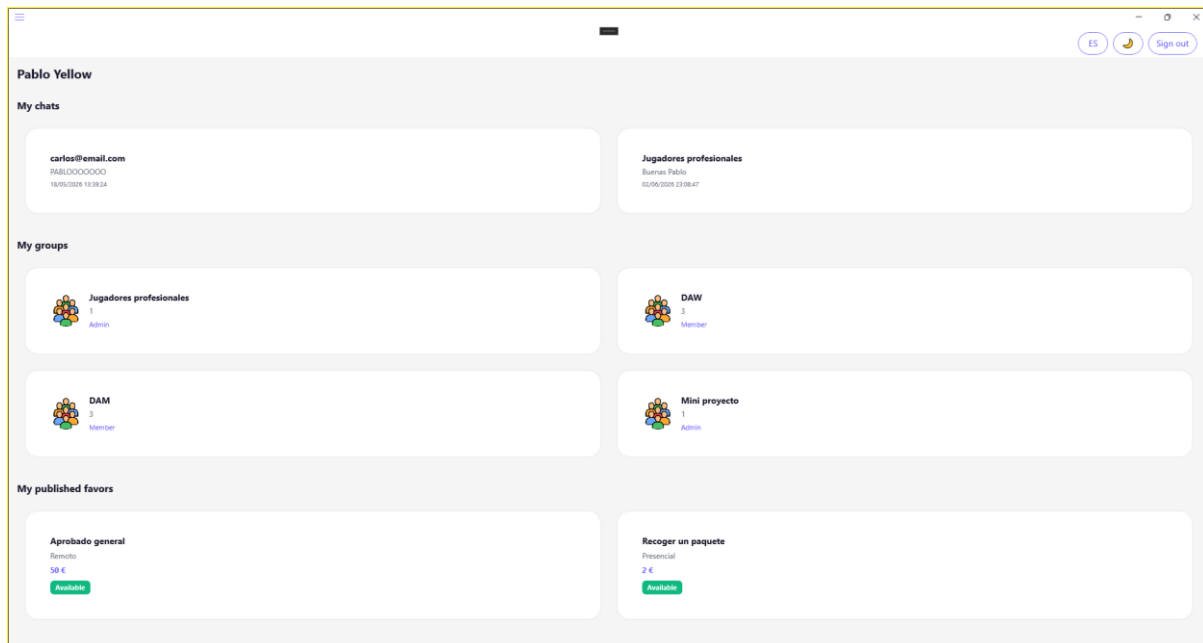
Estructura de layout

Contenedor raíz: RefreshView (comando ComandoRefrescar).

- ScrollView → VerticalStackLayout (Padding=16, Spacing=28):
 - Label de bienvenida con el nombre del usuario (recurso Inicio_BienvenidaFormato).
 - Label MensajeError.
 - Sección Mis Chats: Label + CollectionView con GridItemsLayout (2 columnas, HorizontalItemSpacing=12). Cada tarjeta muestra título del chat, último mensaje y fecha de última actividad. Al pulsar navega a chatprivado?correoOtro=....
 - Sección Mis Grupos: Label + CollectionView con foto de grupo circular (EllipseGeometry), nombre, número de miembros y rol del usuario. Al pulsar navega a detallechatgrupo?grupold=....
 - Sección Mis Favores: Label + CollectionView con título, modalidad, compensación y badge de estado con color dinámico.

Lógica destacada (InicioVistaModelo)

- Las tres peticiones de carga (chats, grupos y favores) se lanzan en paralelo con Task.WhenAll() para minimizar el tiempo de espera.
- Los chats duplicados de grupos (el mismo grupo puede generar varios chats) se eliminan agrupando por NombreGrupo y quedándose con el más reciente.
- El TextoRol de cada grupo se resuelve con las claves de idioma RolAdmin / RolMiembro del diccionario activo.



8.3.5. PaginaFavoresDisponibles

Ruta Shell: //PaginaFavoresDisponibles. Accesible desde el menú Flyout.

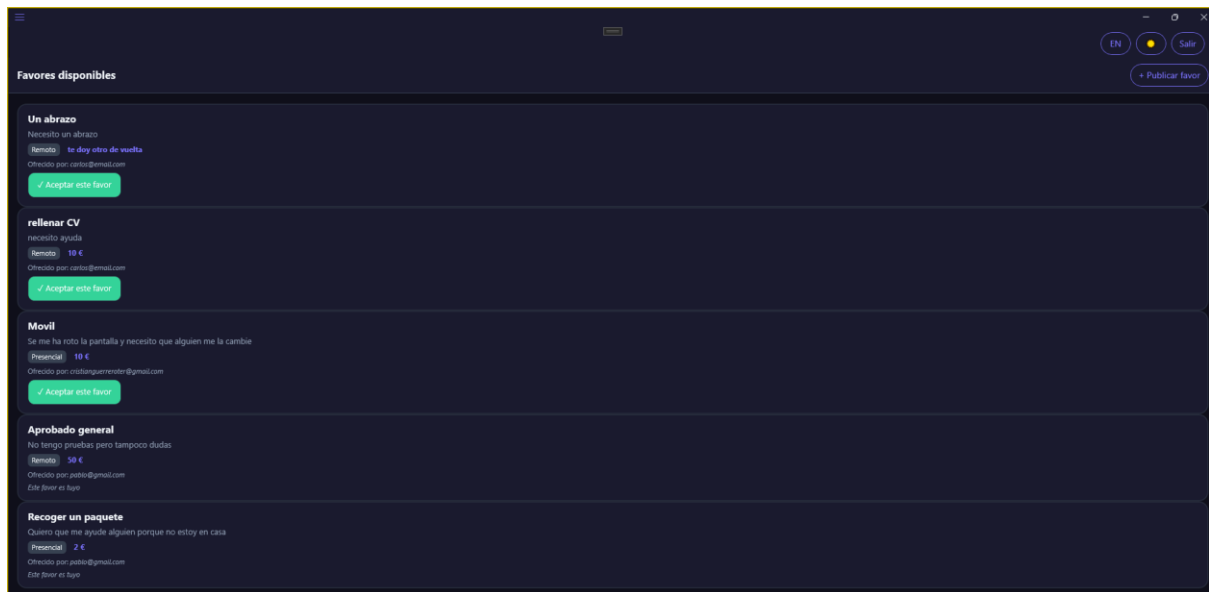
Estructura de layout

Contenedor raíz: Grid (RowDefinitions=Auto,Auto,*).

- Fila 0 — Cabecera inline: Grid (ColumnDefinitions=*,Auto) con Label del título y Button "+ Publicar favor" que navega a crearfavor al hacer clic.
- Fila 1 — BoxView separador de 1px con ColorBorde.
- Fila 2 — RefreshView (comando ComandoRefrescar):
 - Grid (Padding=16) con Label de error + CollectionView de favores.
 - Cada tarjeta muestra: título, descripción (2 líneas), badge de modalidad, compensación, correo del ofertante y botón "Aceptar este favor" (solo visible si PuedeAceptar=True).
 - Si el favor es propio, muestra el texto "Este favor es tuyo" en lugar del botón.

Lógica destacada (FavoresDisponiblesVistaModelo)

- Al cargar, asigna PuedeAceptar = true a todos los favores cuyo IdUsuarioOfrece sea distinto del correo del usuario logueado.
- AceptarFavorAsync() llama al endpoint PUT /favores/{id}/aceptar y recarga la lista completa si tiene éxito.



8.3.6. PaginaCrearFavor

Navegación: GoToAsync("crearfavor") desde PaginaFavoresDisponibles.

Estructura de layout

Contenedor raíz: ScrollView → VerticalStackLayout (Padding=24).

- Border (tarjeta, StrokeShape RoundedRectangle 20) con formulario completo:
 - Título: Label + EntryLimpio + BoxView (mínimo 5 caracteres).
 - Descripción: Label + Editor (100px alto, mínimo 10 caracteres).
 - Modalidad: HorizontalStackLayout con Label "Presencial" + CheckBox (EsPresencial).
 - Ubicación (visible solo si MostrarUbicacion=True): Label + EntryLimpio.
 - Tipo de compensación: Label + Picker con opciones Favor / Dinero / Ambos (IndexCompensacion).
 - Cantidad en euros (visible si MostrarCamposDinero=True): Label + EntryLimpio (teclado Numeric).
 - ¿Qué pides a cambio? (visible si MostrarCamposFavor=True): Label + EntryLimpio.
 - Label de error, ActivityIndicator.
 - Button "Publicar favor" (ComandoCrear, EstiloBotonPrimario).
 - Button "Cancelar" (ComandoCancelar, EstiloBotonSecundario).

The screenshot shows a mobile application interface for creating a favor. The form is titled 'Título' with a minimum of 5 characters. Below it is a 'Descripción' field with a minimum of 10 characters. There is a 'Presencial' checkbox which is checked. The 'Ubicación' field is for a Google Maps link. The 'Tipo de compensación' field is set to 'Ambos'. The 'Cantidad (€)' field has an example of '10.50'. The '¿Qué pides a cambio?' field is for describing the favor. At the bottom, there are two buttons: 'Publicar favor' (orange) and 'Cancelar' (grey).

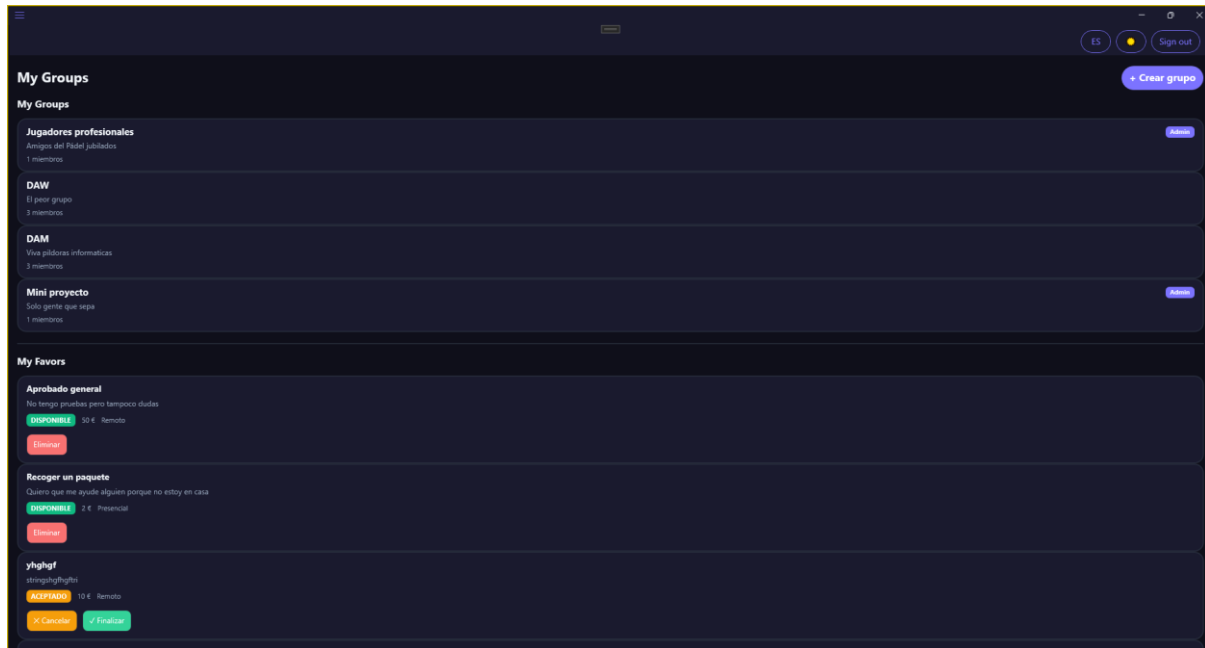
8.3.7. PaginaMisGrupos y Mis Favores)

Ruta Shell: //PaginaMisGrupos.

Estructura de layout

Contenedor raíz: RefreshView → ScrollView → VerticalStackLayout (Padding=16).

- Cabecera: Grid (ColumnDefinitions=*,Auto) con Label MisGruposTitulo y Button "+ Crear grupo" (navega a creargrupo).
- Label de error y ActivityIndicator.
- Sección Mis Grupos: Label + CollectionView. Cada tarjeta tiene: badge "Admin" (visible si EsAdmin=True), nombre del grupo, descripción y número de miembros. Al pulsar, navega a detallegrupo?grupold=....
- BoxView separador.
- Sección Mis Favores: Label + CollectionView. Cada tarjeta tiene badge de estado con ColorEstadoVisual, resumen de compensación, modalidad y tres botones de acción condicionales:
 - Button "Eliminar" (rojo, visible si MostrarEliminar=True).
 - Button "X Cancelar" (naranja, visible si MostrarCancelarFinalizar=True).
 - Button "✓ Finalizar" (verde, visible si MostrarCancelarFinalizar=True).



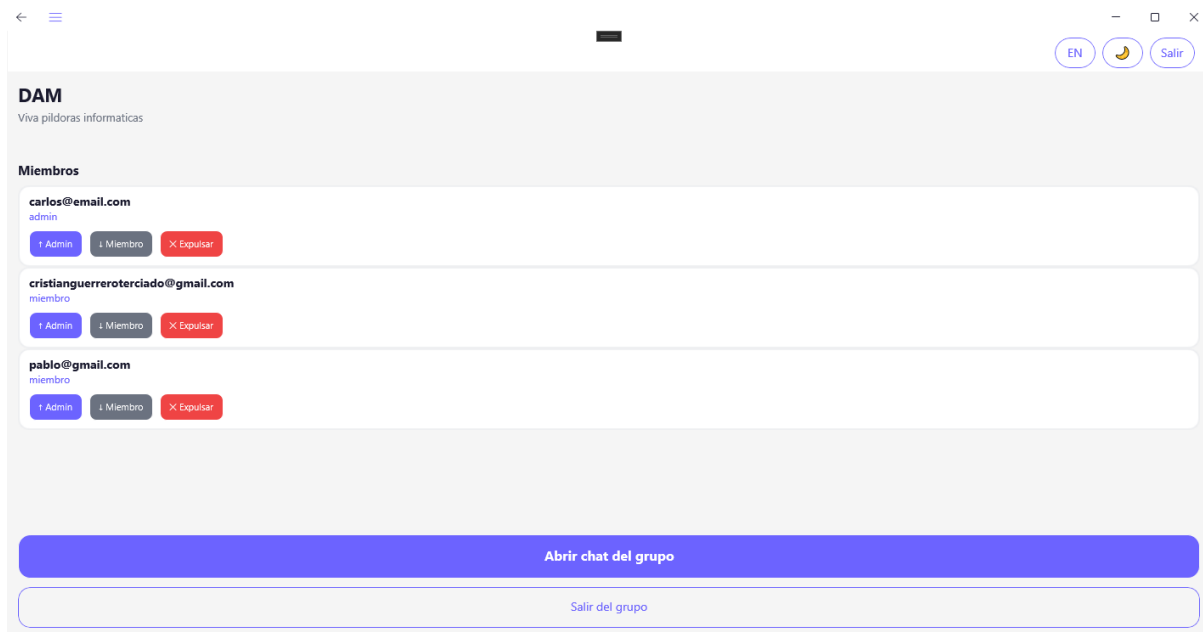
8.3.8. PaginaDetalleGrupo

Navegación: `GoToAsync("detallegrupo?grupold={id}")`.

Estructura de layout

Contenedor raíz: `Grid (RowDefinitions=Auto,Auto,*,Auto, Padding=16)`.

- Image del grupo (circular, `EllipseGeometry Radio=40`, centrada).
- Fila 0 — Cabecera: `Label NombreGrupo (FontSize=22)` + `Label Descripcion`.
- Fila 1 — `Label de error` y `ActivityIndicator`.
- Fila 2 — `CollectionView` de miembros. Cada tarjeta muestra correo y rol. Si `EsAdmin=True` en el `VistaModelo`, cada tarjeta expone tres botones de gestión: "↑ Admin" (`ComandoHacerAdmin`), "↓ Miembro" (`ComandoQuitarAdmin`) y "× Expulsar" (`ComandoExpulsar`).
- Fila 3 — Botones de acción: "Abrir chat del grupo" (`ComandoAbrirChat`, navega a `detallechatgrupo?grupold=...`) y "Salir del grupo" (`ComandoSalir`).



8.3.9. PaginaDetalleChatGrupo y PaginaChatPrivado

Navegación: `GoToAsync("detallechatgrupo?grupold={id}")` y `GoToAsync("chatprivado?correoOtro={correo}")`.

Ambas pantallas comparten la misma estructura de layout y solo difieren en el `VistaModelo` que las gestiona.

Estructura de layout

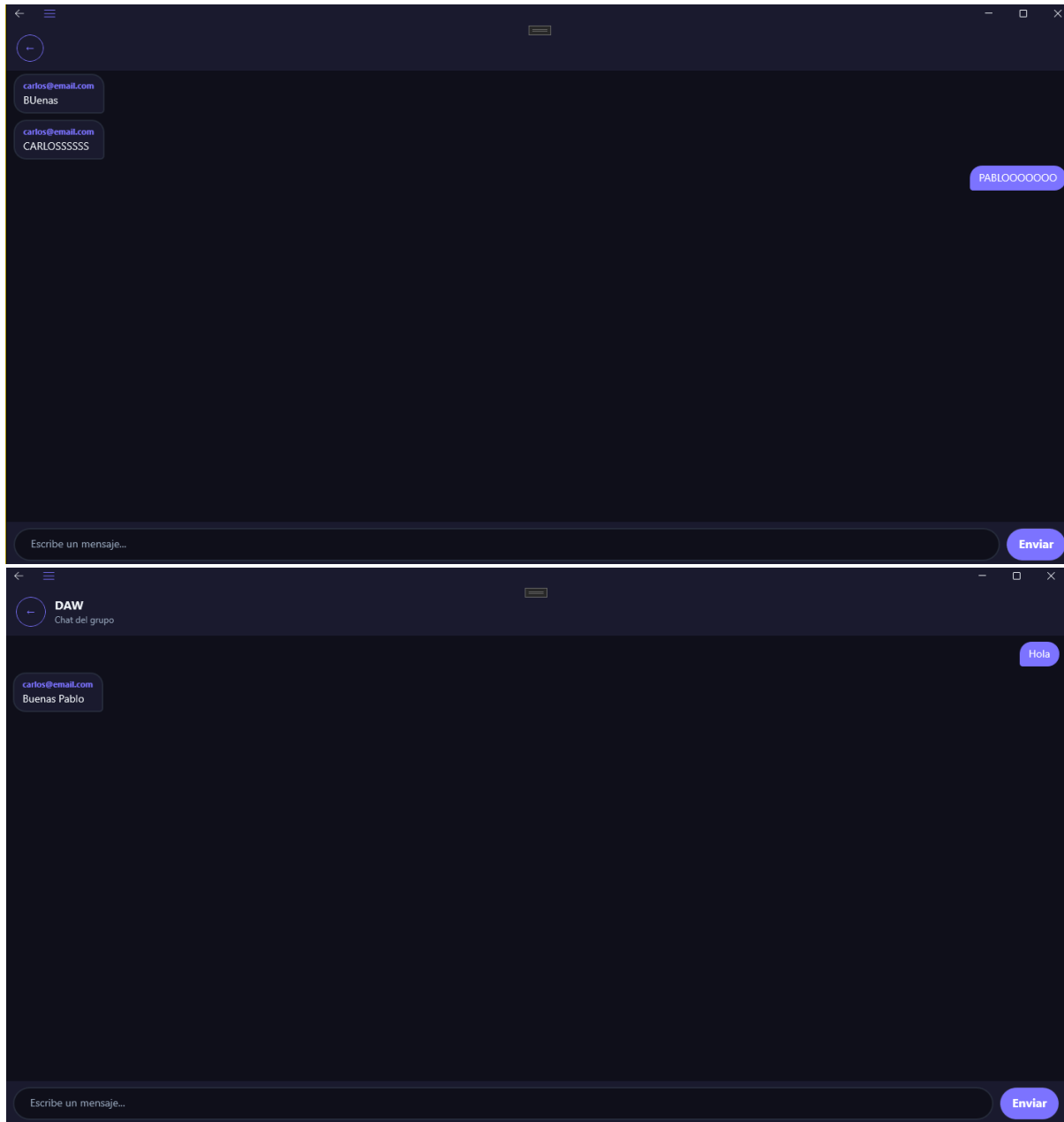
Contenedor raíz: `Grid (RowDefinitions=Auto,*,Auto, Shell.NavBarIsVisible=False)`.

- Fila 0 — Barra de cabecera personalizada: `Grid (ColumnDefinitions=Auto,*)` con `Button "<"` (navega hacia atrás) y `Label` con el nombre del interlocutor o grupo.
- Fila 1 — `CollectionView` de mensajes (`mensajesCollectionView`). Cada mensaje tiene dos plantillas según `EsMio`:
 - Mensaje propio: `Border` con fondo `ColorPrimario`, alineado a la derecha, esquinas redondeadas 16,16,4,16.
 - Mensaje ajeno: `Border` con borde `ColorBorde`, alineado a la izquierda, esquinas 16,16,16,4, con `Label` del emisor en `ColorPrimario`.
- Fila 2 — Barra de entrada: `Border` circular con `Entry` (`ReturnCommand=ComandoEnviar`) + `Button "Enviar"`.

Lógica destacada

- Al aparecer la pantalla (`OnAppearing`), carga los mensajes y llama a `ScrollTo(count-1, End)` con un pequeño retraso para posicionarse en el último mensaje.
- Tras cada envío exitoso, añade el mensaje a la `ObservableCollection` directamente sin recargar todo el chat.

- Marca los mensajes como leídos llamando a PUT /chat/{id}/leer-chats tras la carga inicia



8.3.10. PaginaNotificaciones

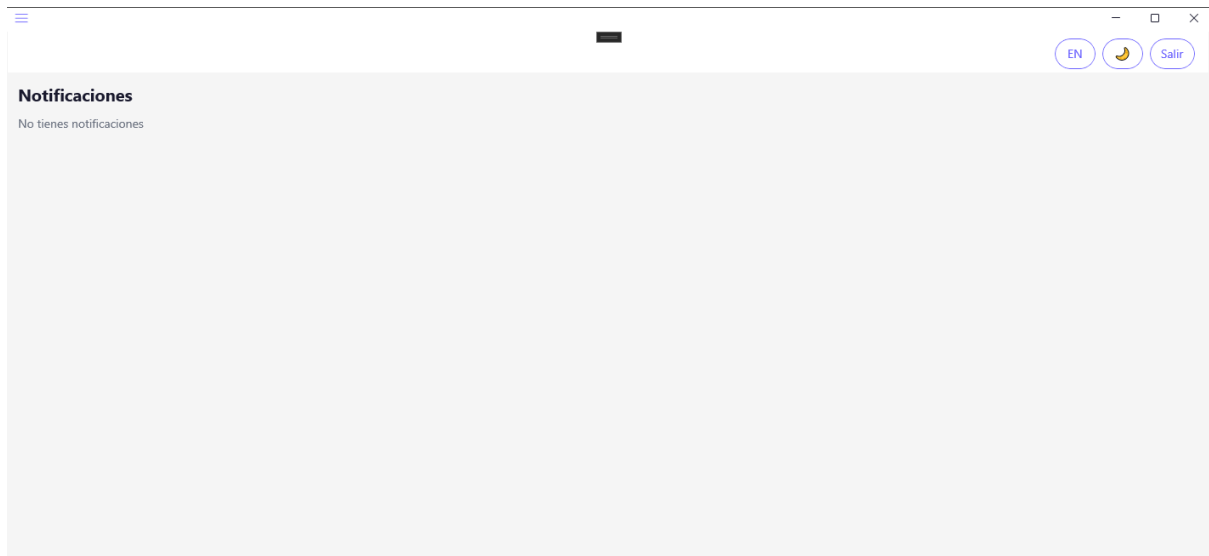
Ruta Shell: //PaginaNotificaciones.

Estructura de layout

Contenedor raíz: Grid (RowDefinitions=Auto,*, Padding=16).

- Fila 0 — Grid (ColumnDefinitions=*,Auto) con Label "Notificaciones" y Button "Marcar todas como leídas" (visible solo si HayNoLeidas=True, comando ComandoMarcarTodasLeidas).
- Fila 1 — RefreshView → CollectionView. Cada notificación tiene: badge de tipo con ColorTipoVisual, texto del mensaje y fecha. Las notificaciones no leídas se distinguen

visualmente del borde en ColorPrimario mediante un DataTrigger sobre la propiedad Leida.



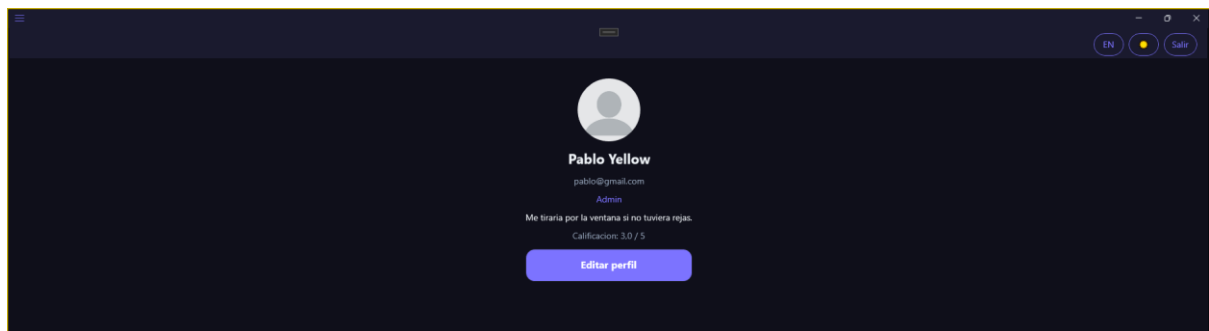
8.3.11. PaginaPerfil

Ruta Shell: //PaginaPerfil.

Estructura de layout

Contenedor raíz: ScrollView (Padding=16, MaximumWidthRequest=480).

- Image de perfil circular (EllipseGeometry Radio=50).
- Label de error y ActivityIndicator.
- Vista normal (visible si ModoEdicion=False): nombre, correo, rol, descripción, texto de calificación media y Button "Editar perfil" (ComandoActivarEdicion).
- Vista de edición (visible si ModoEdicion=True): Entry para nombre, Editor para descripción, HorizontalStackLayout con botones "Guardar" (ComandoGuardar) y "Cancelar" (ComandoCancelar).



8.3.12. PaginalInformes

Ruta Shell: //PaginalInformes.

Estructura de layout

Contenedor raíz: RefreshView → ScrollView → VerticalStackLayout (Padding=20).

- Labels de título y subtítulo.
- Label de error.
- Border (tarjeta gráfico): Label "Distribución por estado" + SfCircularChart (Syncfusion) con DoughnutSeries (serieFavores). Solo visible si HayDatos=True.
- Border (tarjeta resumen): Grid (2 filas × 2 columnas) con 4 badges de color fijo: Disponible (#10B981), Aceptado (#F59E0B), Finalizado (#6366F1) y Cancelado (#EF4444), cada uno con su contador numérico.
- ActivityIndicator.

Lógica destacada

- Lanza cuatro peticiones en secuencia para obtener la cuenta de favores por estado.
- Rellena PaletteBrushes de serieFavores con los cuatro colores en el mismo orden que los datos para que el gráfico de dona muestre el color correcto en cada porción.
- SinDatos y HayDatos son propiedades calculadas que controlan la visibilidad del gráfico y del mensaje vacío.



8.3.13. PaginaAdministracion

Ruta Shell: //PaginaAdministracion. Solo visible en el menú Flyout si SesionApp.Instancia.EsAdmin = True.

Estructura de layout

Contenedor raíz: Grid (RowDefinitions=Auto,*,Auto, Padding=16).

- Fila 0 — Picker con tres opciones: Usuarios, Favores, Grupos. Al cambiar la selección dispara OnCategoriaChanged que actualiza CategoriaSeleccionada y recarga los datos.
- Fila 1 — RefreshView → CollectionView. Cada elemento muestra el texto descriptivo del ítem y un Button "Eliminar" (rojo) cuyo CommandParameter es el ítem completo.

- Fila 2 — ActivityIndicator mientras IsCargando=True.

Lógica destacada (AdministracionVistaModelo)

- Los ítems se encapsulan en ItemWrapper (clase privada) que unifica el identificador y el texto de visualización independientemente de si el objeto subyacente es un usuario, un favor o un grupo.
- Antes de eliminar, muestra un DisplayAlert de confirmación. Si el usuario confirma, llama al endpoint de administración correspondiente y recarga la lista.

El cambio de rol de usuario se realiza mediante PATCH /admin/usuarios/{correo}/rol.



8.3.14. Mini Aplicación Móvil (Kotlin / Android)

La aplicación móvil está desarrollada en Kotlin, el lenguaje oficial de Android, utilizando Android Studio como entorno de desarrollo y View System con ViewBinding para los layouts XML. La arquitectura interna sigue el patrón MVVM (ViewModel + LiveData + Repository), paralelo al del cliente de escritorio. La comunicación con el backend se realiza mediante Retrofit 2 con un interceptor de OkHttp que inyecta automáticamente el token JWT en la cabecera Authorization: Bearer de cada petición. El token se persiste de forma segura usando EncryptedSharedPreferences con cifrado AES-256. La geolocalización utiliza FusedLocationProviderClient (Google Play Services) y la geocodificación inversa usa el Geocoder nativo de Android.

La aplicación usa dos Activities y dos NavGraphs separados. MainActivity carga nav_graph_auth.xml (Login y Registro) y, tras el login exitoso, lanza ActividadInicio que carga nav_graph.xml con el BottomNavigationView de cuatro pestañas. El cambio se realiza con FLAG_ACTIVITY_NEW_TASK or FLAG_ACTIVITY_CLEAR_TASK para destruir la pila de autenticación.

8.3.15. MainActivity + FragmentoLogin

Actividad de entrada. Al arrancar, MainActivity.onCreate() llama a PreferenciasSesion.cargarSesion(). Si existe un token guardado en EncryptedSharedPreferences, asigna los datos a SesionApp y lanza directamente ActividadInicio, saltándose el login. Si no hay sesión, infla su propio layout con el NavHostFragment de autenticación.

Estructura de layout — fragmento_login.xml

Contenedor raíz: ScrollView con fillViewport=true.

- LinearLayout (vertical, padding=24dp):
 - TextView con el nombre de la app (Favsy), tamaño 32sp, color colorPrimary, centrado.
 - TextInputLayout + TextInputEditText (editCorreo) — inputType emailAddress.
 - TextInputLayout + TextInputEditText (editContrasena) — inputType textPassword.
 - ProgressBar (progressBar) — visible mientras EstaOcupado.
 - MaterialButton (btnConectar) — llama a intentarLogin().
 - TextView (textRegistro) — enlace a registro, color colorPrimary.

Lógica destacada (FragmentoLogin)

- intentarLogin(): valida que correo y contraseña no estén en blanco → llama VistaModeloAutenticacion.login().

- Observa viewModel.login: en Cargando muestra ProgressBar y desactiva el botón; en Exito llama (activity as? MainActivity)?.irAHome(); en Error muestra Snackbar con el mensaje de la API.
- irAHome() lanza ActividadInicio con FLAG_ACTIVITY_NEW_TASK or FLAG_ACTIVITY_CLEAR_TASK y llama finish() para destruir MainActivity.



8.3.16. FragmentoRegistro

Navegación: action_loginFragment_to_registroFragment desde Login.

Estructura de layout — fragmento_registro.xml

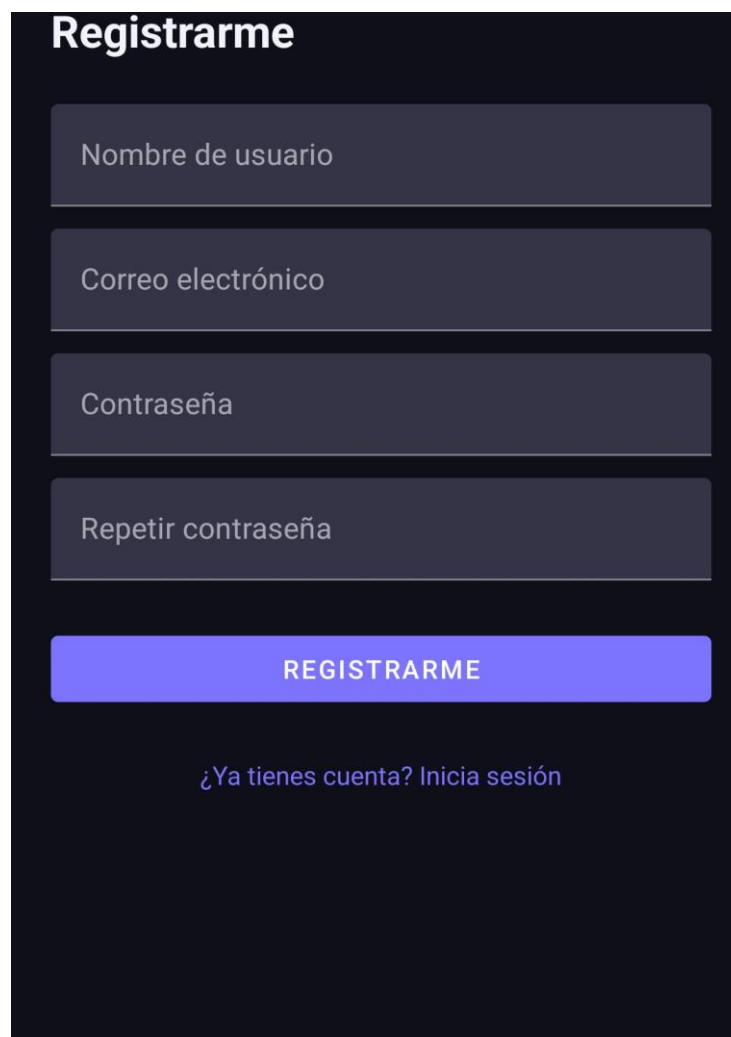
Contenedor raíz: ScrollView.

- LinearLayout (vertical, padding=24dp):
 - TextView "Registrarme" (24sp, negrita).
 - TextInputLayout + TextInputEditText (editNombre) — nombre de usuario.
 - TextInputLayout + TextInputEditText (editCorreo) — inputType emailAddress.
 - TextInputLayout + TextInputEditText (editContrasena) — inputType textPassword.
 - TextInputLayout + TextInputEditText (editContrasenaRepetir) — inputType textPassword.
 - ProgressBar.

- MaterialButton (btnRegistrarme).
- TextView (textLogin) — enlace de vuelta al login.

Validaciones antes de registrar

- Todos los campos obligatorios deben estar rellenos.
- El correo debe contener @.
- La contraseña debe tener al menos 6 caracteres.
- Las dos contraseñas deben coincidir.
- Si el registro tiene éxito, muestra Snackbar de confirmación y navega atrás al login con `navigateUp()`.



The image shows a registration form with a dark background. At the top, the title 'Registrarme' is displayed in white. Below the title, there are four text input fields, each with a light gray placeholder text: 'Nombre de usuario', 'Correo electrónico', 'Contraseña', and 'Repetir contraseña'. Below these fields is a prominent blue button with the text 'REGISTRARME' in white. At the bottom of the form, there is a link in light blue text that reads '¿Ya tienes cuenta? Inicia sesión'.

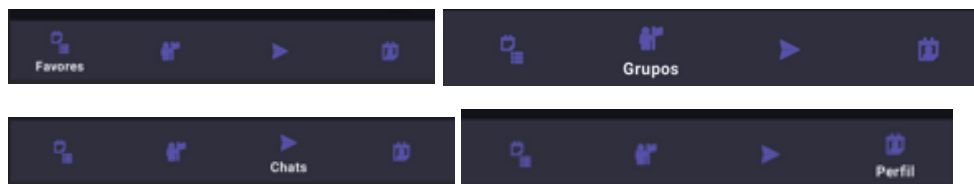
8.3.17. ActividadInicio + BottomNavigationView

Actividad principal tras el login. Infla `actividad_inicio.xml`, que contiene un `FragmentContainerView` (`nav_host_home`) vinculado a `nav_graph.xml` y un `BottomNavigationView` (`bottom_navigation`) con cuatro pestañas configuradas en

menu_bottom_nav.xml. Llama a `setupWithNavController()` para sincronizar la barra de navegación con el `NavController` automáticamente.

ID del destino	Fragmento	Etiqueta
favoresDisponiblesFragment	FragmentoFavoresDisponibles	Favores
misGruposFragment	FragmentoMisGrupos	Grupos
misChatsFragment	FragmentoMisChats	Chats
perfilFragment	FragmentoPerfil	Perfil

El método `cerrarSesionYVolverALogin()` de `ActividadInicio` lanza `MainActivity` con `FLAG_ACTIVITY_NEW_TASK` or `FLAG_ACTIVITY_CLEAR_TASK` y llama `finish()`, destruyendo la pila completa de `ActividadInicio`.



8.3.18. FragmentoFavoresDisponibles

Destino de inicio del `NavGraph` principal.

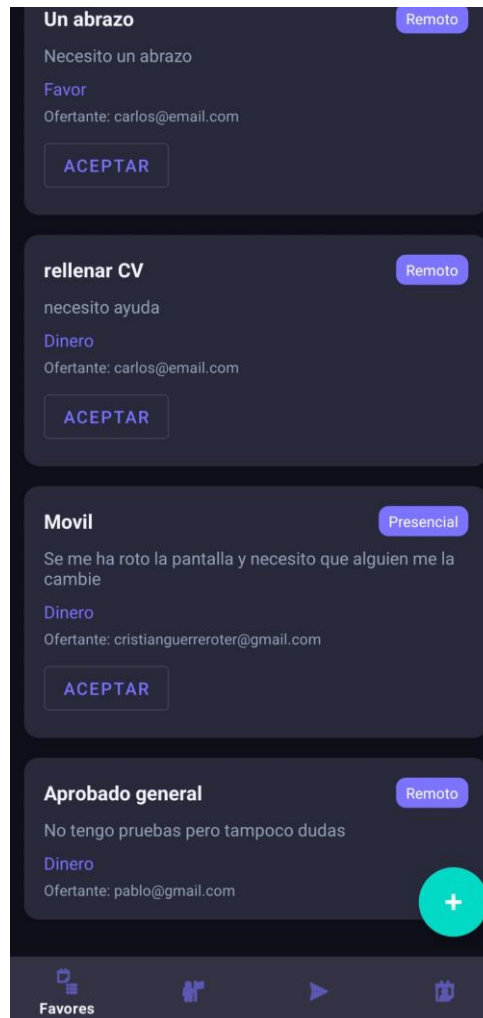
Estructura de layout — `fragmento_favores_disponibles.xml`

Contenedor raíz: `CoordinatorLayout`.

- `SwipeRefreshLayout` (`swipeRefresh`) — al deslizar hacia abajo llama `viewModel.cargarFavores()`.
 - `FrameLayout`:
 - `RecyclerView` (`recyclerFavores`) — lista de favores con `AdaptadorFavor`.
 - `ProgressBar` — centrado, visible mientras Cargando.
 - `TextView` (`textVacio`) — mensaje "No hay favores disponibles", visible si la lista está vacía.
- `FloatingActionButton` (`fabCrear`) — en la esquina inferior derecha, navega a `crearFavorFragment`.

Lógica destacada (`FragmentoFavoresDisponibles`)

- onViewCreated: configura el AdaptadorFavor con onAceptar = { favor -> viewModel.aceptarFavor(favor.id) }, conecta el SwipeRefreshLayout y el FAB, y observa viewModel.favores y viewModel.aceptar.
- Al observar aceptar con Exito: muestra Snackbar de confirmación y recarga la lista.
- onResume no recarga automáticamente; el refresco se hace mediante el SwipeRefreshLayout.



8.3.19. FragmentoCrearFavor

Navegación: action_favoresDisponiblesFragment_to_crearFavorFragment.

Estructura de layout — fragmento_crear_favor.xml

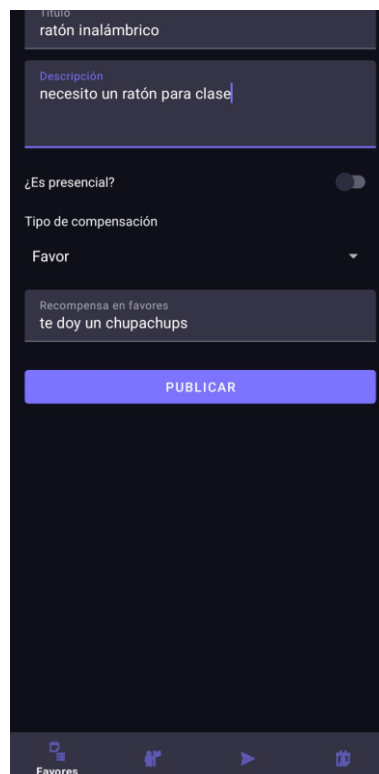
Contenedor raíz: ScrollView.

- LinearLayout (vertical, padding=16dp):
 - TextInputLayout + TextInputEditText (editTitulo).
 - TextInputLayout + TextInputEditText (editDescripcion) — minLines=3.
 - SwitchMaterial (switchPresencial) — al activarse muestra layoutUbicacion.

- LinearLayout (layoutUbicacion, visibility=gone inicialmente):
 - TextInputLayout + TextInputEditText (editUbicacion).
 - MaterialButton (btnUsarUbicacion) — estilo outlined.
- TextView "Tipo de compensación" + Spinner (spinnerCompensacion): opciones Favor / Dinero / Ambos.
- TextInputLayout (layoutCantidad, visibility=gone) + editCantidad (inputType numberDecimal).
- TextInputLayout (layoutRecompensa, visibility=gone) + editRecompensa.
- ProgressBar.
- MaterialButton (btnPublicar).

Lógica de geolocalización (AyudanteUbicacion)

Al pulsar btnUsarUbicacion: comprueba si tiene permiso ACCESS_FINE_LOCATION con tienePermisoUbicacion(). Si no, lanza el launcher permisoUbicacion (ActivityResultContracts.RequestPermission). Si el permiso es concedido, llama ayudanteUbicacion.obtenerDireccionActual() en una corrutina. Este método usa FusedLocationProviderClient.lastLocation de forma asíncrona con suspendCancellableCoroutine y convierte las coordenadas a dirección legible mediante Geocoder.getFromLocation(). La dirección se formatea como "Calle Número, Ciudad, PaísCode" y se rellena automáticamente en editUbicacion. Si falla, muestra Snackbar de error y el usuario introduce la dirección manualmente.



8.3.20. FragmentoMisGrupos

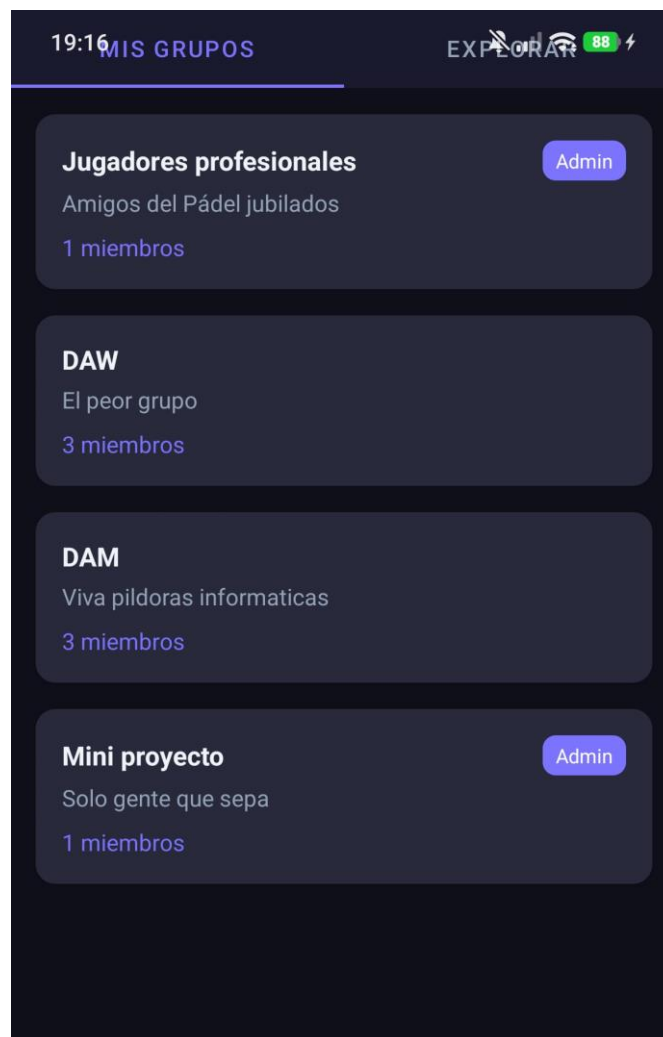
Fragmento contenedor con TabLayout + ViewPager2 de dos pestañas gestionadas por AdaptadorGruposPager.

Estructura de layout — fragmento_mis_grupos.xml

Contenedor raíz: CoordinatorLayout.

- LinearLayout (vertical):
 - TabLayout (tabLayout) — fondo colorSurface, indicador y texto seleccionado en colorPrimary.
 - ViewPager2 (viewPager) — ocupa el resto del espacio (layout_weight=1).
- FloatingActionButton (fabCrear) — navega a crearGrupoFragment.

El TabLayoutMediator asigna las etiquetas "Mis grupos" y "Explorar" a cada pestaña. El VistaModeloGrupo se comparte entre el fragmento contenedor y los dos fragmentos internos mediante viewModels({ requireParentFragment() })), de modo que el estado de búsqueda y el de la lista propia persisten al cambiar de pestaña.



8.3.21. FragmentoExplorarGrupos

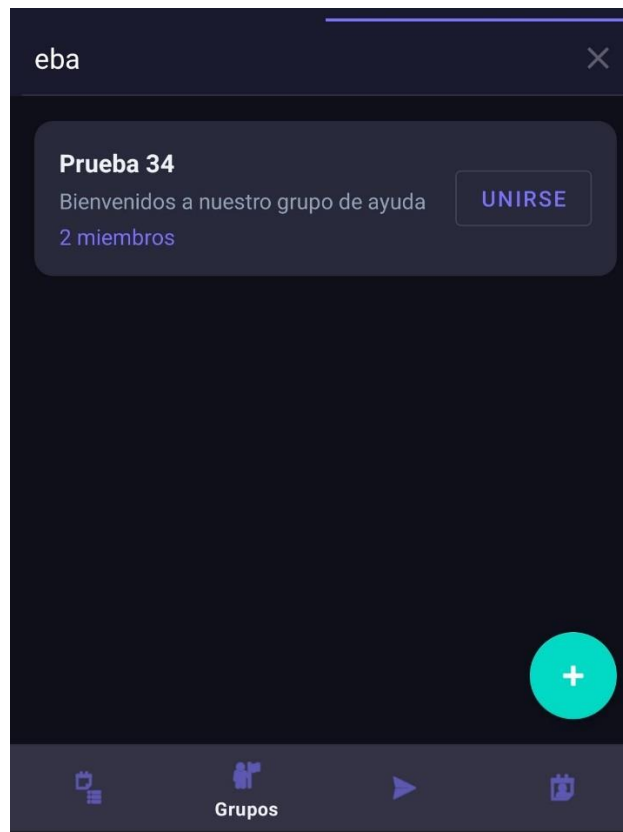
Estructura de layout — fragmento_explorar_grupos.xml

Contenedor raíz: LinearLayout vertical.

- SearchView (searchView) — fondo colorSurface.
- FrameLayout:
 - RecyclerView (recyclerBusqueda) — AdaptadorGrupoBusqueda.
 - ProgressBar.
 - TextView (textVacio) — "No se encontraron grupos".

Lógica de búsqueda

- onQueryTextChange: si newText.length >= 3 llama viewModel.buscarGrupos(newText.trim()).
- onQueryTextSubmit: llama viewModel.buscarGrupos(query.trim()).
- VistaModeloGrupo.buscarGrupos() valida que el término tenga al menos 3 caracteres antes de llamar al repositorio; si no, emite Exit(emptyList()).
- Al pulsar "Unirse" en una tarjeta: llama viewModel.unirseGrupo(grupo.id). Si tiene éxito, muestra Snackbar y recarga misGrupos para reflejar el nuevo grupo.



8.3.22. FragmentoCrearGrupo

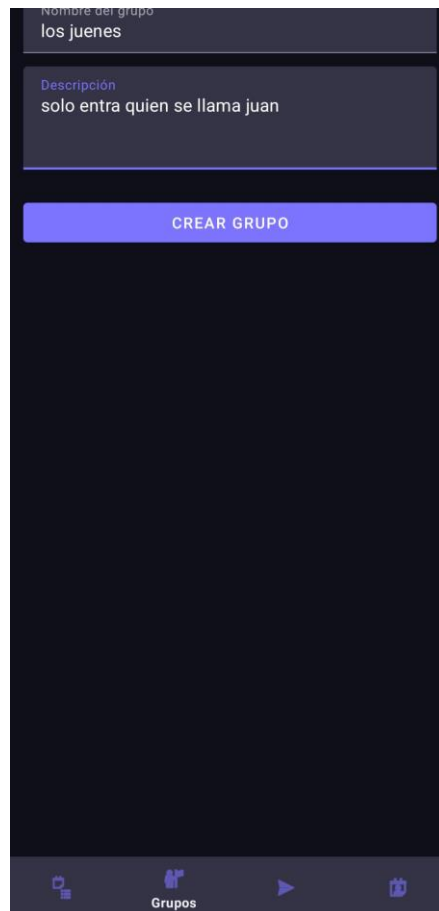
Navegación: `action_misGruposFragment_to_crearGrupoFragment`.

Estructura de layout — `fragmento_crear_grupo.xml`

Contenedor raíz: `ScrollView`.

- `LinearLayout` (vertical, `padding=16dp`):
 - `TextInputLayout` + `editNombre` (mínimo 3 caracteres).
 - `TextInputLayout` + `editDescripcion` (opcional, `minLines=3`).
 - `ProgressBar`.
 - `MaterialButton` (`btnCrear`).

Al crear con éxito: muestra `Snackbar` "Grupo creado correctamente" y navega atrás con `onBackPressedDispatcher.onBackPressed()`.



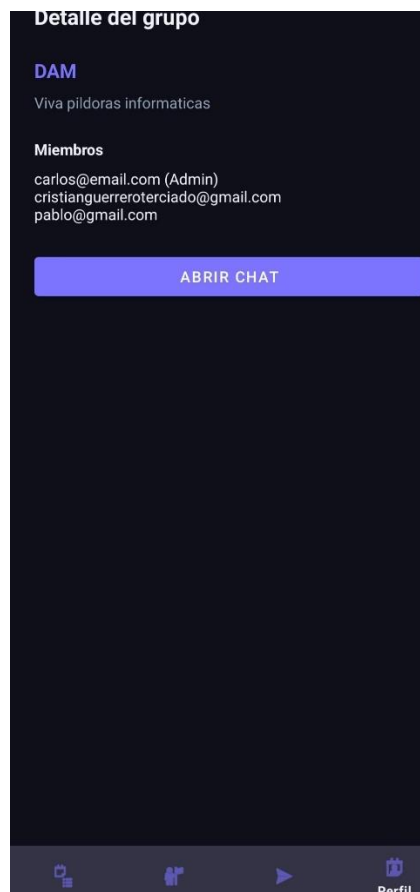
8.3.23. FragmentoDetalleGrupo

Navegación: `action_misGruposFragment_to_detalleGrupoFragment` con argumentos `Bundle`.

Contenedor raíz: `ScrollView`.

- LinearLayout (vertical, padding=24dp):
 - TextView "Detalle del grupo" (20sp, negrita).
 - TextView (textNombre) — color colorPrimary, 18sp.
 - TextView (textDescripcion) — color colorSecondaryText.
 - TextView "Miembros" (negrita).
 - TextView (textMiembros) — lista de correos con sufijo " (Admin)" para los administradores, generada uniendo los elementos del JSON con saltos de línea.
 - MaterialButton (btnAbrirChat) — navega a chatDetalleFragment pasando la lista de correos de todos los miembros como participantesJson.

Los datos se obtienen directamente de los argumentos del Bundle (no hay llamada a la API en este fragmento). El JSON de miembros se deserializa con `Gson().fromJson<List<MiembroGrupo>>()`.



8.3.24. FragmentoMisChats

Estructura de layout — `fragmento_mis_chats.xml`

Contenedor raíz: `FrameLayout`.

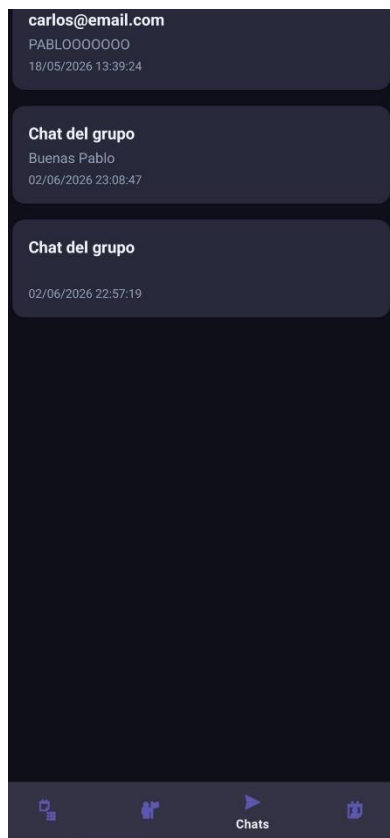
- RecyclerView (recyclerChats) — `AdaptadorChat`.

- ProgressBar.
- TextView (textVacio) — "No tienes conversaciones".

El fragmento observa dos LiveData en paralelo: viewModelChat.chats y viewModelGrupo.misGrupos. Cuando ambos tienen datos, ejecuta combinarYMostrar(): recorre los chats de tipo "grupo" y busca el grupo cuya lista de correos de miembros (ordenada) coincida exactamente con la lista de participantes del chat (también ordenada). Si hay coincidencia, asigna chat.nombreGrupo = grupo.nombreGrupo; si no, usa "Chat del grupo". Esto permite que el adaptador muestre el nombre real del grupo en lugar del ID.

Cada tarjeta de chat (elemento_chat.xml) muestra: nombre del interlocutor (si es privado, el correo del otro participante; si es de grupo, chat.nombreGrupo), último mensaje y fecha. Al pulsar, navega a chatDetalleFragment pasando chatId y participantesJson="".

onResume recarga chats y grupos para actualizar la lista al volver desde el detalle.



8.3.25. FragmentoChatDetalle

Navegación: desde misChatsFragment o desde detalleGrupoFragment.

Estructura de layout — fragmento_chat_detalle.xml

Contenedor raíz: LinearLayout vertical.

- FrameLayout (weight=1):

- RecyclerView (recyclerMensajes) — AdaptadorMensaje, LinearLayoutManager con stackFromEnd=true para mostrar el último mensaje al fondo.
- ProgressBar.
- Barra de entrada: LinearLayout horizontal con fondo colorSurface:
 - EditText (editMensaje) — inputType textCapSentences|textMultiLine, maxLines=4.
 - MaterialButton (btnEnviar).

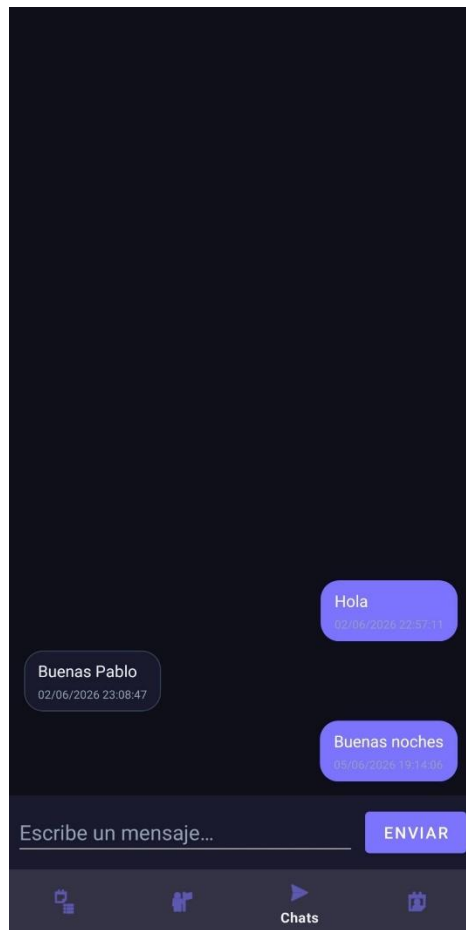
Lógica destacada

- onViewCreated: parsea participantesJson con Gson. Si hay participantes (viene de detalle de grupo o de un favor), llama viewModel.cargarChat(chatId="", participantes) que llama a repositorio.obtenerOCrear(participantes). Si participantesJson está vacío (viene de lista de chats), llama viewModel.cargarChat(chatId, null) que busca el chat en la lista de todos los chats del usuario.
- Tras cargar el chat con éxito, guarda chatIdActual del resultado y llama repositorio.marcarLeido().
- Al enviar: comprueba que contenido no está en blanco ni chatIdActual vacío → llama viewModel.enviarMensaje(). Si tiene éxito, recarga el chat completo para mostrar el nuevo mensaje y hace scroll al último elemento.

AdaptadorMensaje — burbujas de chat

Cada mensaje se renderiza en elemento_mensaje.xml (un FrameLayout con un LinearLayout burbuja dentro). El adaptador compara mensaje.emisor con SesionApp.correo para determinar si es propio:

- **Mensaje propio:** Gravity.END, fondo bubble_own.xml (color colorPrimary), texto blanco (colorOnPrimary).
- **Mensaje ajeno:** Gravity.START, fondo bubble_other.xml (fondo colorSurface con borde bubbleOtherBorder), texto colorOnSurface.



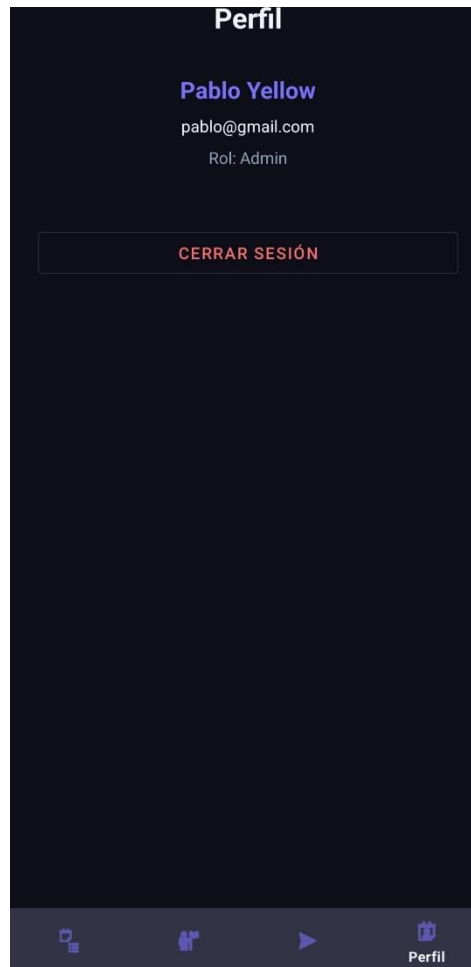
8.3.26. FragmentoPerfil

Estructura de layout — fragmento_perfil.xml

Contenedor raíz: LinearLayout vertical, centrado horizontalmente, padding=24dp.

- TextView "Perfil" (24sp, negrita).
- TextView (textNombre) — SesionApp.nombre si no está en blanco, sino SesionApp.correo. Color colorPrimary, 20sp.
- TextView (textCorreo) — SesionApp.correo.
- TextView (textRol) — "Rol: " + SesionApp.rol, color colorSecondaryText.
- MaterialButton (btnCerrarSesion) — estilo outlined, texto rojo (colorError).

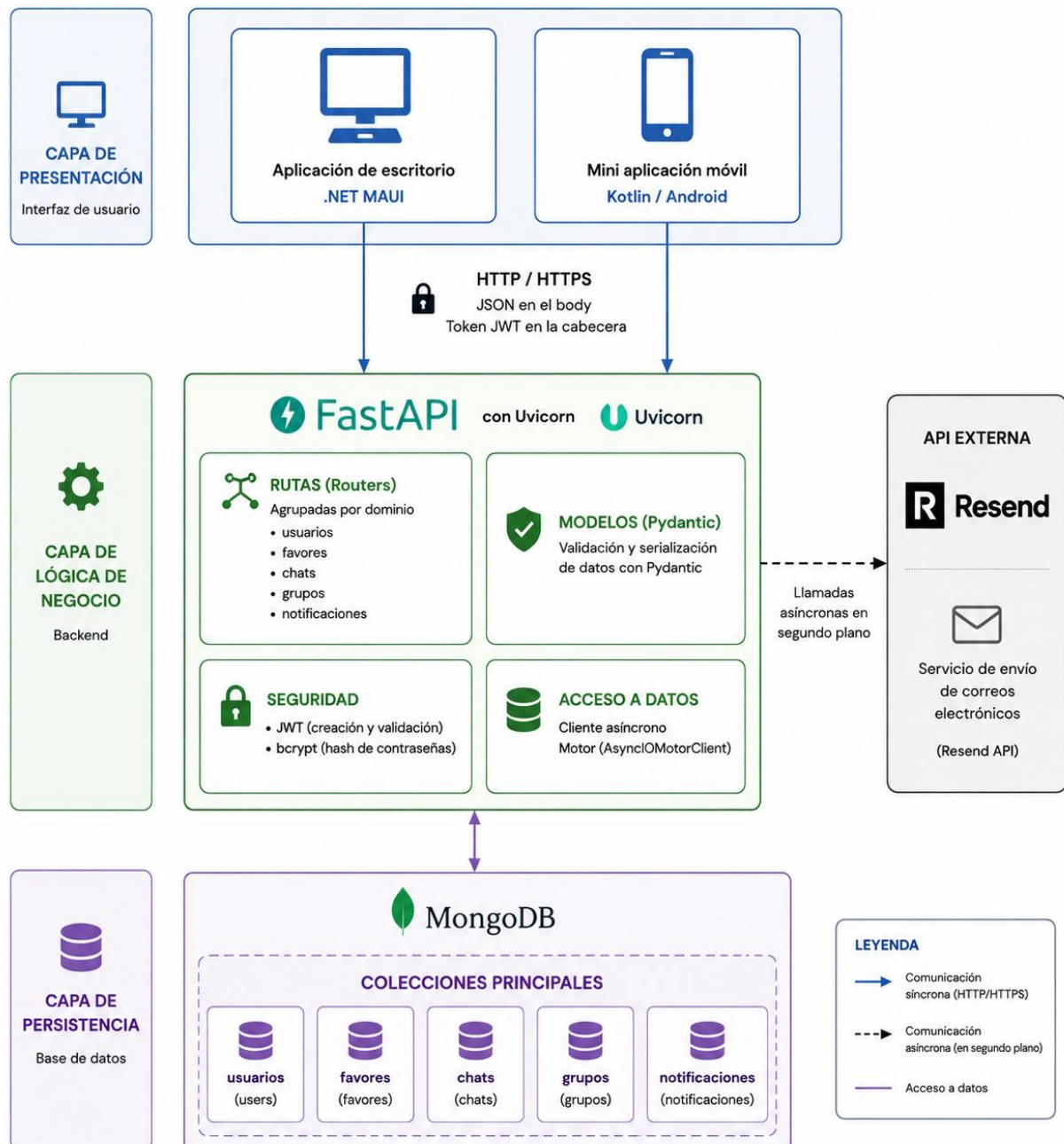
Al pulsar "Cerrar sesión": llama authViewModel.cerrarSesion() (que limpia EncryptedSharedPreferences y borra SesionApp) → llama (activity as? ActividadInicio)?.cerrarSesionYVolverALogin().



9. Desarrollo

9.1.Arquitectura del Sistema

El sistema sigue una arquitectura de tres capas: presentación (clientes MAUI), lógica de negocio (FastAPI backend) y persistencia (MongoDB). La comunicación entre capas se realiza mediante HTTP/HTTPS con payloads JSON. La autenticación se gestiona mediante tokens JWT firmados con el algoritmo HS256.



El diagrama muestra tres capas verticales. En la parte superior, la capa de presentación agrupa la aplicación de escritorio (.NET MAUI) y la mini aplicación móvil (Kotlin/Android), ambas comunicadas con la capa intermedia mediante HTTP/HTTPS con payloads JSON y token JWT en la cabecera de cada petición. En el centro, la capa de lógica de negocio contiene el servidor FastAPI con Uvicorn, que organiza su código en cuatro módulos: las rutas agrupadas por dominio (usuarios, favores, chats, grupos, notificaciones), los modelos Pydantic para validación, las utilidades de seguridad (JWT y bcrypt) y el cliente asíncrono Motor. En la parte inferior, la capa de persistencia contiene MongoDB con sus cinco colecciones principales. A la derecha del backend se representa la API externa de Resend, a la que el servidor llama en segundo plano para el envío de notificaciones por correo electrónico.

Capa	Tecnología
Cliente de escritorio	.NET MAUI (C#, XAML), patrón MVVM
Mini app móvil	kotlin, patrón MVVM
Backend API	Python 3.11, FastAPI, Uvicorn
Autenticación	JWT (python-jose), bcrypt (passlib)
Base de datos	MongoDB 7.x, Motor (cliente asíncrono)
Validación de datos	Pydantic v2
Control de versiones	Git / GitHub

9.2.Estructura del Backend

El backend sigue una estructura modular organizada en cuatro bloques principales:

- Configuración (Configuracion/): parámetros del entorno (.env) y conexión a MongoDB.
- Modelos (Modelos/): clases Pydantic que definen la estructura de los datos y aplican validaciones automáticas.
- Rutas (Rutas/): routers FastAPI que exponen los endpoints de la API agrupados por dominio.
- Utilidades (Utilidades/): funciones de apoyo como formateo de fechas y verificación de tokens JWT.

La conexión a MongoDB se establece en bbdd.py mediante el cliente asíncrono Motor, que es compatible con el modelo async/await de FastAPI:

```
cliente = AsyncIOMotorClient(MONGO_URL)
```

```
db = cliente[NOMBRE_BASE_DATOS]
```

```
coleccion_usuarios = db.usuarios
```

```
coleccion_favores = db.favores
```

```
coleccion_chats = db.chats
```

```
coleccion_grupos = db.grupos
```

```
coleccion_notificaciones = db.notificaciones
```

9.3.Endpoints Principales de la API

A continuación se detallan los endpoints más relevantes agrupados por dominio:

9.3.1. Autenticación y Usuarios

Método + Ruta	Descripción
POST /usuarios/registro	Registra un nuevo usuario. Hashea la contraseña con bcrypt.
POST /usuarios/login	Autentica al usuario y devuelve un token JWT.
POST /usuarios/{correo}/calificar	Añade una valoración y recalcula la media.
GET /perfil/mis-datos	Devuelve el perfil del usuario autenticado.
PUT /perfil/editar	Actualiza los datos del perfil del usuario autenticado.

El endpoint de registro valida mediante Pydantic que el correo tenga formato válido y que la contraseña tenga al menos seis caracteres. Antes de insertar el documento en MongoDB comprueba que no exista ya un usuario con ese correo, devolviendo un error 400 en caso contrario. La contraseña nunca se almacena en texto plano: se transforma mediante el algoritmo bcrypt, que aplica un factor de coste computacional deliberadamente alto para dificultar ataques de fuerza bruta. El endpoint de login sigue el flujo estándar OAuth2 y devuelve un token JWT con tiempo de expiración configurable mediante variable de entorno. El endpoint de calificación recalcula la media aritmética del usuario cada vez que recibe una nueva puntuación, actualizando el campo calificacionMedia en el mismo documento del usuario.

9.3.2. Favores

Método + Ruta	Descripción
POST /favores/crear	Crea un nuevo favor. El propietario se extrae del token JWT.
GET /favores/lista	Lista favores con filtros opcionales de estado y modalidad.
PUT /favores/{id}/aceptar	Acepta un favor disponible. Notifica al propietario.
PUT /favores/{id}/cancelar	Cancela un favor aceptado. Notifica al otro usuario.
PUT /favores/{id}/finalizar	Marca el favor como finalizado. Notifica al otro usuario.
DELETE /favores/{id}/eliminar	Elimina el favor si el usuario es el propietario.

El ciclo de vida de un favor se gestiona exclusivamente a través de estos endpoints, que validan en el servidor tanto la existencia del recurso como los permisos del usuario que realiza la acción. El endpoint de creación extrae automáticamente la identidad del propietario del token JWT, de modo que el cliente nunca puede suplantar a otro usuario como ofertante. Los endpoints de aceptar, cancelar y finalizar generan automáticamente una notificación interna y lanzan un correo al otro participante como tarea en segundo plano, de forma que la respuesta al cliente no se retrasa por el tiempo de envío del correo.

9.3.3. Chats

Método + Ruta	Descripción
POST /chat/obtener-o-crear_chat	Devuelve el chat existente o crea uno nuevo entre los participantes.
POST /chat/{id_chat}/enviar	Envía un mensaje al chat indicado.
PUT /chat/{id_chat}/leer-chats	Marca como leídos todos los mensajes del chat no enviados por el usuario.

El endpoint de obtención o creación de chat privado utiliza un identificador determinista formado por los dos correos ordenados alfabéticamente, lo que garantiza que dos usuarios siempre accedan al mismo documento de chat independientemente de quién inicie la conversación. Para los chats grupales se utiliza el ObjectId generado por MongoDB. El marcado de mensajes como leídos actualiza únicamente los mensajes cuyo emisor sea distinto al usuario autenticado, evitando que un usuario marque sus propios mensajes como leídos.

9.3.4. Grupos

Método + Ruta	Descripción
GET /grupos/buscar-grupos	Busca grupos por nombre (búsqueda parcial, insensible a mayúsculas).
POST /grupos/crear	Crea un grupo. El creador queda como administrador.
GET /grupos/mis-grupos	Lista los grupos a los que pertenece el usuario autenticado.
POST /grupos/{id}/unirse	Une al usuario al grupo si no era miembro.
POST /grupos/{id}/pedir-invitation	Envía una solicitud de unión al grupo.
POST /grupos/{id}/aceptar-solicitud	El admin acepta una solicitud pendiente.
POST /grupos/{id}/rechazar-solicitud	El admin rechaza una solicitud pendiente.
POST /grupos/{id}/expulsar	El admin expulsa a un miembro del grupo.
PATCH /grupos/{id}/cambiar-rol	El admin cambia el rol de un miembro.
PATCH /grupos/{id}/editar	El admin edita el nombre, descripción o foto del grupo.
POST /grupos/{id}/salir	El usuario sale del grupo. Si era el único miembro, el grupo se elimina.

Los endpoints de administración de grupos verifican antes de ejecutar cualquier operación que el usuario autenticado tenga el rol de administrador dentro del grupo concreto. Esta validación se realiza en el propio filtro de la consulta a MongoDB, de modo que si el usuario no es administrador la consulta no devuelve resultado y se lanza un error 403. El endpoint de salida del grupo incluye una lógica especial: si el usuario que sale es el único miembro, el grupo se elimina automáticamente; si es el único administrador pero quedan otros miembros, la operación se rechaza y se obliga a nombrar un nuevo administrador antes de poder salir.

9.4. Estructura del Cliente de Escritorio (.NET MAUI)

El cliente de escritorio sigue el patrón MVVM y está organizado en las siguientes carpetas:

Carpeta	Contenido
Modelos/	Clases que mapean directamente la respuesta JSON de la API (Usuario, Favor, Chat, Mensaje, Grupo, MiembroGrupo, SolicitudInvitation, Notificacion).

Carpeta	Contenido
Servicios/	Capa de comunicación HTTP. ServicioApi.cs actúa como clase base, centraliza la configuración del HttpClient y gestiona la aplicación del token JWT en la cabecera de todas las peticiones. De ella heredan ServicioAutenticacion, ServicioChat, ServicioFavor, ServicioGrupo, ServicioNotificacion y ServicioPerfil, cada uno responsable de encapsular las llamadas a su dominio correspondiente.
VistaModelos/	Contiene la lógica de cada pantalla: comandos, propiedades observables y binding. Todos heredan de VistaModeloBase, que implementa INotifyPropertyChanged y expone las propiedades comunes EstaOcupado, MensajeError y HayError, así como el método Notificar para disparar el mecanismo de binding.
Vistas/	Archivos XAML con la interfaz gráfica y su code-behind. Cada vista está vinculada a su VistaModelo mediante BindingContext y se resuelve mediante inyección de dependencias en el contenedor de MAUI.
Controles/	Componentes visuales reutilizables, como EntryLimpio, que elimina los bordes y fondos nativos del control de texto en Windows para lograr una apariencia uniforme.
Utilidades/	Contiene GestorTema, GestorIdioma y GestorRecursos, responsables de intercambiar dinámicamente los diccionarios de recursos de tema e idioma sin reiniciar la aplicación, así como InvertirBoolConverter para transformaciones de binding en XAML.
Conversores/	Conversores XAML para transformar valores en la vista (ej. BoolToColorConverter para indicar mensajes leídos).
Singleton/	SesionApp almacena los datos del usuario autenticado durante la sesión activa. ConfiguracionApp persiste las preferencias de tema e idioma entre sesiones usando la API Preferences de .NET MAUI.

La inyección de dependencias se configura en MauiProgram.cs, donde todos los servicios se registran como Singleton (una única instancia compartida por toda la aplicación) y todos los ViewModels y vistas se registran como Transient (una nueva instancia en cada navegación). Esto garantiza que el estado HTTP, incluido el token JWT, sea consistente en toda la aplicación, mientras que cada pantalla parte de un estado limpio al ser creada.

El sistema de temas claro y oscuro y el sistema de internacionalización en español e inglés funcionan intercambiando los diccionarios de recursos activos en tiempo de ejecución. Cuando el usuario cambia de tema o de idioma desde los botones de la barra de navegación, los nuevos colores y cadenas de texto se aplican inmediatamente a todas las pantallas visibles sin necesidad de reiniciar la aplicación ni recargar ninguna página.

El token JWT obtenido en el inicio de sesión se almacena de forma segura mediante SecureStorage, que en Windows utiliza la API DPAPI del sistema operativo para cifrar el valor. Al arrancar la aplicación, si existe un token almacenado, se recupera y se aplica automáticamente en todos los servicios HTTP, restaurando la sesión del usuario sin necesidad de volver a introducir las credenciales.

El flujo de navegación tras el login desactiva la barra de pestañas de autenticación, muestra el menú lateral Flyout con todas las secciones de la aplicación y navega automáticamente a la pantalla de inicio. Al cerrar sesión, el proceso inverso limpia el token de SecureStorage, desactiva el Flyout, reactiva la barra de pestañas y devuelve al usuario a la pantalla de login.

9.4.1. Modelos del cliente escritorio

Los modelos de la carpeta Modelos/ mapean directamente las respuestas JSON de la API usando [JsonPropertyName]. Además, cada modelo expone propiedades calculadas ([JsonIgnore]) para su uso en XAML:

Clases	Campos principales	Propiedades calculadas
UsuarioPublico	Id, Correo, NombreUsuario, Descripcion, FotoPerfil, CalificacionMedia, Rol	
UsuarioCreacion	Correo, Contraseña, NombreUsuario, Descripcion, FotoPerfil	
RespuestaLogin	AccessToken, TokenType, Usuario (UsuarioResumen)	
FavorPublico	Id, Titulo, Descripcion, Modalidad, TipoCompensacion, CantidadDinero, RecompensaFavor, Ubicacion, Estado, FechaCreacion, IdUsuarioOfrece, IdUsuarioSolicita	ResumenCompensacion, TextoModalidad, ColorEstado, ColorEstadoVisual, TextoEstado, MostrarEliminar, MostrarCancelarFinalizar
FavorCreacion	Titulo, Descripcion, Modalidad, Ubicacion, TipoCompensacion, CantidadDinero, RecompensaFavor	

Clases	Campos principales	Propiedades calculadas
GrupoPublico	Id, NombreGrupo, Descripcion, FotoGrupo, Miembros[], Solicitudes[], FechaCreacion	NumeroMiembros, EsAdmin, TextoRol
GrupoBusqueda	Id, NombreGrupo, Descripcion, FotoGrupo, NumeroMiembros	
MiembroGrupo	CorreoUsuario, Rol	
ChatPublico	Id, Tipo, Participantes[], Mensajes[], UltimaActividad	TituloMostrar, UltimoMensaje, NoLeidos, NombreGrupo, MiCorreo
MensajePublico	Emisor, Contenido, FechaEnvio, Leido	EsMio (compara Emisor con SesionApp.Instancia.Usuario.Correo)
Notificacion	Id, UsuarioDestino, Mensaje, Tipo, Leida, IdReferencia, Fecha	TextoTipo, ColorTipo, ColorTipoVisual

9.4.2. Métodos de los Servicios HTTP

Todos los servicios heredan de ServicioApi, que centraliza la URL base (<http://localhost:8000/>) y el método AplicarToken(string?) que establece la cabecera Authorization: Bearer en todas las peticiones. El método estático ValidarRespuestaAsync() lanza HttpRequestException con el campo detail del JSON de FastAPI si la respuesta no es 2xx.

Servicio	Método	Descripción
ServicioAutenticacion	LoginAsync(correo, contrasenia)	POST form-urlencoded a /usuarios/login. Persiste el token en SecureStorage.
	RegistrarAsync(usuario)	POST JSON a /usuarios/registro.
	ObtenerTokenAsync()	Lee el token de SecureStorage.
	CerrarSesionAsync()	Limpia SecureStorage y elimina la cabecera de autorización.
ServicioFavor	ObtenerFavoresDisponiblesAsync()	GET /favores/lista?estado=disponible.

Servicio	Método	Descripción
	ObtenerMisFavoresPublicadosAsync()	GET /favores/mis-publicados.
	ObtenerFavoresPorEstadoAsync(estado)	GET /favores/lista?estado={estado}.
	CrearFavorAsync(favor)	POST JSON a /favores/crear.
	AceptarFavorAsync(id)	PUT /favores/{id}/aceptar.
	CancelarFavorAsync(id)	PUT /favores/{id}/cancelar.
	FinalizarFavorAsync(id)	PUT /favores/{id}/finalizar.
	EliminarFavorAsync(id)	DELETE /favores/{id}/eliminar.
ServicioGrupo	ObtenerMisGruposAsync()	GET /grupos/mis-grupos.
	BuscarGruposAsync(nombre)	GET /grupos/buscar-grupos?nombre={nombre}.
	CrearGrupoAsync(nombre, desc)	POST JSON a /grupos/crear.
	UnirseGrupoAsync(id)	POST /grupos/{id}/unirse.
	SalirGrupoAsync(id)	POST /grupos/{id}/salir.
	ExpulsarMiembroAsync(id, correo)	POST /grupos/{id}/expulsar?usuario_a_expulsar={correo}.
	CambiarRolAsync(id, correo, rol)	PATCH /grupos/{id}/cambiar-rol?correo_miembro={correo}&nuevo_rol={rol}.
	EditarGrupoAsync(id, ...)	PATCH JSON a /grupos/{id}/editar.
	AceptarSolicitudAsync(id, correo)	POST /grupos/{id}/aceptar-solicitud?correo_aspirante={correo}.
	RechazarSolicitudAsync(id, correo)	POST /grupos/{id}/rechazar-solicitud?correo_aspirante={correo}.

Servicio	Método	Descripción
<u>ServicioChat</u>	ObtenerMisChatsAsync()	GET /chat/mis-chats.
	ObtenerOCrearChatPrivadoAsync(correo)	POST JSON a /chat/obtener-o-crear_chat.
	ObtenerOCrearChatGrupoAsync(participantes)	POST JSON con lista de participantes.
	EnviarMensajeAsync(idChat, contenido)	POST JSON a /chat/{id}/enviar.
	MarcarLeidoAsync(idChat)	PUT /chat/{id}/leer-chats.
ServicioNotificación	ObtenerNotificacionesAsync()	GET /notificaciones/mis-notificaciones.
	MarcarTodasLeidasAsync()	POST /notificaciones/marcar-leida.
ServicioPerfil	ObtenerMiPerfilAsync()	GET /perfil/mis-datos.
	EditarPerfilAsync(nombre, desc)	PUT JSON a /perfil/editar.
ServicioAdmin	ObtenerTodosUsuariosAsync()	GET /admin/usuarios.
	EliminarUsuarioAsync(correo)	DELETE /admin/usuarios/{correo}.
	CambiarRolUsuarioAsync(correo, rol)	PATCH JSON a /admin/usuarios/{correo}/rol.
	ObtenerTodosFavoresAsync()	GET /admin/favores.
	EliminarFavorAsync(id)	DELETE /admin/favores/{id}.
	ObtenerTodosGruposAsync()	GET /admin/grupos.
	EliminarGrupoAsync(id)	DELETE /admin/grupos/{id}.

9.5. Estructura del Cliente de móvil (Kotlin/Android)

Carpeta	Contenido
data/modelos/	Clases de datos (data classes) que mapean directamente la respuesta JSON de la API usando @SerializedName de Gson. Se organizan en subcarpetas por dominio: autenticacion/ (RespuestaLogin, UsuarioResumen, UsuarioCreacion, UsuarioPublico), chats/ (ChatPublico, MensajePublico), favores/ (FavorPublico, FavorCreacion), grupos/ (GrupoPublico, GrupoBusqueda, MiembroGrupo, SolicitudGrupo) y notificaciones/ (Notificacion).
data/servicios/	Capa de comunicación HTTP. ServicioApi.kt es la interfaz Retrofit que declara todos los endpoints de la API agrupados por dominio. ClienteRetrofit.kt actúa como objeto singleton que construye la instancia de Retrofit con un interceptor de autenticación (inyecta el token JWT en cada petición leyéndolo desde SesionApp.token) y un interceptor de logging para depuración. Detecta automáticamente si la app corre en emulador o en dispositivo físico para seleccionar la URL base correcta.
data/repositorios/	Capa intermedia entre los ViewModels y el servicio HTTP. Cada repositorio (RepositorioAutenticacion, RepositorioChat, RepositorioFavor, RepositorioGrupo) encapsula las llamadas a ServicioApi dentro de una función ejecutar { } que captura HttpException, IOException y cualquier excepción genérica, devolviendo siempre un ResultadoApi<T> sellado con los estados Exitó, Error y Cargando.
ui/vistamodelos/	Contiene la lógica de cada dominio: VistaModeloAutenticacion, VistaModeloFavor, VistaModeloGrupo y VistaModeloChat. Cada uno expone sus estados mediante MutableLiveData<ResultadoApi<T>> observables desde los fragmentos. Todos heredan de ViewModel (o AndroidViewModel cuando necesitan el contexto de la aplicación) y usan viewModelScope.launch para ejecutar corrutinas sin bloquear el hilo principal.
ui/vistas/	Fragmentos con la interfaz gráfica y su lógica de presentación, organizados por dominio: autenticacion/ (FragmentoLogin, FragmentoRegistro), favores/ (FragmentoFavoresDisponibles, FragmentoCrearFavor), grupos/ (FragmentoMisGrupos, FragmentoMisGruposTab, FragmentoExplorarGrupos, FragmentoCrearGrupo, FragmentoDetalleGrupo), chats/ (FragmentoMisChats, FragmentoChatDetalle) y perfil/ (FragmentoPerfil). Cada fragmento usa ViewBinding y observa el LiveData de su VistaModelo para actualizar la UI.

Carpeta	Contenido
ui/adaptadores/	Adaptadores para los RecyclerView de cada pantalla, todos implementando ListAdapter con DiffUtil.ItemCallback para actualizaciones eficientes de la lista sin redibujar elementos innecesarios: AdaptadorFavor, AdaptadorGrupo, AdaptadorGrupoBusqueda, AdaptadorChat, AdaptadorMensaje y AdaptadorGruposPager (para el ViewPager2 de grupos).
singleton/	SesionApp es un objeto Kotlin (singleton) que almacena en memoria el estado de la sesión activa: token, correo, nombre y rol. El método limpiar() resetea todos los campos al cerrar sesión. El interceptor de Retrofit lee SesionApp.token en cada petición para que el token se aplique de forma inmediata tras el login sin necesidad de recrear el cliente HTTP.
utilidades/	Clases de apoyo transversales. ResultadoApi<T> es una sealed class con tres estados (Exito, Error, Cargando) usada en todos los repositorios y ViewModels. PreferenciasSesion encapsula el acceso a EncryptedSharedPreferences (cifrado AES-256) para persistir y recuperar el token y los datos del usuario entre sesiones. AyudanteUbicacion gestiona la geolocalización con FusedLocationProviderClient y la geocodificación inversa con el Geocoder nativo de Android para convertir coordenadas en una dirección legible.
res/layout/	Un fichero XML por cada fragmento y por cada elemento de lista. Los layouts de fragmento usan MaterialCardView, TextInputLayout, MaterialButton, SwipeRefreshLayout, CoordinatorLayout y FloatingActionButton de Material Components. Los layouts de elemento de lista (elemento_favor.xml, elemento_grupo.xml, elemento_chat.xml, elemento_mensaje.xml, elemento_grupo_busqueda.xml) definen las tarjetas del RecyclerView.
res/navigation/	Dos grafos de navegación separados. nav_graph_auth.xml gestiona el flujo de autenticación (LoginFragment → RegistroFragment) y es el grafo de MainActivity. nav_graph.xml gestiona la navegación principal tras el login (Favores, Grupos, Chats, Perfil y pantallas de detalle) y es el grafo de ActividadInicio.
res/values/ y res/values-night/	Definen los temas claro y oscuro respectivamente. colors.xml contiene todos los tokens de color del sistema de diseño. themes.xml define Theme.AppFavoresMovil heredando de Theme.MaterialComponents.DayNight.NoActionBar. El sistema aplica automáticamente el tema correcto según el modo del sistema operativo gracias a AppCompatDelegate.MODE_NIGHT_FOLLOW_SYSTEM.

9.5.1. Modelos del cliente móvil

Todos los modelos usan @SerializedName de Gson para mapear los nombres de campo del JSON de la API. El campo _id de MongoDB se mapea a id en todos los modelos que lo contienen.

Clases	Campos serializados	Campos mutables extra
RespuestaLogin	accessToken, tokenType, usuario (UsuarioResumen)	
UsuarioResumen	nombre, correo, id, rol	
UsuarioCreacion	correo, contrasenia, nombreUsuario	
UsuarioPublico	id, correo, nombreUsuario, descripcion, fotoPerfil, calificacionMedia, rol	
FavorPublico	id, titulo, descripcion, modalidad, tipoCompensacion, cantidadDinero, recompensaFavor, ubicacion, estado, fechaCreacion, idUsuarioOfrece, idUsuarioSolicita	
FavorCreacion	titulo, descripcion, modalidad, ubicacion, tipoCompensacion, cantidadDinero, recompensaFavor	
GrupoPublico	id, nombreGrupo, descripcion, fotoGrupo, miembros, solicitudes, fechaCreacion	
GrupoBusqueda	id, nombreGrupo, descripcion, fotoGrupo, numeroMiembros	
MiembroGrupo	correoUsuario, rol	
SolicitudGrupo	correoUsuario, comentario, fechaSolicitud	
ChatPublico	id, tipo, participantes, mensajes, ultimaActividad	var nombreGrupo: String = "" (asignado en combinarYMostrar)
MensajePublico	emisor, contenido, fechaEnvio, leido	
Notificacion	id, tipo, mensaje, leida, fechaCreacion	

9.5.2. Repositorios y métodos de acceso a datos

Todos los repositorios envuelven las llamadas a ServicioApi en una función ejecutar { } que captura `HttpException`, `IOException` y `Exception` genéricas, devolviendo siempre un `ResultadoApi<T>` sellado (Exito, Error o Cargando). Los ViewModels emiten Cargando antes de lanzar la corrutina y actualizan el LiveData con el resultado.

Repositorio	Método	Llamada a la API
RepositorioAutenticacion	login(correo, contrasena)	POST form-urlencoded /usuarios/login. Persiste el token y datos en EncryptedSharedPreferences y SesionApp.
	registro(usuario)	POST JSON /usuarios/registro.
	cerrarSesion()	Llama PreferenciasSesion.limpiarSesion() y limpia SesionApp.
RepositorioFavor	obtenerDisponibles()	GET /favores/lista?estado=disponible.
	crear(favor)	POST JSON /favores/crear.
	aceptar(id)	PUT /favores/{id}/aceptar.
RepositorioGrupo	obtenerMisGrupos()	GET /grupos/mis-grupos.
	buscar(nombre)	GET /grupos/buscar-grupos?nombre={nombre}.
	crear(nombre, descripcion)	POST JSON /grupos/crear con Map<String, String?>.
	unirse(id)	POST /grupos/{id}/unirse.
RepositorioChat	obtenerMisChats()	GET /chat/mis-chats.
	obtenerOCrear(participantes)	POST JSON /chat/obtener-o-crear_chat con {"participantes": [...]}.
	obtenerPorId(id)	GET /chat/mis-chats y filtra por id localmente.
	enviarMensaje(idChat, contenido)	POST /chat/{id}/enviar con EnviarMensajeRequest(contenido).

Repositorio	Método	Llamada a la API
	marcarLeido(idChat)	PUT /chat/{id}/leer-chats.

9.5.3. ClienteRetrofit y detección de emulador

ClienteRetrofit es un object (singleton) que construye la instancia de Retrofit con dos interceptores de OkHttp:

- **Interceptor de autenticación:** lee SesionApp.token en cada petición (no en el momento de crear el cliente) y añade Authorization: Bearer {token} si no está en blanco. Esto permite que el token se aplique inmediatamente después del login sin reiniciar Retrofit.
- **HttpLoggingInterceptor:** registra en logcat el body completo de peticiones y respuestas en modo DEBUG.

La URL base se selecciona automáticamente mediante esEmulador(), que comprueba Build.FINGERPRINT, Build.MODEL, Build.MANUFACTURER y Build.PRODUCT:

Entorno	URL base
Emulador Android (10.0.2.2)	http://10.0.2.2:8000/
Dispositivo físico en red local	http://192.168.1.142:8000/

El archivo res/xml/network_security_config.xml permite tráfico HTTP en texto plano (cleartextTrafficPermitted=true) para el desarrollo local, referenciado desde AndroidManifest.xml mediante android:networkSecurityConfig.

9.5.4. Utilidades

PreferenciasSesion

Objeto singleton que encapsula el acceso a EncryptedSharedPreferences con esquema AES256_SIV para claves y AES256_GCM para valores. Almacena y recupera token, correo, nombre y rol. En guardarSesion() también actualiza los campos de SesionApp en memoria para que estén disponibles de inmediato sin necesidad de leer de disco. limpiarSesion() borra todas las entradas del almacén cifrado y llama SesionApp.limpiar().

AyudanteUbicacion

Clase que encapsula la lógica de geolocalización. Usa FusedLocationProviderClient de Google Play Services. El método obtenerDireccionActual() es suspend y usa suspendCancellableCoroutine para convertir el callback de lastLocation en una llamada suspendida. Si lastLocation devuelve null (GPS sin caché previa), la función también devuelve null y el fragmento muestra el error. La geocodificación inversa usa Geocoder.getFromLocation() con @Suppress("DEPRECATION") para compatibilidad con APIs

anteriores al 33. El resultado se formatea como "Calle Número, Ciudad, PaísCode" omitiendo los componentes que sean null o estén en blanco.

9.6.Gestión de Concurrency

La aplicación permite que múltiples usuarios estén conectados simultáneamente y realicen operaciones sobre los mismos recursos. La concurrencia se gestiona a tres niveles: en el backend, en la base de datos y en el cliente de escritorio.

Concurrencia en el backend (FastAPI + Motor): FastAPI es un framework asíncrono basado en el modelo `async/await` de Python. Todas las rutas de la API están definidas como funciones asíncronas, y el cliente Motor accede a MongoDB de forma no bloqueante. Esto significa que mientras una petición espera la respuesta de la base de datos, el servidor puede atender otras peticiones entrantes de forma concurrente sin necesidad de lanzar hilos adicionales.

- **Aceptación simultánea de un favor:** Cuando dos usuarios intentan aceptar el mismo favor al mismo tiempo, el endpoint verifica el estado actual del favor antes de ejecutar la actualización. Si el primer usuario completa la operación y cambia el estado a "aceptado", el segundo usuario recibe un error 400 porque la condición de guarda detecta que el estado ya no es "disponible". MongoDB garantiza atomicidad a nivel de documento, por lo que no puede producirse una escritura parcial ni un estado inconsistente.
- **Envío simultáneo de mensajes en el chat:** El operador `$push` de MongoDB añade un nuevo mensaje al array del documento de chat en una sola operación atómica. Aunque dos usuarios envíen mensajes al mismo instante, cada operación `$push` se ejecuta de forma independiente y atómica sobre el documento, por lo que ningún mensaje puede perderse ni sobreescribirse.
- **Actualización de roles en grupos:** El endpoint de cambio de rol utiliza la notación posicional de MongoDB para localizar y actualizar únicamente el elemento del array de miembros que cumple la condición indicada, sin afectar al resto de miembros del grupo. De este modo, dos administradores que modifiquen roles distintos de forma simultánea no pueden interferir entre sí.
- **Concurrencia en el cliente .NET MAUI:** Todas las llamadas HTTP se realizan de forma asíncrona mediante `async/await` para no bloquear el hilo principal de la interfaz de usuario. Cuando los datos regresan del servidor, las actualizaciones sobre las colecciones observables se ejecutan siempre en el hilo de UI para evitar excepciones de acceso cruzado entre hilos. Además, en la pantalla de inicio, las tres peticiones de carga (chats, grupos y favores) se lanzan en paralelo y se esperan conjuntamente, reduciendo el tiempo total de carga respecto a ejecutarlas de forma secuencial.

9.7.Sistema de Roles y Perfiles de Usuario

El sistema define tres roles con permisos progresivos:

Rol	Permisos
Usuario básico	Registrarse, iniciar sesión, crear/aceptar/cancelar/finalizar favores, enviar mensajes, unirse a grupos, pedir invitación, valorar usuarios.
Administrador de grupo	Todo lo anterior + expulsar miembros, aceptar/rechazar solicitudes, cambiar roles, editar grupo, eliminar grupo.
Administrador global	Supervisión completa de la plataforma, bloqueo de usuarios, eliminación de grupos.

El rol se almacena en el documento del usuario en MongoDB y se valida en el backend antes de ejecutar cualquier operación sensible. El middleware de autenticación (obtener_usuario_actual) inyecta el correo del usuario autenticado en cada endpoint mediante el token JWT.

9.8.Ciclo de Vida de un Favor

Cada favor sigue un ciclo de vida estructurado con cuatro estados posibles:

- Disponible: el favor ha sido publicado y está visible para todos los usuarios. Cualquier usuario (excepto el propietario) puede aceptarlo.
- Aceptado: un usuario ha aceptado el favor. El estado no puede volver a 'disponible'. Solo se puede cancelar o finalizar.
- Finalizado: ambas partes han completado el intercambio. El sistema notifica al otro usuario y permite valorarlo.
- Cancelado: uno de los participantes ha cancelado el favor en curso. Se notifica al otro usuario. El favor no puede borrarse mientras está en estado 'aceptado'.

9.9.Sistema de Notificaciones

El backend genera notificaciones automáticamente ante eventos relevantes: aceptación, cancelación o finalización de un favor; unión o expulsión de un grupo; cambio de rol; y cambio de nombre de grupo. Las notificaciones se almacenan en la colección notificaciones y el usuario puede consultarlas y marcarlas como leídas mediante los endpoints /notificaciones/mis-notificaciones y /notificaciones/marcar-leida.

10. Desarrollo Empresarial

10.1. PRESENTACIÓN DEL PROYECTO Y DEL EMPRENDEDOR

10.1.1. a) Presentación del proyecto

El proyecto que se presenta consiste en el desarrollo de una plataforma colaborativa de intercambio de favores entre particulares, concebida como una herramienta moderna de economía colaborativa orientada a comunidades locales, barrios, asociaciones vecinales, comunidades educativas y pequeños grupos temáticos. La plataforma permitirá a sus usuarios solicitar ayuda para tareas cotidianas, ofrecer servicios o habilidades a otras personas y comunicarse de manera privada y en tiempo real durante todo el proceso, garantizando así una experiencia de uso ágil, segura y cercana.

El sistema se compone de dos aplicaciones perfectamente integradas entre sí: por un lado, una aplicación de escritorio desarrollada en Python que concentra el grueso de la funcionalidad en la gestión de perfiles, administración de grupos temáticos, mapa con geolocalización, creación y modificación de favores, panel de chats y herramientas de administración; y por otro, una mini aplicación móvil ligera, pensada para consultas rápidas, recepción de notificaciones en tiempo real, aceptación de favores estando fuera de casa y mensajería instantánea. Ambas aplicaciones se conectan al mismo backend, lo que asegura la coherencia total de los datos y permite al usuario continuar su actividad desde cualquier dispositivo.

Técnicamente, el proyecto se construye sobre una arquitectura Modelo-Vista-Controlador (MVC) que separa con claridad la lógica de negocio, el acceso a datos y la interfaz gráfica. La capa de persistencia se apoya en MongoDB, una base de datos NoSQL que ofrece la flexibilidad necesaria para gestionar usuarios, favores, grupos y, sobre todo, las grandes cantidades de mensajes que genera la mensajería en tiempo real. La comunicación entre clientes y servidor se resuelve mediante sockets e hilos, lo que permite que múltiples usuarios estén conectados simultáneamente sin que el sistema se bloquee: cada cliente conectado dispone de su propio hilo en el servidor, y se han implementado mecanismos de control de concurrencia para situaciones críticas, como cuando dos usuarios intentan aceptar el mismo favor al mismo tiempo.

El proyecto integra de forma natural las competencias adquiridas a lo largo del ciclo formativo. De Acceso a Datos se toma el trabajo con bases de datos NoSQL (MongoDB); de Programación de Servicios y Procesos (PSP), la gestión de sockets, hilos y comunicación cliente-servidor; de Seguridad y Alta Disponibilidad / Sistemas de Gestión Empresarial (SGE), la definición de perfiles, roles y permisos junto con el almacenamiento seguro de credenciales; de Desarrollo de Interfaces (DI), la arquitectura MVC y el diseño avanzado de la interfaz gráfica de escritorio;

y de Programación Multimedia y Dispositivos Móviles (PMDM), el desarrollo de la mini aplicación móvil conectada al backend principal.

Las funcionalidades principales del sistema son: registro e inicio de sesión seguro, gestión completa del perfil de usuario (foto, descripción, valoración media), publicación y modificación de favores indicando título, descripción, ubicación y tipo (favor gratuito, remunerado o intercambio), búsqueda de favores cercanos mediante geolocalización opcional, chat privado en tiempo real entre los dos usuarios implicados, creación y administración de grupos temáticos (por ejemplo, “recogida de paquetes”, “cuidado de mascotas” o “ayuda informática”), sistema de valoración obligatoria de 1 a 5 estrellas al finalizar cada favor, y un sistema de roles claramente diferenciados: usuario básico, administrador de grupo y administrador global del sistema.

Desde el punto de vista de la metodología de desarrollo, se ha optado por un enfoque incremental e iterativo que ha permitido avanzar por fases (análisis de requisitos, diseño de la base de datos, implementación del backend, desarrollo del cliente de escritorio, construcción de la mini app móvil y pruebas finales), añadiendo nuevas funcionalidades de forma progresiva y validando cada bloque antes de continuar con el siguiente.

En el plano empresarial, el proyecto se plantea como una iniciativa emprendedora real, que arranca con una inversión mínima y una estructura unipersonal: el promotor asume directamente las áreas de desarrollo, diseño, marketing y atención al cliente. El modelo de negocio elegido es freemium, con funcionalidades básicas gratuitas y servicios premium opcionales (anuncios destacados, grupos ilimitados o gestión de pagos con pequeña comisión). La estrategia de lanzamiento es de proximidad: comenzar en un entorno reducido barrio, municipio o grupo de confianza, generar una comunidad activa con valoraciones reales y crecer de forma orgánica hacia ámbitos cada vez más amplios.

En definitiva, este proyecto persigue un doble objetivo. Por un lado, demostrar de manera integrada las competencias técnicas adquiridas durante el ciclo formativo a través de un producto software completo, robusto y bien estructurado. Por otro, aportar a la sociedad una herramienta útil con verdadero impacto social: facilitar la colaboración entre vecinos, reducir la dependencia de servicios profesionales para tareas sencillas, ofrecer un espacio seguro a personas vulnerables (especialmente personas mayores o con movilidad reducida) y, en definitiva, reforzar el sentido de comunidad mediante el uso responsable de la tecnología.

10.1.2. b) El emprendedor

A continuación se justifica la capacitación del promotor del proyecto, tanto desde el punto de vista de las actitudes emprendedoras como de las aptitudes técnicas y profesionales adquiridas en el Ciclo Formativo de Grado Superior en Desarrollo de Aplicaciones

Multiplataforma (DAM), para acometer con garantías de éxito el desarrollo e implantación de la plataforma.

Actitudes

Tras completar el cuestionario de autodiagnóstico de actitudes emprendedoras, el resultado obtenido refleja un perfil con una fuerte vocación comercial y adaptabilidad, destacando con las puntuaciones máximas en las áreas de **autonomía y compromiso personal, dedicación al negocio, comunicación y capacidad comercial, actitud al cambio e innovación**. A continuación se muestra el gráfico de barras resultante:



Resultados del autodiagnóstico:

Actitud emprendedora	Puntuación	Comentario
Autonomía y compromiso personal	100 / 100	Excelente capacidad para autogestionar el proyecto y un nivel máximo de compromiso personal con el mismo.
Importancia de dinero	79 / 100	Valoración alta de la retribución económica como factor motivador para el desarrollo del proyecto.
Iniciativa y Organización del trabajo	54 / 100	Nivel medio. Existe margen de mejora en la proactividad estructurada y la organización de las tareas diarias.

Actitud emprendedora	Puntuación	Comentario
Dedicación al negocio y confianza en la suerte	100 / 100	Dedicación total y absoluta al proyecto, combinada con una fuerte creencia en las posibilidades de que salga bien.
Comunicación y capacidad comercial	100 / 100	Habilidades excelentes para la venta, la atención directa al cliente y la transmisión clara de la idea de negocio.
Orientación al mercado	87 / 100	Muy buena comprensión de las necesidades de los clientes y capacidad para adaptar el producto a la demanda real.
Planificación y Calidad	40 / 100	Área de mejora prioritaria. Se necesita mayor rigor en la planificación por fases y en los procesos de control de calidad.
Propensión al riesgo	54 / 100	Perfil moderado/conservador frente al riesgo; preferencia por pasos seguros frente a grandes inversiones o decisiones inciertas.
Perfeccionismo y liderazgo	89 / 100	Alta exigencia personal y capacidad para guiar el proyecto o a futuros colaboradores manteniendo altos estándares.
Actitud al cambio y reconocimiento externo	100 / 100	Total apertura a nuevas ideas, alta adaptabilidad y receptividad ante el feedback y la valoración de los demás.

Como puede observarse en el gráfico, las puntuaciones máximas (100) corresponden a actitudes clave como la autonomía y compromiso, dedicación al negocio, comunicación, actitud al cambio e innovación. Esto indica un perfil altamente comprometido, muy adaptable, con excelentes dotes comunicativas y comerciales. Sin embargo, el autodiagnóstico también revela áreas que requieren especial atención: la planificación y calidad (40) y la creatividad (41), así como la organización del trabajo (54). Para garantizar el éxito del proyecto, el promotor deberá apoyarse en metodologías estrictas de trabajo o herramientas externas que compensen estas debilidades operativas y creativas.

Aptitudes

Las aptitudes profesionales del promotor se acreditan a través de las competencias y cualificaciones establecidas en el Real Decreto 450/2010, de 16 de abril, por el que se establece el título de Técnico Superior en Desarrollo de Aplicaciones Multiplataforma (DAM) y se fijan sus enseñanzas mínimas. A continuación se detallan las competencias profesionales, personales y sociales (Art. 5) y las cualificaciones profesionales (Art. 6) que avalan la capacitación técnica para llevar a cabo el proyecto.

COMPETENCIAS PROFESIONALES, PERSONALES Y SOCIALES

(Art. 5 del Real Decreto 450/2010, que establece el Título de Técnico Superior en Desarrollo de Aplicaciones Multiplataforma)

Competencia profesional	Capacidades que exige
a) Configurar y explotar sistemas informáticos, adaptando la configuración lógica del sistema según las necesidades de uso y los criterios establecidos.	Capacidad para configurar entornos de desarrollo, gestionar permisos de sistema y adaptar la configuración del servidor doméstico que da servicio a la aplicación.
b) Aplicar técnicas y procedimientos relacionados con la seguridad en sistemas, servicios y aplicaciones, cumpliendo el plan de seguridad.	Capacidad de diseñar e implementar autenticación segura, hash de contraseñas, control de acceso por roles y protección frente a accesos no autorizados al backend.
c) Gestionar bases de datos, interpretando su diseño lógico y verificando integridad, consistencia, seguridad y accesibilidad de los datos.	Capacidad para diseñar el modelo de datos (entidad-relación y relacional), implementarlo en MongoDB y garantizar la coherencia de la información (usuarios, favores, mensajes, grupos).
d) Desarrollar aplicaciones multiplataforma con acceso a bases de datos utilizando lenguajes, librerías y herramientas adecuados a las especificaciones.	Capacidad técnica de programar en Python tanto la aplicación de escritorio como el backend, y desarrollar la mini app móvil conectada al mismo servicio.
e) Desarrollar aplicaciones implementando un sistema completo	Capacidad de diseñar interfaces gráficas avanzadas (MVC), formularios

Competencia profesional	Capacidades que exige
de formularios e informes que permitan gestionar de forma integral la información almacenada.	de creación/edición de favores y panel administrativo de grupos.
f) Integrar contenidos gráficos y componentes multimedia en aplicaciones multiplataforma, empleando herramientas específicas y cumpliendo los requisitos establecidos.	Capacidad para integrar mapas, imágenes de perfil, iconografía y notificaciones visuales tanto en escritorio como en móvil.
g) Desarrollar interfaces gráficos de usuario interactivos y con la usabilidad adecuada, empleando componentes visuales estándar o implementando componentes específicos.	Capacidad de diseñar pantallas intuitivas (login, dashboard, chat, mapa, grupos) priorizando la experiencia de usuario y la accesibilidad.
h) Participar en la implantación de sistemas ERP-CRM evaluando la utilidad de cada uno de sus módulos.	Capacidad de pensar la plataforma como un sistema modular escalable, donde cada bloque (favores, mensajería, grupos, valoraciones) actúa como un módulo independiente y reutilizable.
i) Gestionar dispositivos e implantar aplicaciones para móviles y otros dispositivos, atendiendo a las particularidades de cada uno y a los requisitos de uso.	Capacidad para desarrollar la mini aplicación móvil teniendo en cuenta tamaño de pantalla, conectividad, batería y notificaciones push.
j) Aplicar técnicas de control de versiones, gestionando el código fuente para su evolución y mantenimiento.	Capacidad de utilizar Git y plataformas tipo GitHub/GitLab para versionar el código, mantener ramas y revertir cambios cuando es necesario.
k) Adaptarse a las nuevas situaciones laborales, manteniendo actualizados los conocimientos científicos, técnicos y tecnológicos relativos a su entorno profesional.	Capacidad de aprendizaje autónomo continuo de nuevas tecnologías (MongoDB, sockets, frameworks móviles) y disposición para reciclarse profesionalmente.

Competencia profesional	Capacidades que exige
l) Resolver situaciones, problemas o contingencias con iniciativa y autonomía en el ámbito de su competencia, con creatividad, innovación y espíritu de mejora.	Capacidad de resolución autónoma de problemas (conflictos de concurrencia, fallos del servidor, errores de despliegue) y mejora iterativa del producto.
m) Organizar y coordinar equipos de trabajo, supervisando el desarrollo del mismo, manteniendo relaciones fluidas y asumiendo el liderazgo.	Capacidad de liderazgo y coordinación; en este proyecto unipersonal, capacidad de autoorganizarse y, en un futuro, de gestionar un pequeño equipo de colaboradores.
n) Comunicarse con los compañeros, transmitiendo la información necesaria con claridad, eficacia y respeto a las personas.	Capacidad comunicativa: redacción de documentación técnica, soporte al usuario y elaboración de materiales divulgativos del proyecto.
ñ) Generar entornos seguros en el desarrollo de su trabajo y el de su equipo, supervisando y aplicando los procedimientos de prevención de riesgos laborales y ambientales.	Capacidad de respetar las normas de ergonomía del puesto de trabajo, descansos y prevención del estrés en el desarrollo de software.
o) Supervisar y aplicar procedimientos de gestión de calidad, accesibilidad universal y diseño para todos en las actividades profesionales incluidas en los procesos.	Capacidad de aplicar pautas de accesibilidad (contrastes, tamaños de fuente, lectores de pantalla) y validar la calidad mediante pruebas automatizadas.
p) Realizar la gestión básica para la creación y funcionamiento de una pequeña empresa y tener iniciativa en su actividad profesional con sentido de la responsabilidad social.	Capacidad para darse de alta como autónomo, llevar una contabilidad básica, presentar impuestos trimestrales y operar la empresa de forma responsable.
q) Ejercer sus derechos y cumplir con las obligaciones derivadas de su actividad profesional, de acuerdo con lo establecido en la legislación vigente,	Capacidad de conocer mis obligaciones legales (LOPD/RGPD respecto a los datos de los usuarios,

Competencia profesional	Capacidades que exige
participando activamente en la vida económica, social y cultural.	propiedad intelectual, fiscalidad como autónomo) y cumplirlas con rigor.

CUALIFICACIONES PROFESIONALES

(Art. 6 del Real Decreto 450/2010, que establece el Título de Técnico Superior en Desarrollo de Aplicaciones Multiplataforma)

Cualificaciones profesionales completas incluidas en el título:

Código	Cualificación profesional
IFC154_3	Desarrollo de aplicaciones con tecnologías web.
IFC155_3	Programación con lenguajes orientados a objetos y bases de datos relacionales.

Unidades de competencia que la integran:

Código	Cualificación profesional
UC0491_3	Desarrollar elementos software en el entorno cliente.
UC0492_3	Desarrollar elementos software en el entorno servidor.
UC0493_3	Implementar, verificar y documentar aplicaciones web en entornos internet, intranet y extranet.
UC0223_3	Configurar y explotar sistemas informáticos.
UC0226_3	Programar bases de datos relacionales.
UC0227_3	Desarrollar componentes software en lenguajes de programación orientados a objetos.

Estas cualificaciones acreditan oficialmente la capacitación del promotor para desarrollar aplicaciones multiplataforma de forma profesional, abarcando tanto la programación cliente

y servidor como la gestión de bases de datos y la integración de servicios, que son precisamente los pilares técnicos sobre los que se construye la plataforma de intercambio de favores.

Currículum Europass

Se aporta como Anexo I, al final del plan de empresa, el Currículum Vitae en formato Europass del promotor, en el que se detallan la formación reglada, la experiencia profesional, las competencias técnicas y los idiomas, complementando así la justificación de aptitudes que se ha desarrollado en este apartado.

Ver [Anexo I – Currículum Vitae Europass](#) al final del documento.

10.2. EL MERCADO Y LOS CLIENTES

10.2.1. a) Tipo de mercado

La plataforma colaborativa de intercambio de favores se encuadra dentro del sector de la economía colaborativa digital y, más concretamente, en el mercado de los servicios comunitarios entre particulares. Se trata de un mercado en pleno crecimiento, impulsado por la digitalización de las relaciones cotidianas, la necesidad de optimizar recursos y la creciente preferencia de los usuarios por confiar en personas reales vecinos, conocidos, miembros de su comunidad antes que en grandes empresas impersonales para resolver tareas del día a día.

Desde el punto de vista de la teoría económica, el mercado en el que opera la aplicación reúne las siguientes características:

- **Mercado de competencia monopolística:** existen varias empresas y plataformas (Wallapop, TaskRabbit, Nextdoor, grupos de Facebook...) que ofrecen servicios parcialmente sustitutivos, pero ninguna cubre exactamente el mismo nicho, por lo que cada una compite mediante diferenciación.
- **Mercado de servicios (B2C / C2C):** el producto que se ofrece no es un bien físico, sino una plataforma digital que conecta a particulares entre sí. La empresa actúa como intermediaria tecnológica y los usuarios son, simultáneamente, oferentes y demandantes.
- **Mercado digital y multiplataforma:** la operativa se desarrolla íntegramente en línea, tanto en aplicación de escritorio como móvil, lo que elimina barreras geográficas más allá de la lógica restricción del alcance local de los favores.
- **Mercado de proximidad:** aunque la tecnología es global, la utilidad real del servicio se concreta a nivel local barrio, municipio, comunidad de vecinos, lo que aproxima el mercado al concepto de hiperlocal.
- **Mercado en expansión:** la economía colaborativa, según diversos estudios sectoriales, crece a tasas superiores al 20 % anual en Europa, impulsada por la digitalización, la sostenibilidad y la búsqueda de ahorro y eficiencia por parte de los consumidores.

En resumen, se opera en un mercado en crecimiento, fragmentado y dominado por la diferenciación, donde existe espacio real para una propuesta que combine flexibilidad, confianza y cercanía como factores diferenciales.

10.2.2. b) El segmento de mercado

Aunque la plataforma tiene vocación de crecer y, a medio plazo, alcanzar una proyección global, la estrategia inicial se centra en un segmento muy concreto y delimitado: las comunidades locales de confianza, entendidas como vecindarios, barrios, municipios

pequeños, comunidades de vecinos, asociaciones y grupos cerrados de amigos o familiares en los que la cercanía física y el conocimiento mutuo facilitan la generación de confianza necesaria para que el intercambio de favores funcione.

Esta segmentación inicial responde a la lógica de la economía colaborativa de proximidad: empezar con un núcleo reducido donde sea fácil dar visibilidad al producto, generar valoraciones reales y, a partir de ahí, crecer de manera orgánica hacia comunidades cada vez más amplias. El segmento puede caracterizarse aplicando los criterios habituales de segmentación de marketing:

- **Criterio geográfico:** lanzamiento inicial en un entorno local concreto (un barrio o municipio), con escalado progresivo a nivel provincial, autonómico y, finalmente, nacional. La aplicación funciona en cualquier lugar con conexión a internet, pero su utilidad real depende de la masa crítica de usuarios cercanos.
- **Criterio demográfico:** público amplio, de entre 18 y 75 años, sin diferenciación significativa por sexo, ya que el tipo de favores ofrecidos es transversal. Se distinguen subsegmentos por edad (jóvenes, adultos y personas mayores) con necesidades distintas pero complementarias.
- **Criterio psicográfico:** personas con sensibilidad hacia la economía colaborativa, la solidaridad vecinal y el aprovechamiento eficiente de recursos. Valoran la confianza, la cercanía y la flexibilidad por encima del precio puro.
- **Criterio conductual:** usuarios con cierta familiaridad con el uso de smartphones, dispuestos a registrarse, valorar a otros usuarios y participar activamente en una comunidad digital. Se incluye tanto al usuario ocasional (pide o presta ayuda una o dos veces al mes) como al usuario intensivo (autónomos, voluntarios o personas con muchas microtarefas).

En definitiva, se trata de un segmento amplio en términos demográficos pero muy delimitado geográficamente, lo que permite enfocar de manera quirúrgica las primeras campañas de captación y crear un efecto de “bola de nieve” que vaya extendiendo el uso de la aplicación entre vecinos.

10.2.3. c) Estudio de mercado: los clientes

El estudio de los clientes potenciales se ha realizado a partir del análisis de las tendencias del sector, de la observación de plataformas similares y de la identificación de necesidades reales no totalmente cubiertas por la competencia. De este análisis se derivan los siguientes perfiles de cliente, que constituyen el público objetivo del proyecto:

Cliente 1: Estudiantes y jóvenes

- Perfil: nativos digitales, con dominio elevado de smartphones y aplicaciones móviles.
- Recursos: suelen tener tiempo y habilidades tecnológicas, pero poco dinero disponible.
- Necesidades que cubre la app: ofrecer ayuda a otros (montaje de muebles, soporte informático, transporte, apuntes) para construir reputación y, en su caso, obtener una pequeña compensación económica.
- Valor que aportan: dinamizan la plataforma con muchos micro-favores y elevan el grado de actividad.

Cliente 2: Personas mayores

- Perfil: usuarios de mayor edad, en muchos casos con cierta dependencia tecnológica o física.
- Necesidades que cubre la app: ayuda con trámites digitales (cita médica online, gestiones bancarias), pequeñas tareas físicas (mover un mueble, cambiar una bombilla), compañía o pequeñas gestiones cotidianas.
- Aspecto crítico: la interfaz debe ser sencilla, accesible y con buen contraste; el sistema de valoraciones les transmite seguridad.
- Valor social: la app contribuye a paliar la soledad no deseada y a fortalecer el vínculo intergeneracional.

Cliente 3: Familias y personas con poco tiempo

- Perfil: trabajadores activos, parejas con hijos pequeños, personas con cargas familiares.
- Necesidades que cubre la app: recogida de paquetes, ayuda puntual en mudanzas, montaje de muebles, cuidado de mascotas durante un fin de semana, etc.
- Comportamiento de uso: priorizan la rapidez, la fiabilidad y la cercanía geográfica.
- Disposición a pagar: media-alta para favores remunerados que les ahorren tiempo.

Cliente 4: Autónomos y microemprendedores

- Perfil: profesionales que ofrecen pequeños servicios y buscan visibilidad de proximidad.

- Necesidades que cubre la app: tablón gratuito y geolocalizado para anunciar microservicios, sistema de valoraciones que aumente su reputación.
- Comportamiento de uso: publican favores ofrecidos de forma recurrente y son candidatos naturales a las funcionalidades premium.

Cliente 5: Asociaciones y grupos locales

- Perfil: asociaciones vecinales, asociaciones de padres y madres, ONG locales, grupos religiosos, parroquiales o deportivos.
- Necesidades que cubre la app: organizar voluntarios, coordinar actividades internas y mantener un canal de comunicación más ordenado que un grupo de WhatsApp.
- Valor estratégico: actúan como amplificadores naturales del crecimiento de la plataforma dentro de su comunidad.

Como conclusión del estudio de mercado, los cinco perfiles identificados son complementarios y se retroalimentan: los jóvenes y autónomos ofrecen ayuda; las personas mayores, familias con poco tiempo y asociaciones la demandan. Esta complementariedad garantiza que en un mismo entorno local exista demanda y oferta de favores, condición indispensable para que la plataforma funcione.

10.3. ENTORNO Y LA COMPETENCIA

10.3.1. a) El entorno general (macroentorno)

El macroentorno está compuesto por aquellos factores externos a la empresa que escapan a su control, pero que influyen de forma significativa en la viabilidad y el desarrollo del proyecto. Se analizan a continuación los cuatro grandes grupos de factores siguiendo el modelo PEST (Político-legales, Económicos, Socioculturales y Tecnológicos), identificando para cada uno cómo afecta en clave de oportunidad o de amenaza al lanzamiento de la plataforma.

Factores políticos y legales

Aspectos clave del marco normativo aplicable:

- **Protección de datos personales (RGPD y LOPDGDD):** obligación de implementar un tratamiento seguro de los datos de los usuarios (nombre, correo, ubicación, mensajes privados, valoraciones), redactar una política de privacidad clara, recabar consentimientos explícitos y notificar posibles brechas de seguridad. Supone un coste de cumplimiento pero también una garantía de confianza para el usuario.
- **Normativa sobre economía colaborativa:** la legislación española y europea sobre intercambio entre particulares se encuentra en proceso de actualización constante. Hay que distinguir claramente entre favores gratuitos, intercambios y servicios remunerados con implicaciones fiscales.
- **Normativa fiscal y de autónomos:** el promotor opera como trabajador autónomo, lo que implica el pago de la cuota a la Seguridad Social (acogiéndose a la tarifa plana para nuevos autónomos) y la presentación trimestral de IVA e IRPF.
- **Propiedad industrial e intelectual:** registro de la marca en la OEPM (Oficina Española de Patentes y Marcas) para proteger el nombre de la aplicación y el logotipo.
- **Políticas de las tiendas de aplicaciones:** cumplir las condiciones de Google Play y App Store (cuotas, revisiones, posibles cambios de política).

Influencia sobre el proyecto: estos factores actúan principalmente como una exigencia (coste de cumplimiento) pero también como una oportunidad, ya que diferenciarse por una gestión escrupulosa de la privacidad refuerza la propuesta de valor en un mercado especialmente sensible a la confianza.

Factores socioculturales

- **Auge de la economía colaborativa:** tras la pandemia, la sociedad ha incrementado su demanda de soluciones comunitarias y de proximidad. Esto representa una clara oportunidad para la aplicación.

- **Soledad no deseada y envejecimiento de la población:** España tiene una población cada vez más envejecida y un porcentaje creciente de personas que viven solas. La plataforma puede convertirse en una herramienta de apoyo y de creación de vínculos vecinales.
- **Cambio de mentalidad en las nuevas generaciones:** los jóvenes valoran el acceso por encima de la propiedad y se sienten cómodos con el intercambio y la compartición de recursos.
- **Desconfianza hacia los extraños:** principal amenaza sociocultural. Para combatirla se diseña un sistema de valoraciones obligatorio, perfiles verificados y comunicación privada cifrada.
- **Conciencia social y voluntariado:** creciente sensibilidad hacia la ayuda mutua y la solidaridad de proximidad, especialmente en entornos urbanos y semiurbanos.

Factores económicos

- **Coyuntura económica y poder adquisitivo:** la inflación y la pérdida de poder adquisitivo en los hogares españoles incentivan la búsqueda de alternativas más económicas a los servicios profesionales para tareas sencillas. Oportunidad clara para favores gratuitos o de bajo coste.
- **Tipos de interés y financiación:** factor irrelevante en esta fase, ya que el proyecto se financia íntegramente con recursos propios y no requiere acudir a financiación bancaria.
- **Costes tecnológicos:** el coste del hardware, las licencias de las tiendas y la conectividad se mantiene estable y permite una inversión inicial muy contenida (en torno a 725 €) y unos costes fijos mensuales reducidos (115 € aproximadamente).
- **Mercado laboral:** la creciente precariedad y la aparición de la “economía de los pequeños encargos” aporta oferentes naturales para favores remunerados.
- **Subvenciones a emprendedores:** existen ayudas a la creación de empresas y a jóvenes autónomos (tarifa plana, programas autonómicos...) que reducen aún más la barrera de entrada.

Factores tecnológicos

- **Penetración del smartphone:** más del 95 % de la población española de entre 16 y 74 años dispone de teléfono móvil, lo que elimina una de las barreras tradicionales al uso de aplicaciones digitales. Oportunidad de primer orden.

- **Conectividad y cobertura:** el despliegue de la fibra óptica y la red 4G/5G garantiza que la mensajería en tiempo real (basada en sockets) funcione de forma estable en prácticamente todo el territorio.
- **Madurez del software libre:** tecnologías como Python, MongoDB y las librerías de sockets son gratuitas, robustas y cuentan con comunidades de soporte muy activas. Permite mantener los costes de desarrollo en niveles mínimos.
- **Servicios cloud y backups:** el acceso a almacenamiento en la nube a precios reducidos (Google Drive, OneDrive, etc.) permite garantizar copias de seguridad fiables sin grandes inversiones.
- **Ciberseguridad:** principal amenaza tecnológica. La aplicación gestiona datos personales y mensajes privados, lo que obliga a un diseño cuidadoso (hash de contraseñas, validación de roles, control de acceso). Se mitiga con buenas prácticas de programación segura.
- **Inteligencia artificial:** oportunidad a medio plazo para incorporar recomendaciones automáticas de favores, moderación de contenido y detección de fraudes.

10.3.2. b) El entorno específico (microentorno)

El microentorno engloba aquellos factores próximos a la empresa que sí pueden influirse o gestionarse y que afectan directamente a la operativa diaria del negocio. Se analizan los clientes, la competencia, el producto y los proveedores.

Clientes

Los perfiles de cliente ya han sido detallados en el apartado 3.c. Desde el punto de vista del microentorno, lo relevante es cómo este conocimiento se traduce en decisiones concretas de marketing:

- **Política de precios:** dado que los segmentos jóvenes y personas mayores son sensibles al precio, el modelo principal es freemium con funciones básicas gratuitas. La monetización se concentra en autónomos y usuarios intensivos a través de funciones premium.
- **Promoción:** campañas geolocalizadas (carteles físicos en barrios, redes sociales locales, colaboración con asociaciones vecinales) en lugar de campañas masivas en medios generalistas.
- **Distribución:** descarga directa desde Google Play y App Store; la versión de escritorio se ofrece desde la web oficial del proyecto.
- **Atención al cliente:** directa, personal y rápida, asumida por el propio promotor en la fase inicial.

Competencia

El análisis detallado de los competidores se realiza en el apartado 4.c). De forma resumida, se identifican como competidores directos las plataformas de servicios entre particulares (TaskRabbit, Etece), como competidores indirectos las plataformas de compraventa con sección de servicios (Wallapop, Milanuncios) y como competidores sustitutivos los grupos informales de WhatsApp, Facebook o Nextdoor.

Producto

La plataforma se diferencia del resto del mercado por una combinación única de atributos que aportan valor añadido al usuario:

- Flexibilidad total: favores gratuitos, intercambios recíprocos o servicios remunerados, todo en la misma aplicación.
- Doble acceso: aplicación de escritorio para gestión completa y aplicación móvil ligera para uso en movilidad.
- Chat en tiempo real con sockets, sin necesidad de pasar por aplicaciones externas como WhatsApp.
- Grupos temáticos administrables por los propios usuarios, con roles y permisos diferenciados.
- Sistema de valoraciones de 1 a 5 estrellas obligatorio tras cada favor, que construye una reputación real.
- Geolocalización opcional para encontrar favores cercanos.
- Diseño orientado a la confianza: perfiles completos, descripciones, fotos y valoraciones visibles.

Proveedores

Al tratarse de un producto puramente digital, no existen proveedores de materias primas en el sentido clásico. Los proveedores relevantes son:

- **Proveedor de conectividad a internet:** compañía telefónica que da servicio de fibra al domicilio (≈ 20 €/mes imputados al proyecto).
- **Compañía eléctrica:** suministro energético del servidor doméstico (≈ 15 €/mes).
- **Tiendas de aplicaciones:** Google Play (pago único de aproximadamente 25 €) y Apple App Store (suscripción anual de aproximadamente 95 €).
- **Proveedor de dominio y registro de marca:** registrador de dominios para la web oficial y Oficina Española de Patentes y Marcas (≈ 155 € por la primera categoría).

- **Tecnologías de desarrollo:** Python, MongoDB y librerías de sockets son gratuitas y de código abierto, lo que elimina la dependencia de proveedores propietarios.
- **Almacenamiento en la nube:** servicios gratuitos o de muy bajo coste para copias de seguridad (Google Drive, OneDrive).

La baja dependencia de proveedores y la diversificación entre alternativas gratuitas refuerzan la competitividad y la resiliencia del proyecto.

10.3.3. c) La competencia

Aunque ya se ha tratado de forma general en el apartado 4.b), conviene profundizar en el análisis de los principales competidores presentes en el mercado, identificando sus puntos fuertes, sus puntos débiles y la posición diferencial de la plataforma propuesta.

Wallapop

- Tipo: competidor indirecto.
- Fortalezas: marca consolidada, base de usuarios masiva, geolocalización integrada.
- Debilidades: orientado a la compraventa de productos físicos, no permite gestionar favores ni dispone de un sistema estructurado de seguimiento de tareas. El chat no es realmente en tiempo real.
- Diferencia clave: la app desarrollada se centra en favores y microtareas (no en productos) y dispone de chat en tiempo real con sockets.

TaskRabbit / Etece

- Tipo: competidor directo en el ámbito de servicios.
- Fortalezas: red de profesionales verificados, pasarela de pago integrada, presencia internacional (TaskRabbit) o nacional (Etece).
- Debilidades: orientadas exclusivamente a servicios de pago, sus tarifas son elevadas y eliminan la posibilidad del favor gratuito o el intercambio entre vecinos.
- Diferencia clave: la plataforma propia permite combinar favores gratuitos, recíprocos y remunerados, lo que la convierte en una opción más accesible para tareas pequeñas y no profesionales.

Nextdoor

- Tipo: competidor directo en el ámbito vecinal.

- Fortalezas: enfoque hiperlocal por barrios, comunidad fiel en algunas ciudades.
- Debilidades: funciona como un tablón de anuncios, sin estructura para gestionar un favor de principio a fin, sin sistema de valoraciones serio ni gestión de pagos.
- Diferencia clave: el proyecto incorpora ciclo de vida completo del favor (creación → publicación → aceptación → comunicación → finalización → valoración), grupos administrables y, en el futuro, gestión segura de pagos.

Grupos de WhatsApp, Facebook Groups y Telegram

- Tipo: competidores sustitutivos informales.
- Fortalezas: gratuitos, sin barreras de entrada, ya instalados en el móvil de los usuarios.
- Debilidades: caóticos, sin estructura, sin reputación verificable, sin geolocalización, sin gestión de roles ni administración real, mensajería ruidosa que mezcla anuncios con conversaciones.
- Diferencia clave: la plataforma aporta orden, búsqueda, valoraciones y especialización por grupos temáticos.

Mapa de posicionamiento

Comparativa resumida frente a los principales competidores en los atributos críticos para el usuario:

Atributo	Mi plataforma	Wallapop	TaskRabbit	Nextdoor
Favores gratuitos	Sí	No	No	Parcial
Favores remunerados	Sí	No	Sí	No
Chat en tiempo real	Sí	Si	Sí	No

Atributo	Mi plataforma	Wallapop	TaskRabbit	Nextdoor
Grupos temáticos administrables	Sí	No	No	Parcial
Sistema de valoraciones	Sí (obligatorio)	Sí	Sí	Parcial
Geolocalización	Sí (opcional)	Sí	Sí	Sí
App escritorio + móvil	Sí	Solo móvil/web	Solo móvil/web	Solo móvil/web
Coste para el usuario	Gratis (freemium)	Gratis + comisión	Alto	Gratis

La conclusión del análisis competitivo es clara: la plataforma propuesta no compite cara a cara con ninguno de los grandes nombres del sector, sino que cubre un hueco intermedio entre todos ellos, combinando lo mejor de cada uno. Esta posición diferencial constituye la principal fortaleza comercial del proyecto.

10.3.4. d) Análisis DAFO

El análisis DAFO permite cruzar el estudio del macroentorno y del microentorno para evaluar la solidez del proyecto, del promotor y de su interacción con el mercado. A continuación se presenta la matriz DAFO obtenida tras realizar el cuestionario en línea.

 Debilidades	 Amenazas
 Dificultad para verificar la calidad y cumplimiento real de los favores.	 E Riesgo de fraudes, perfiles falsos o incumplimientos.
 Necesidad de moderación y soporte constante en la plataforma.	 T Pérdida de usuarios si la experiencia no es segura o rápida.
 Dependencia de que los usuarios sean activos y participen constantemente.	 T Competencia de aplicaciones grandes de servicios o economía colaborativa.
 Dependencia tecnológica de la app/web y servidores.	 T Ciberataques o robo de información de usuarios.
 Posibles conflictos o malas experiencias entre usuarios.	 L Problemas legales relacionados con responsabilidad entre usuarios.
 Fortalezas	 Oportunidades
 Bajo coste comparado con plataformas de servicios tradicionales.	 S Posibilidad de integrar sistema de reputación y valoraciones.
 Fomenta comunidad y confianza entre usuarios.	 E Crecimiento de la economía colaborativa y consumo responsable.
 Plataforma accesible desde móvil y desde el ordenador.	 S Expansión a universidades, barrios o comunidades locales.
 Idea innovadora basada en colaboración y ayuda mutua.	 T Aumento del uso de aplicaciones móviles sociales.
 Posibilidad de crecimiento rápido mediante recomendaciones.	

Matriz CAME

A partir de la matriz DAFO se elabora la matriz CAME, que determina las medidas estratégicas a adoptar: Corregir las debilidades, Afrontar las amenazas, Mantener las fortalezas y Explotar las oportunidades.



Estrategia Adaptativa

Implementación de validación social (Ratings) para escalabilidad barrial y reducción de moderación.

Utilizar la predisposición actual hacia el consumo colaborativo y las apps móviles para expandirnos bloque a bloque en barrios y universidades. Para solventar la dificultad de comprobar si un favor se hizo bien y evitar conflictos entre usuarios, implementaremos un sistema de reputación obligatoria



Debilidades

- Necesidad de moderación y soporte constante en la plataforma.
- Dificultad para verificar la calidad y cumplimiento real de los favores.
- Dependencia de que los usuarios sean activos y participen constantemente.



Oportunidades

- \$ Posibilidad de integrar sistema de reputación y valoraciones.
- \$ Expansión a universidades, barrios o comunidades locales.
- E Crecimiento de la economía colaborativa y consumo responsable.
- T Aumento del uso de aplicaciones móviles sociales.



Estrategia Adaptativa

Desarrollo de un ecosistema de confianza autogestionado en comunidades locales.

Aprovechar el auge de las apps sociales y la economía colaborativa para lanzar la plataforma en nichos cerrados como universidades y barrios, garantizando así una masa crítica inicial de usuarios activos. En estas comunidades, se integrará un fuerte sistema automatizado de reputación



Debilidades

- Necesidad de moderación y soporte constante en la plataforma.
- Posibles conflictos o malas experiencias entre usuarios.
- Dificultad para verificar la calidad y cumplimiento real de los favores.
- Dependencia de que los usuarios sean activos y participen constantemente.
- Dependencia tecnológica de la app/web y servidores.



Oportunidades

- \$ Posibilidad de integrar sistema de reputación y valoraciones.
- \$ Expansión a universidades, barrios o comunidades locales.
- E Crecimiento de la economía colaborativa y consumo responsable.
- T Aumento del uso de aplicaciones móviles sociales.



Estrategia Defensiva

— Blindaje Integral Comunitario: Seguridad, Confianza y Accesibilidad como Ventaja Competitiva.

Frente a la amenaza de grandes plataformas del sector y el riesgo de pérdida de usuarios por malas experiencias, utilizaremos nuestra esencia innovadora de ayuda mutua y el fomento de una comunidad sólida para crear un entorno de máxima confianza.



Fortalezas

- Plataforma accesible desde móvil y desde el ordenador.
- Fomenta comunidad y confianza entre usuarios.
- Bajo coste comparado con plataformas de servicios tradicionales.
- Idea innovadora basada en colaboración y ayuda mutua.
- Posibilidad de crecimiento rápido mediante recomendaciones.



Amenazas

- T Competencia de aplicaciones grandes de servicios o economía colaborativa.
- T Ciberataques o robo de información de usuarios.
- E Riesgo de fraudes, perfiles falsos o incumplimientos.
- L Problemas legales relacionados con responsabilidad entre usuarios.
- T Pérdida de usuarios si la experiencia no es segura o rápida.



Estrategia Defensiva

— Estrategia de Retención y Protección basada en la Auto-regulación y Prevención.

Para defendernos de la competencia de los gigantes de la economía colaborativa, nos apalancaremos en nuestra accesibilidad total (ordenador y móvil) y nuestro modelo de bajo coste para ofrecer un servicio más cercano, rápido y económico.



Fortalezas

- Plataforma accesible desde móvil y desde el ordenador.
- Fomenta comunidad y confianza entre usuarios.
- Bajo coste comparado con plataformas de servicios tradicionales.
- Idea innovadora basada en colaboración y ayuda mutua.
- Posibilidad de crecimiento rápido mediante recomendaciones.



Amenazas

- T Competencia de aplicaciones grandes de servicios o economía colaborativa.
- T Ciberataques o robo de información de usuarios.
- E Riesgo de fraudes, perfiles falsos o incumplimientos.
- L Problemas legales relacionados con responsabilidad entre usuarios.
- T Pérdida de usuarios si la experiencia no es segura o rápida.



Estrategia Ofensiva

— Expansión viral e hiperlocal impulsada por la confianza y el auge de la economía colaborativa.

Aprovechar el fuerte crecimiento actual de la economía colaborativa y el uso masivo de aplicaciones móviles sociales para potenciar nuestra idea innovadora basada en la ayuda mutua.



Fortalezas

- Plataforma accesible desde móvil y desde el ordenador.
- Fomenta comunidad y confianza entre usuarios.
- Bajo coste comparado con plataformas de servicios tradicionales.
- Idea innovadora basada en colaboración y ayuda mutua.
- Posibilidad de crecimiento rápido mediante recomendaciones.



Oportunidades

- **S** Posibilidad de integrar sistema de reputación y valoraciones.
- **S** Expansión a universidades, barrios o comunidades locales.
- **E** Crecimiento de la economía colaborativa y consumo responsable.
- **T** Aumento del uso de aplicaciones móviles sociales.



Estrategia Ofensiva

— Posicionamiento disruptivo en comunidades locales mediante validación social y escalabilidad de bajo

Utilizar nuestra ventaja competitiva de bajo coste y la accesibilidad multiplataforma para liderar la tendencia del consumo responsable y las apps sociales. El plan consiste en desplegar nuestra innovadora propuesta de favores directamente en entornos clave como universidades



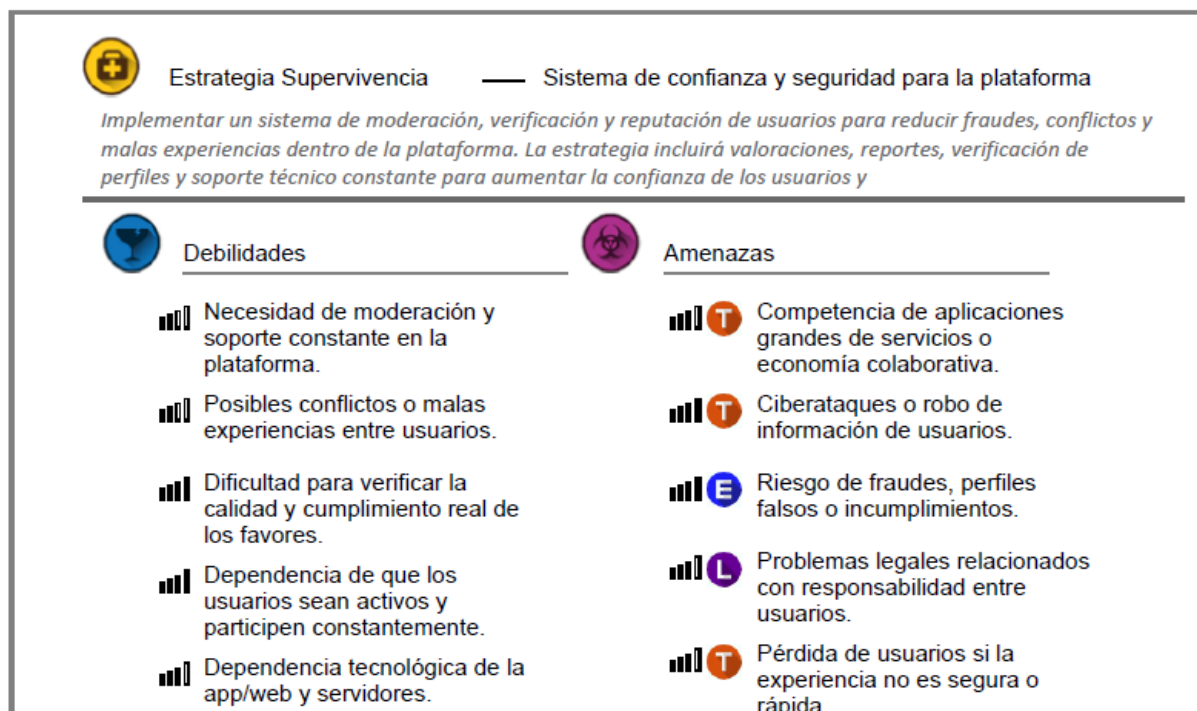
Fortalezas

- Plataforma accesible desde móvil y desde el ordenador.
- Fomenta comunidad y confianza entre usuarios.
- Bajo coste comparado con plataformas de servicios tradicionales.
- Idea innovadora basada en colaboración y ayuda mutua.
- Posibilidad de crecimiento rápido mediante recomendaciones.



Oportunidades

- **S** Posibilidad de integrar sistema de reputación y valoraciones.
- **S** Expansión a universidades, barrios o comunidades locales.
- **E** Crecimiento de la economía colaborativa y consumo responsable.
- **T** Aumento del uso de aplicaciones móviles sociales.



La conjunción del análisis DAFO y la matriz CAME confirma que el proyecto es sólido: las debilidades son corregibles a corto plazo, las amenazas tienen contramedidas claras, las fortalezas constituyen ventajas competitivas reales y las oportunidades del entorno permiten un crecimiento progresivo y sostenible.

10.3.5. e) La localización de la empresa

La empresa adopta una estructura de localización realista y austera, operando desde un despacho específico habilitado en Cuenca. Para cumplir estrictamente con la normativa de operaciones vinculadas y evitar cualquier contingencia fiscal (como una donación encubierta o una aportación encubierta de capital), la sociedad formaliza un contrato de arrendamiento con el promotor por el uso de este espacio. Se fija una renta a precio de mercado completamente defensiva y justificada de 300 € mensuales (3.600 € anuales) por el uso proporcional del inmueble.

La elección de esta localización y este planteamiento responde a varios criterios estratégicos:

- Coste operativo óptimo: Una renta de 300 € al mes permite a la sociedad disponer de una sede operativa legal a un precio muy inferior al de un local comercial o una oficina independiente en el mercado convencional.
- Cumplimiento normativo absoluto: Al registrar un contrato de alquiler real entre el socio y la S.L., se elimina el riesgo de inspección por cesiones gratuitas irregulares, declarándose correctamente el gasto en la sociedad y el correspondiente rendimiento de capital inmobiliario en el IRPF del arrendador.
- Disponibilidad total: Permite operar las 24 horas del día, los 7 días de la semana de forma flexible, algo fundamental para el mantenimiento técnico de la plataforma y el soporte a los usuarios fuera de las horas de oficina estándar.

- **Conexión al territorio:** Mantener la sede en Cuenca facilita el testeo inicial del servicio en un entorno controlado y cercano (comercios locales, asociaciones y redes del promotor) antes de escalar la actividad.

A medio y largo plazo, cuando la plataforma alcance una masa crítica de usuarios y el flujo de caja lo permita, se planteará la transición de la infraestructura a proveedores cloud profesionales y el alquiler de una oficina compartida o un espacio de coworking en la propia ciudad de Cuenca.

10.3.6. f) Misión, visión y valores

Misión

Facilitar el intercambio de favores y microservicios entre personas de una misma comunidad de forma fácil, segura y cercana, ofreciendo una plataforma digital que combine flexibilidad económica (favores gratuitos, recíprocos o remunerados), comunicación en tiempo real y un sistema de reputación basado en la confianza, contribuyendo así a recuperar el sentido de comunidad mediante el uso responsable de la tecnología.

Visión

Convertirnos en la plataforma de referencia para la ayuda mutua entre vecinos en España, logrando que cada barrio cuente con una comunidad activa de usuarios que se apoyen entre sí en las tareas cotidianas y proyectando, a medio plazo, el modelo a otros países europeos con una cultura social similar.

Valores

- **Ayuda mutua:** creemos en el valor del apoyo entre iguales como motor de bienestar social.
- **Confianza y seguridad:** todo está diseñado para que el usuario se sienta cómodo y protegido al interactuar con desconocidos.
- **Transparencia:** claridad total en la política de privacidad, en las valoraciones y en las (eventuales) comisiones por favores remunerados.
- **Respeto y convivencia:** tolerancia cero ante comportamientos abusivos; normas claras y aplicación firme de las mismas.
- **Innovación responsable:** usamos la tecnología para acercar a las personas, no para sustituir el contacto humano.
- **Inclusión:** la plataforma debe ser accesible a personas de distintas edades, capacidades y niveles tecnológicos.
- **Sostenibilidad:** operación con costes mínimos y huella ambiental reducida.

- **Compromiso con la comunidad local:** el éxito se mide en términos de utilidad real para los vecinos, no únicamente en cifras de descargas.

10.3.7. g) Responsabilidad Social Corporativa

La Responsabilidad Social Corporativa (RSC) constituye uno de los pilares del proyecto, no como un elemento añadido a posteriori, sino como un componente intrínseco de la propia naturaleza de la plataforma. La aplicación tiene, desde su concepción, una clara vocación social: facilitar el contacto entre vecinos, reducir la soledad no deseada y promover una economía más humana y cercana.

Las líneas de actuación en materia de RSC se articulan en torno a cuatro ejes:

1. Compromiso social

- Espacio seguro para personas vulnerables (mayores, personas con movilidad reducida, personas en situación de soledad) que necesiten ayuda en tareas cotidianas.
- Diseño inclusivo: la interfaz se adapta a usuarios con distintos niveles tecnológicos, contemplando opciones de accesibilidad (tamaño de fuente, alto contraste, navegación simplificada).
- Colaboración con asociaciones vecinales, ONG y entidades sociales para utilizar la plataforma como herramienta de organización de voluntariado.
- Promoción de los favores gratuitos como valor diferencial y como mecanismo de ayuda solidaria.

2. Compromiso con la privacidad y la ética

- Estricto cumplimiento del RGPD y de la LOPDGDD en el tratamiento de datos personales.
- Transparencia total: políticas de privacidad redactadas en lenguaje claro, accesibles desde cualquier pantalla de la aplicación.
- Mínima recogida de datos: se solicita únicamente la información imprescindible para el funcionamiento del servicio.
- Cifrado del chat y almacenamiento seguro de las credenciales (hash) para garantizar la confidencialidad.
- No reventa ni cesión a terceros de los datos de los usuarios.

3. Compromiso medioambiental

- Operación digital y teletrabajo: sin desplazamientos profesionales ni huella de carbono asociada a oficinas físicas.
- Uso de un único servidor doméstico optimizado en lugar de infraestructuras sobredimensionadas.
- Fomento, a través de la propia plataforma, de prácticas sostenibles: compartir herramientas, reutilizar objetos, evitar desplazamientos innecesarios o reducir el consumo de servicios profesionales para tareas que pueden resolverse entre vecinos.
- Apuesta por proveedores energéticos con energías renovables cuando sea posible.

4. Compromiso económico y fiscal

- Cumplimiento riguroso de las obligaciones fiscales y de Seguridad Social del autónomo.
- Política de precios accesibles: el modelo freemium garantiza que las funciones básicas estén siempre al alcance de cualquier usuario, independientemente de su capacidad económica.
- Reinversión de los beneficios iniciales en la mejora del producto y en la accesibilidad de la plataforma.
- Si en el futuro se incorporan colaboradores, se hará bajo condiciones laborales dignas, con contratos justos y conciliación familiar.

En definitiva, la RSC del proyecto no se concibe como un mero requisito formal, sino como su razón de ser: la plataforma existe precisamente para generar un impacto social positivo, y todas las decisiones técnicas, comerciales y estratégicas se toman teniendo presente esta finalidad.

10.4. EL MARKETING

10.4.1. a) Estrategias de promoción o posicionamiento

El objetivo de la estrategia de promoción es lograr que, cuando alguien necesite una pequeña ayuda en su entorno cercano, la aplicación sea el primer recurso al que acuda de forma natural. Para ello se han seleccionado los canales de comunicación que mejor se ajustan al perfil del público objetivo, al presupuesto disponible y a la naturaleza hiperlocal del producto en su fase de lanzamiento.

Canales de comunicación seleccionados

Redes sociales (Instagram, TikTok y grupos de Facebook y WhatsApp)

Se crearán perfiles oficiales de la aplicación en Instagram y TikTok, donde se publicará contenido breve y cercano: situaciones cotidianas que la app resuelve, casos de uso reales y testimonios de los primeros usuarios. Instagram permite trabajar tanto con publicaciones en el feed como con historias y reels, con un alcance elevado entre adultos de entre 25 y 45 años. TikTok resulta especialmente eficaz para llegar al segmento de estudiantes y jóvenes.

Los grupos locales de Facebook y los grupos de vecinos en WhatsApp son, sin embargo, el canal más estratégico en la fase inicial. Estos grupos ya reúnen exactamente al público objetivo: personas que comparten una ubicación geográfica y que conviven en un mismo barrio o comunidad, por lo que el mensaje de la aplicación encaja de forma natural en esos espacios sin necesidad de inversión económica.

Marketing de contenidos y posicionamiento en buscadores (SEO)

Se desarrollará un pequeño blog dentro de la web oficial de la aplicación con artículos sobre temas relacionados con la economía colaborativa, la convivencia vecinal y la solidaridad de proximidad, lo que permite aparecer de forma orgánica en los resultados de Google sin coste adicional.

Cartelería física en el lanzamiento local

Durante la fase de lanzamiento se diseñarán y distribuirán carteles en tabloneros de anuncios de comunidades de vecinos, comercios de proximidad, bibliotecas, centros cívicos y asociaciones locales. La cartelería física tiene un impacto especialmente notable entre los segmentos de personas mayores y familias, que tienen menos presencia en redes sociales pero sí frecuentan esos espacios.

Optimización en las tiendas de aplicaciones (ASO)

Se trabajará el título, la descripción, las palabras clave y las capturas de pantalla de la ficha de la aplicación en Google Play y App Store para mejorar su visibilidad dentro de cada tienda.

Justificación de los canales elegidos

Las redes sociales presentan como principal ventaja que su coste de entrada es prácticamente nulo cuando se gestiona de forma orgánica. La desventaja es que requieren una dedicación constante para generar contenido de calidad y que el alcance orgánico ha disminuido en los

últimos años. Para contrarrestar este efecto, se contempla la posibilidad de realizar pequeñas campañas de publicidad de pago puntual, con presupuesto mínimo, una vez la plataforma esté en funcionamiento y genere sus primeros ingresos.

La cartelería física tiene como ventaja su capacidad de impactar a segmentos de la población con menor presencia digital, refuerza la credibilidad del proyecto y genera un efecto de proximidad que las redes sociales no pueden replicar. Su limitación es evidente: el alcance geográfico es reducido y el coste de impresión y distribución, aunque pequeño, es superior a cero. Sin embargo, esta limitación encaja perfectamente con la estrategia de arranque local.

El sistema de valoraciones y reputación de la propia aplicación actuará, con el tiempo, como el mejor canal de comunicación posible: la recomendación directa de un vecino a otro es el tipo de promoción más eficaz en un producto basado en la confianza.

10.4.2. b) Estrategias de producto

Descripción detallada del producto

La aplicación es una plataforma digital de gestión de favores y microtarefas diseñada para fomentar la ayuda mutua dentro de comunidades cercanas. Su propuesta de valor se apoya en una combinación de funcionalidades que, en conjunto, no tiene ningún competidor directo en el mercado español:

Flexibilidad total en el tipo de compensación

La aplicación no impone un único modelo económico. El usuario que publica un favor puede elegir entre tres modalidades: ofrecerlo de forma completamente gratuita por solidaridad, proponer un intercambio recíproco de habilidades o servicios entre ambas partes, o acordar una pequeña compensación económica si las dos personas están de acuerdo.

Sistema de chat privado en tiempo real

Cada favor genera la posibilidad de abrir una conversación privada entre los dos usuarios implicados, integrada dentro de la propia aplicación, lo que elimina la necesidad de intercambiar números de teléfono con desconocidos. La conversación queda registrada y el sistema garantiza que únicamente los participantes autorizados tienen acceso a ella.

Grupos temáticos con gestión avanzada

Los usuarios pueden crear o unirse a grupos de interés común: "Cuidado de mascotas en el barrio", "Ayuda informática", "Intercambio de idiomas", "Voluntariado en la asociación", entre otros. Cada grupo dispone de su propio espacio de comunicación y de un sistema de administración que distingue entre miembros ordinarios y administradores. El administrador puede aprobar o rechazar solicitudes de entrada, invitar a usuarios por correo electrónico, cambiar los roles de los miembros o expulsarlos si vulneran las normas del grupo.

Sistema de reputación mediante valoraciones

Al finalizar cada favor, los usuarios implicados pueden calificarse mutuamente con una puntuación de una a cinco estrellas. La media de todas las valoraciones recibidas es pública y

visible en el perfil de cada usuario, lo que construye un sistema de confianza que incentiva el buen comportamiento dentro de la plataforma.

Disponibilidad en escritorio y en dispositivos móviles

La aplicación está diseñada para funcionar tanto en ordenadores con Windows como en teléfonos móviles con Android o iOS. La versión de escritorio está orientada a la gestión completa del perfil, la administración de grupos y la publicación detallada de favores; la versión móvil prioriza la consulta rápida, las notificaciones en tiempo real y la mensajería instantánea. Ambas versiones se conectan al mismo sistema central, garantizando que la información esté siempre actualizada independientemente del dispositivo utilizado.

Sistema de notificaciones automáticas

La plataforma informa a los usuarios de todos los eventos relevantes que les afectan: cuando alguien acepta su favor, cuando son admitidos en un grupo, cuando reciben una solicitud de entrada, cuando su rol dentro de un grupo cambia o cuando un favor ha sido marcado como finalizado.

Módulo de favores con ciclo de vida completo

Cada favor atraviesa un ciclo definido: disponible (visible para todos los usuarios), aceptado (asignado a un usuario concreto), finalizado (completado satisfactoriamente) o cancelado. Este seguimiento aporta claridad y seguridad a ambas partes, algo que no existe en los grupos informales de WhatsApp o en el tablón de anuncios de Nextdoor.

Búsqueda y filtrado de favores. Los usuarios pueden buscar favores disponibles aplicando filtros por modalidad (presencial o remoto) y por estado, lo que facilita encontrar rápidamente lo que necesitan.

Imagen corporativa

Nombre

El nombre de la aplicación es corto, descriptivo e intuitivo. Cualquier persona que lo lea por primera vez entiende inmediatamente para qué sirve el producto. Además, es fácilmente pronunciable en español e inglés, lo que facilita su proyección internacional a medio plazo.

Logotipo

El logotipo combina la silueta de dos manos que se estrechan (símbolo universal del acuerdo y la colaboración) integrada en el icono redondeado propio de una aplicación móvil moderna. La tipografía es de palo seco, de trazo limpio y equilibrado, que transmite modernidad y accesibilidad sin resultar fría ni impersonal.

Eslogan

" Hoy por ti, mañana por mí" Este es el eslogan donde refleja el valor fundamentales del producto que es la predisposición a ayudar. Es memorable, positivo y diferencial respecto a cualquier competidor.

Colores corporativos

Se adopta el verde esmeralda como color principal, asociado al crecimiento, la confianza y el bienestar comunitario. Se combina con el blanco para los fondos, aportando limpieza y claridad visual, y con un naranja cálido para los elementos de llamada a la acción (botones, alertas y notificaciones), que transmite energía y cercanía. Esta paleta es coherente con los valores de la empresa y con el perfil del público objetivo.

10.4.3. c) Estrategias de precio

Factores tenidos en cuenta para la fijación del precio

Costes de operación

Los costes fijos mensuales del proyecto ascienden a 115 €, lo que condiciona directamente el número mínimo de suscriptores de pago necesario para alcanzar el umbral de rentabilidad.

Análisis de la competencia

Las plataformas de servicios entre particulares como TaskRabbit o Etece aplican comisiones de entre el 15 % y el 30 % sobre el importe del servicio. Nextdoor y los grupos de WhatsApp son completamente gratuitos pero sin ninguna estructura de gestión. La aplicación se posiciona entre ambos extremos: acceso gratuito para las funciones esenciales y un precio Premium bajo para quienes deseen funcionalidades avanzadas.

Demanda y perfil del cliente

El público objetivo incluye segmentos con capacidad adquisitiva limitada (estudiantes, personas mayores) junto a otros con mayor disposición a pagar (autónomos, familias con poco tiempo). Esta diversidad obliga a adoptar un modelo accesible para todos en su nivel básico y con una opción de pago atractiva para usuarios intensivos.

Ciclo vital del producto

En la fase de lanzamiento el objetivo prioritario no es la rentabilidad inmediata, sino la captación de usuarios y la construcción de una comunidad activa. Por este motivo se adopta en primer lugar una estrategia de penetración mediante acceso gratuito, reservando la monetización para la fase de consolidación.

Modelo de precios y tabla de tarifas

Se aplica un modelo freemium, con un nivel gratuito completo para el uso básico y un nivel de pago para funcionalidades avanzadas:

Plan	Precio	Contenido incluido
Plan Básico	0 € / mes	• Publicación y aceptación de favores sin límite • Chat privado con cada usuario • Perfil completo con foto, descripción y valoraciones

Plan	Precio	Contenido incluido
		• Unirse a grupos existentes • Notificaciones automáticas
Plan Premium	5 € / mes	• Todo lo del Plan Básico • Posibilidad de destacar los favores publicados para aumentar su visibilidad • Creación de grupos ilimitados • Acceso prioritario a las nuevas funcionalidades antes de su lanzamiento general
Comisión por pago gestionado	5 % del importe acordado	Aplicable únicamente cuando el usuario decida gestionar el cobro del favor a través de la propia aplicación. Esta funcionalidad se implementará en una fase posterior del desarrollo

Con el precio fijado en 5 € mensuales para el Plan Premium, basta con que 23 usuarios se suscriban para cubrir íntegramente los costes fijos del mes, lo que refleja la eficiencia del modelo de negocio y la baja estructura de costes del proyecto.

10.4.4. d) Estrategias de distribución

Canal de distribución elegida es la distribución directa al 100%

La aplicación llega al usuario final a través de canales digitales directos, sin ningún intermediario comercial:

Google Play Store

Para usuarios de dispositivos Android. La descarga es gratuita para el usuario. El registro como desarrollador supone un pago único de aproximadamente 25 €, ya recogido en el presupuesto de gastos iniciales.

Apple App Store

Para usuarios de iPhone y iPad con iOS. La descarga es igualmente gratuita para el usuario. Apple exige el pago de una suscripción anual de aproximadamente 95 €, también contemplada en los gastos iniciales.

Web oficial del proyecto

La versión de escritorio para Windows estará disponible para descarga directa desde la página web oficial, que también servirá como punto de información, contacto con soporte y acceso al blog de contenidos.

Estrategia de distribución geográfica progresiva

Fase 1 Lanzamiento local

Durante el primer mes, la aplicación se prueba en un entorno reducido y de confianza (amigos, familia y conocidos del promotor en el entorno de Cuenca). El objetivo es detectar errores, mejorar la experiencia de uso y generar las primeras valoraciones reales antes de abrir el registro al público general.

Fase 2 Crecimiento local orgánico

Una vez validado el funcionamiento, se abre el registro para todos los usuarios y se activan las acciones de comunicación. El objetivo es consolidar una primera comunidad activa de entre 100 y 500 usuarios en el área de lanzamiento.

Fase 3 Expansión progresiva

Con una comunidad activa y valoraciones positivas que actúen como prueba social, se replica la misma estrategia en nuevas localidades, apoyándose en el crecimiento orgánico y en campañas de publicidad digital segmentadas por zona geográfica.

Efectos del modelo de distribución elegido sobre el producto

La distribución directa supone que la plataforma mantiene el control total sobre cómo se presenta el producto al usuario: no hay ningún intermediario que pueda alterar el mensaje, añadir costes o modificar la experiencia de descarga. El principal inconveniente es que toda la responsabilidad de la captación y la visibilidad recae sobre el propio emprendedor, lo que se compensa con la inversión en ASO y en la estrategia de contenidos.

Intermediarios

No se utilizarán intermediarios comerciales en ninguna fase del proceso de distribución. Las tiendas de aplicaciones actúan como escaparates tecnológicos que facilitan la descarga, pero no intervienen en la relación entre la plataforma y el usuario final.

10.4.5. e) Atención al cliente

La atención al cliente es, en este proyecto, un pilar estratégico y no un mero servicio de apoyo. Dado que la plataforma opera en el terreno de la confianza entre desconocidos, cualquier incidencia mal gestionada puede dañar de forma irreparable la reputación del producto. Por

este motivo se diseña desde el inicio un sistema de atención sencillo, directo y eficaz, asumido en su totalidad por el propio promotor durante la fase de lanzamiento.

Soporte por correo electrónico

Se habilitará una dirección de correo oficial de soporte a la que cualquier usuario podrá dirigir sus consultas, sugerencias o reclamaciones. El promotor se compromete a responder en un plazo máximo de 24 horas en días laborables.

Sección de preguntas frecuentes (FAQ)

Se redactará una sección de preguntas y respuestas frecuentes accesible directamente desde la aplicación y desde la web oficial, organizada por categorías (registro y perfil, publicación de favores, chat, grupos, pagos y privacidad).

Notificaciones de incidencias técnicas

En caso de producirse una interrupción del servicio, se enviará una notificación dentro de la propia aplicación informando del problema y del tiempo estimado de resolución.

Política de privacidad y protección de datos

Se redactará un documento de privacidad claro y accesible, en lenguaje sencillo, que el usuario deberá aceptar en el momento del registro. En él se detalla qué datos se recogen, para qué finalidad, durante cuánto tiempo se conservan y cómo puede el usuario ejercer sus derechos conforme al Reglamento General de Protección de Datos (RGPD). Cualquier usuario podrá solicitar la eliminación completa de su cuenta y sus datos en cualquier momento.

Canal de resolución de conflictos entre usuarios

Se habilitará un canal específico a través del cual ambas partes de un conflicto pueden exponer su versión al administrador de la plataforma. El administrador podrá revisar el historial de la conversación del favor en cuestión, retirar valoraciones manifiestamente injustificadas y, en casos de conducta abusiva o reiterada, suspender o bloquear la cuenta del usuario infractor.

Actualizaciones periódicas del producto

La aplicación recibirá actualizaciones regulares con correcciones de errores, mejoras de rendimiento y nuevas funcionalidades, gratuitas tanto para el Plan Básico como para el Plan Premium.

Mejora continua basada en el feedback de los usuarios

A través de los comentarios recibidos por soporte y de las valoraciones publicadas en las tiendas de aplicaciones, el promotor recopilará información de forma continua sobre qué funciona bien y qué puede mejorarse, priorizando así el desarrollo de nuevas funcionalidades en función de las necesidades reales de los usuarios.

10.5. RECURSOS HUMANOS

10.5.1. a) Organigrama de la empresa



El promotor actúa como gerente y asume directamente las áreas de desarrollo y mantenimiento de la plataforma, marketing y comunicación, y atención al cliente.

10.5.2. b) Diseño de puestos y selección de personal

A continuación se presentan los dos profesiogramas correspondientes a los puestos de trabajo definidos para este proyecto, elaborados a partir del Convenio Colectivo del Metal para la provincia de Cuenca. El primero corresponde al promotor en su condición de Jefe de Área y Servicios, que es el puesto que ocupa actualmente.

PROFESIOGRAMA

PUESTO DE TRABAJO

Jefe de áreas y servicios

DESCRIPCIÓN DEL PUESTO

CATEGORÍA PROFESIONAL	Personal técnico	ACTIVIDAD DE LA CATEGORÍA PROFESIONAL	Personal que, con o sin subordinados a su cargo, ejerce funciones de planificación, organización y control de un área o servicio de la empresa, con autonomía y responsabilidad plena sobre los resultados de dicha área.
TAREAS DEL PUESTO DE TRABAJO	Planificar el desarrollo técnico de la plataforma y establecer las prioridades de cada fase, supervisar el mantenimiento, la actualización y el correcto funcionamiento del sistema, diseñar y ejecutar las acciones de marketing digital: redes sociales, blog, posicionamiento en tiendas de aplicaciones, gestionar la atención al cliente: responder consultas, resolver incidencias y conflictos entre usuarios, llevar la gestión económica básica: facturación, control de costes, presentación de impuestos trimestrales, gestionar las relaciones con los proveedores (tiendas de aplicaciones, proveedor de internet, registro de marca) y representar a la empresa ante organismos públicos y privados.		

PERFIL PROFESIONAL

TITULACIÓN	Técnico Superior en Desarrollo de Aplicaciones Multiplataforma (DAM)
CONOCIMIENTOS	Servicios y procesos, acceso a datos, desarrollo de interfaces de escritorio y móviles, seguridad y alta disponibilidad, empresa e iniciativa emprendedora, formación y orientación laboral
ACTITUDES PERSONALES	Autonomía y capacidad de autogestión, responsabilidad y compromiso personal, capacidad de comunicación y orientación al cliente, liderazgo y toma de decisiones, adaptabilidad al cambio, orientación al mercado y al usuario, perfeccionismo y exigencia en la calidad del producto
EXPERIENCIA	Experiencia en desarrollo de software tanto para móviles como de escritorio, creación de APIs, gestión del cumplimiento normativo, patrones de arquitectura del software.

CONDICIONES LABORALES

GRUPO PROFESIONAL	CATEGORÍA PROFESIONAL	GRUPO SALARIAL	SALARIO BASE	COMPLEMENTO SALARIAL (Calidad y Cantidad)	CONTRATO	TIPO DE JORNADA
Grupo A	Personal técnico	3-A	2030,73	2030,73	Contrato Indefinido	Completa (40 horas semanales)

10.5.3. c) Plan de motivación laboral

Aunque la plantilla en la fase inicial es de una sola persona contratada, la motivación laboral se considera un factor estratégico desde el primer momento. Las medidas concretas adoptadas son las siguientes:

Integración real en el proyecto

La persona contratada no realizará tareas mecánicas y repetitivas, sino que participará activamente en la mejora del producto: sus observaciones durante las pruebas de funcionamiento y su contacto directo con los usuarios le permitirán aportar ideas y propuestas que el promotor valorará y, en su caso, incorporará al desarrollo. Esta implicación directa en la evolución del producto genera sentido de pertenencia y compromiso.

Horario flexible y teletrabajo

Dado que la empresa opera íntegramente en formato digital, la jornada se organiza con flexibilidad horaria, con un marco de referencia de mañana (9:00-14:00) y tarde (15:00-18:00), pero con autonomía para adaptar los horarios a las necesidades personales de la persona trabajadora, siempre que se garantice la disponibilidad en las horas de mayor actividad de los usuarios. El teletrabajo es la modalidad principal, lo que elimina desplazamientos y favorece la conciliación.

Formación continua

La empresa asumirá el coste de cursos, recursos online y materiales formativos que mejoren las competencias técnicas de la persona contratada, especialmente en áreas relacionadas con el proyecto (diseño de interfaces, atención al cliente digital, herramientas de comunicación).

Reconocimiento del trabajo bien hecho

Se establecerá una revisión trimestral informal del desempeño, en la que el promotor compartirá con la persona contratada los datos de evolución del proyecto (nuevos usuarios, valoraciones, incidencias resueltas) y reconocerá públicamente su contribución. Si el proyecto alcanza los objetivos de rentabilidad previstos, se contempla la revisión del salario al alza a partir del segundo año.

Buen ambiente de trabajo

Al tratarse de un equipo muy reducido, la comunicación es directa, transparente y sin burocracia. Se fomenta un trato de confianza mutua y se garantiza que la persona contratada siempre recibirá feedback claro sobre su trabajo, tanto positivo como constructivo.

10.5.4. d) Contratación y coste de los trabajadores

Nómina del Profesional de Oficio de 1ª

Ver [ANEXO II NOMINA](#) al final del documento.

Costes del personal

TRABAJADOR DE LA EMPRESA	PUESTO DE TRABAJO	TIPO DE CONTRATO	Nº pax	SALARIO MENSUAL (A x B)	SEGURIDAD SOCIAL MENSUAL (A x C)	SALARIO ANUAL (B x 12)	SEGURIDAD SOCIAL ANUAL (C x 12)	BONIFICACIÓN Y REDUCCIONES A LA SEGURIDAD SOCIAL 12 meses	COSTE TOTAL CON BONIFICACIÓN Autónomo (D + F) Contratados (D + E) - F
Promotor / Autónomo	Jefe de Área y Servicios	Gerente	1	1.367,14	80	16.405,68 €	960	960,00(Tarifa CLM)	17.365,68 €
Personal Contratado	Profesional de Oficio de 1ª	Formativo para la Práctica	1	1.367,14 €	106,63 €	16.405,68 €	1.279,56 €	0	17.685,24 €
TOTAL			2	2.734,28 €	186,63 €	32.811,36 €	2.239,56 €	960	35.050,92 €

10.5.5. e) Organización de la prevención.

MODALIDAD ELEGIDA PARA ORGANIZAR Y GESTIONAR LA PREVENCIÓN DE REISGOS LABORALES	REQUISITOS
Servicio de Prevención Ajeno (SPA)	• Empresa de menos de 250 trabajadores, por lo que no está obligada a constituir un servicio de prevención propio • La actividad preventiva no puede asumirse íntegramente con medios propios del empresario, ya que el promotor no cuenta con la cualificación técnica requerida en las cuatro disciplinas preventivas (medicina del trabajo, seguridad en el trabajo, higiene industrial y ergonomía y psicología aplicada) • Se contratará con una entidad especializada acreditada que disponga del personal con cualificación de nivel superior en cada una de las disciplinas preventivas exigidas por la normativa. • Dicha entidad deberá contar con las instalaciones y equipos necesarios para realizar evaluaciones, mediciones, reconocimientos médicos y actividades formativas • Antes de formalizar el concierto con el SPA, el empresario consultará a los delegados de prevención conforme a lo establecido en la Ley de Prevención de Riesgos Laborales. El SPA asumirá directamente todas las funciones preventivas concertadas, sin posibilidad de delegarlas. • Esta modalidad es la que mejor se adapta al proyecto por ser la más sencilla de constituir, la más económica para empresas de arranque y la que garantiza desde el primer día el cumplimiento legal sin necesidad de formar ni designar trabajadores propios para la prevención.

10.6. FORMA JURÍDICA.

10.6.1. a) Forma jurídica y motivos por los que se ha elegido.

FORMA JURÍDICA	Sociedad de Responsabilidad Limitada Unipersonal
CARACTERÍSTICAS	
Razón social	Favsy
Objeto social	Desarrollo, gestión, mantenimiento y explotación de una plataforma tecnológica (aplicación móvil y web) para la intermediación y gestión de servicios colaborativos y favores entre usuarios.
Domicilio social	C/ Fuensanta 3, 16002 Cuenca
Promotor /Socios	Cristian Guerrero Terciado (Participación del 100% del Capital Social)
Responsabilidad social	Limitada. La responsabilidad está limitada al capital aportado a la sociedad.
Capital social	4.000 € (Desembolsado e inmovilizado íntegramente en cuenta bancaria corporativa)
Régimen fiscal	Impuesto sobre Sociedades (tributación de los beneficios de la empresa) e IVA (declaraciones trimestrales).
JUSTIFICACIÓN DE LA ELECCIÓN	
• Protección del patrimonio personal: Al ser una app donde los usuarios interactúan y realizan servicios (favores), existe el riesgo de conflictos o reclamaciones. La S.L. limita la responsabilidad al capital de la empresa, salvaguardando tus bienes personales. • Imagen profesional: Operar como Sociedad Limitada proyecta una imagen de mayor solidez, credibilidad y permanencia frente a posibles inversores, instituciones para pedir subvenciones o grandes clientes, en comparación con la figura del autónomo. • Fiscalidad: Si la aplicación tiene éxito y los beneficios empiezan a ser altos, el Impuesto sobre Sociedades (tipo fijo, generalmente del 25%, o 15% para empresas de nueva creación) resulta más rentable que los tramos altos del IRPF como autónomo.	

10.7. ANÁLISIS ECONÓMICO-FINANCIERO.

10.7.1. a) Análisis de Costes

El análisis de costes permite identificar y clasificar todos los desembolsos que genera la actividad, distinguiendo entre aquellos que dependen directamente del nivel de producción o ventas (costes variables) y aquellos que se producen con independencia de los ingresos obtenidos (costes fijos).

Al tratarse de un producto puramente digital, la estructura de costes del proyecto es inusualmente ligera, lo que constituye una de sus principales ventajas competitivas. No existen materias primas, ni almacenes, ni vehículos, ni gastos de aprovisionamiento en el sentido convencional. La totalidad de los costes se concentra en tres bloques: la cuota del promotor como autónomo, los suministros domésticos imputados a la actividad y los costes de presencia digital.

Costes fijos mensuales

La cuota del Régimen Especial de Trabajadores Autónomos (RETA) en los primeros doce meses de actividad, acogida a la tarifa plana para nuevos autónomos establecida por el Real Decreto-Ley 13/2022, asciende a 80 €/mes. La factura de la conexión de fibra óptica al domicilio se imputa en su parte proporcional a la actividad en 20 €/mes. El consumo eléctrico atribuible al servidor doméstico y al puesto de trabajo se estima en 15 €/mes. La suscripción anual a Apple App Store (95 €/año) equivale a aproximadamente 8 €/mes. No existe alquiler de local. El total de costes fijos mensuales asciende a 123 €/mes en el primer año.

Costes variables

Al no existir producción física ni aprovisionamiento, los únicos costes variables identificados son las posibles campañas de publicidad digital de pago puntual, que no se contemplan en la fase de lanzamiento, y la comisión de las tiendas de aplicaciones sobre los ingresos Premium (un 30% en Google Play y App Store sobre cada suscripción cobrada a través de las mismas, aunque se puede minimizar ofreciendo la suscripción directamente desde la web).

Punto de equilibrio (umbral de rentabilidad)

Con unos costes fijos de 123 €/mes y un precio del Plan Premium de 5 €/mes por usuario, el número mínimo de suscriptores de pago necesario para cubrir los costes sin aplicar la comisión de las tiendas es de 25 usuarios Premium, cifra extremadamente asequible que confirma la viabilidad del modelo de negocio incluso en sus fases más tempranas.

10.7.2. b) Inversiones y gastos iniciales

BIENES DE INVERSIÓN	COSTE			FUENTE DE FINANCIACIÓN
	Alquiler		Compra	
	€ / mes	€ / año		
Edificio / Local (Anexo)	300	3600	0	Contrato de arrendamiento con el socio (operación vinculada a precio de mercado).
Reformas / Acondicionamiento del local	0	0	0	No aplica
Maquinaria / Equipos	0	0	0	No aplica
Herramientas / Utillaje	0	0	0	No aplica
Mobiliario (Anexo)	0	0	150,00 €	Aportación de Socios (Capital Propio)
Equipos informáticos (Anexo)	0	0	550,00 €	Ahorros propios. Ordenador y periféricos ya disponibles, amortización parcial asignada al proyecto.
Vehículos	0	0	0	No aplica
TOTAL BIENES DE INVERSIÓN	300	3600	700	No aplica

El proyecto no requiere la adquisición en propiedad de un local, maquinaria ni vehículo. Para el desarrollo de la actividad, se formaliza un contrato de alquiler por un espacio específico del inmueble del promotor a un precio de mercado de 300 € mensuales, evitando contingencias fiscales. Los únicos bienes de inversión que se adquieren en propiedad para el inicio de la actividad son el mobiliario de oficina y los equipos informáticos necesarios para la gestión de la plataforma digital.

GASTOS INICIALES	COSTE	
	€ / mes	€ / año
Constitución de la empresa	0	0
Capital social mínimo exigido legalmente	0	3000
Reservas (dinero primeros meses)	0	500
Seguros, (local, vehículos, etc.)	0	0
Publicidad	0	25
Servicios externos, (gestoría, Registro de dominio web, Google Play Store (registro desarrollador), Apple App Store (suscripción anual), Registro de marca en la OEPM)	0	287
Suministros, (agua, luz, teléfono, etc.)	35	420
TOTAL GASTOS INICIALES		4232

10.7.3. c) Plan de financiación

TOTAL BIENES DE INVERSIÓN	700
TOTAL GASTOS INICIALES	4.232,00
INVERSIÓN NECESARIA	4.932,00

FINANCIACIÓN DE INVERSIONES Y GASTOS INICIALES	
FINANCIACIÓN PROPIA	4.932,00
Ahorros personales.	4.932,00
Préstamos familiares.	0
FINANCIACIÓN AJENA	5000
AYUDAS Y SUBVENCIONES(PREVISIÓN TRAMITACIÓN)	0
<u>Ayuntamiento de Cuenca.</u>	0
<u>Diputación de Cuenca.</u>	0
<u>Junta de comunidades de Castilla-La Mancha.</u>	0
ü Creación.	0
ü Inversión.	0
ü Financiación.	0
<u>Gobierno de España.</u>	0
PRÉSTAMOS	0
<u>Préstamo bancario</u>	0
<u>Líneas ICO</u>	0
FINANCIACIÓN NECESARIA	4.932,00

La decisión de financiar el proyecto íntegramente con ahorros propios es coherente con la filosofía de mínima inversión que define el modelo de negocio. No se solicita préstamo bancario alguno porque la baja inversión inicial no justifica los costes financieros asociados y porque mantener la empresa libre de deuda desde el primer día proporciona una mayor solidez y flexibilidad para adaptarse a la evolución del mercado. No se solicitan ayudas ni subvenciones en esta fase inicial, aunque se contempla explorar las convocatorias de la Junta de Comunidades de Castilla-La Mancha para jóvenes emprendedores una vez que el proyecto esté operativo y genere sus primeros ingresos, ya que estas ayudas están orientadas precisamente a consolidar proyectos en marcha más que a financiar el arranque.

AMORTIZACIÓN DEL PRÉSTAMO

CONDICIONES DEL PRÉSTAMO	
Entidad financiera	Banco de Cuenca
Capital solicitado	5000
<u>Tipo de interés</u> (i) (Euribor + Diferencial)	5%

	A	B	C	D	E
Período	Cuota anual	Interés anual ($D \times i$)	Capital devuelto ($A - B$)	Capital pendiente ($D - C$)	Capital Amortizado ($C_{1,2,3..} + E_{1,2,3,..}$)
0	0	0		5000	0
1	1836,04	250	1586,04	3143,96	1588,04
2	1836,04	170,7	1665,34	1748,62	3251,38
3	1836,04	87,42	1748,62	0	5000

AMORTIZACIÓN DE LOS BIENES DE INVERSIÓN

BIENES DE INVERSIÓN (<u>Adquiridos en propiedad, los alquilados NO</u>)	Valor de adquisición (A)	Coefficiente lineal máximo (B)	Cuota de amortización anual ($A \times B$)
Edificio / Local	0	0	0
Reformas / Acondicionamiento del local	0	0	0
Maquinaria / Equipos	0	0	0
Herramientas / Utillaje	0	0	0
Mobiliario	150	10%	15
Equipos informáticos	550	25%	137,5
Vehículos	0	0	0
TOTAL	700		152,5

Los coeficientes de amortización aplicados son los establecidos en las tablas de amortización simplificada aprobadas por la Agencia Tributaria para trabajadores autónomos en estimación directa simplificada: el 10% anual para mobiliario (vida útil de 10 años) y el 25% anual para equipos informáticos (vida útil de 4 años)

10.7.4. d) Plan de tesorería

ENTRADAS	Enero	Febrero	Marzo	Abril	Mayo	Junio	Julio	Agosto	Septiembre	Octubre	Noviembre	Diciembre	TOTAL
Ahorro personal / Préstamo familia	4.932	0	0	0	0	0	0	0	0	0	0	0	4.932
Préstamos	5000	0	0	0	0	0	0	0	0	0	0	0	5000
Ventas de bienes/servicios	200	300	300	300	500	500	500	700	700	700	700	700	6100
Ayudas y subvenciones	0	0	0	0	0	0	0	0	0	0	0	0	0
Intereses de c/c e inversiones	0	0	0	0	0	0	0	0	0	0	0	0	0
TOTAL ENTRADAS	10132	300	300	300	500	500	500	700	700	700	700	700	16032

SALIDAS	Enero	Febrero	Marzo	Abril	Mayo	Junio	Julio	Agosto	Septiembre	Octubre	Noviembre	Diciembre	TOTAL
Devolución de préstamo e intereses	0	0	0	0	0	0	0	0	0	0	0	1836,04	1836,04
Alquileres (local, vehículos, etc.)	300	300	300	300	300	300	300	300	300	300	300	300	3600
Maquinaria y herramientas (propiedad)	0	0	0	0	0	0	0	0	0	0	0	0	0
Vehículos (propiedad)	0	0	0	0	0	0	0	0	0	0	0	0	0
Mobiliario (propiedad)	150	0	0	0	0	0	0	0	0	0	0	0	150
Equipos informáticos (propiedad)	550	0	0	0	0	0	0	0	0	0	0	0	550
Reformas y acondicionamientos	0	0	0	0	0	0	0	0	0	0	0	0	0
Gastos de constitución	1850	0	0	0	0	0	0	0	0	0	0	0	1850
Seguros (local y vehículo en propiedad)	0	0	0	0	0	0	0	0	0	0	0	0	0
Publicidad	25	0	0	0	0	0	0	0	0	0	0	0	25
Compra de mercaderías	0	0	0	0	0	0	0	0	0	0	0	0	0
Servicios externos (gestoría, google play + apple app store, registro de dominio + marca OEPM)	287	60	60	60	60	60	60	60	60	60	60	60	947
Personal (salarios, seg.social)	80	80	80	80	80	80	80	80	80	80	80	80	960
Pagos a cuenta del IRPF o del IS	0	0	0	0	0	0	0	0	0	0	0	0	0
Suministros (agua, telefono, luz etc.)	35	35	35	35	35	35	35	35	35	35	35	35	420
TOTAL SALIDAS	3277	475	475	475	475	475	475	475	475	475	475	2311,04	10338,04
RESULTADO (Entradas - Salidas)	6855	-175	-175	-175	25	25	25	225	225	225	225	-1611,04	
SALDO EN CAJA	6855	6680	6505	6330	6355	6380	6405	6630	6855	7080	7305	5693,96	

Análisis del resultado del plan de tesorería

El saldo en caja se mantiene en valores positivos durante todos los meses del año, lo que confirma que la empresa dispone de liquidez suficiente para afrontar sus obligaciones de pago en todo momento. El sólido colchón financiero obtenido en enero (6.855 €) gracias a la financiación inicial permite absorber sin dificultades los meses con flujos mensuales negativos (febrero, marzo y abril), periodos en los que el saldo desciende de forma controlada hasta un mínimo intermedio de 6.330 € en abril debido al peso del nuevo coste de alquiler. A partir de mayo, gracias al crecimiento progresivo en la facturación por la venta de servicios, la tesorería inicia una recuperación constante y sostenida mes a mes, alcanzando su punto máximo en noviembre con un saldo acumulado de 7.305 €. Finalmente, en el mes de diciembre se asume con total solvencia el desembolso de la primera cuota anual de amortización del préstamo e intereses (1.836,04 €), cerrando el primer ejercicio económico con una excelente salud financiera y un remanente final en caja de 5.693,96 €. En consecuencia, queda demostrado que el proyecto no presenta riesgos de impago ni tensiones de liquidez, consolidando una base operativa muy estable durante su lanzamiento.

10.7.5. e) Previsión de resultados y balance de situación.

(A) INGRESOS DE EXPLOTACIÓN			(B) GASTOS DE EXPLOTACIÓN	7954,5
Ventas		6100	Amortización de inversiones	152,5
			Alquileres (local, vehículo, mobiliario...)	3600
			Gastos de constitución	1.850
			Seguros, (local, vehículos adquiridos...)	0
			Publicidad	25
			Compra de mercaderías/materia prim	0
			Servicios externos (gestoría, google play + apple app store, registro de dominio + marca OEPM))	947
			Personal, (Salarios, Seg. Social)	960

		Pagos a cuenta del IRPF o del IS	
		Suministros (agua, teléfono, gas, etc.)	420
(C) INGRESOS FINANCIEROS		(D) GASTOS FINANCIEROS	
Intereses de c/c e inversiones	0	Intereses de préstamos	250,00
Resultado de explotación (E) = (A – B)		-1854,50	
Resultado financiero (F) = (C – D)		-250	
RESULTADO ANTES DE IMPUESTOS (G) (E + F)		-2104,50	
Impuesto sobre Sociedades (H) (G x 15%)		0	
Pagos a cuenta del IRPF o del I.S (I)		0	
Liquidación del I.S (J) = (H - I)		0	
RESULTADO FINAL (K) = (G + H)		-2104,50	

Análisis de la cuenta de resultados

El primer ejercicio arroja unas pérdidas de 2.104,50 €, resultado absolutamente normal y esperado en cualquier proyecto de nueva creación. Como se indicó en el análisis del plan de tesorería, la estrategia de lanzamiento prioriza la captación de usuarios sobre la rentabilidad inmediata. Lo más relevante es que estas pérdidas están ya previstas y cubiertas por el colchón de ahorros iniciales: el promotor inicia la actividad sabiendo que el primer año no será rentable debido al peso de los gastos de constitución y el nuevo coste del alquiler, habiendo dimensionado su inversión para soportar este escenario sin comprometer la viabilidad del proyecto. El segundo año, con una base de usuarios más consolidada y una mayor proporción de suscriptores Premium, se prevé que el resultado sea positivo o próximo al equilibrio.

BALANCE PATRIMONIAL

ACTIVO			PASIVO		
Muestra en qué ha invertido la empresa el dinero del pasivo. (Bienes y derechos en propiedad)			Muestra de dónde obtiene el dinero la empresa para financiar el activo. (Deudas a devolver)		
			PATRIMONIO NETO	2.827,50	
			Capital social mínimo exigido	4.932,00	
			Cuenta de resultados (+) si ganancias, (-) si pérdidas	-2104,50	
			Aportación complementaria / Reservas socios	0	
ACTIVO CORRIENTE	(Derechos permanecen en la empresa menos de 1 año)	5.693,96	PASIVO CORRIENTE	(Deudas a devolver en menos de 1 año)	0
Existencias.		0,00	Créditos pendientes de devolución.		0
Facturas de clientes pendientes de cobro.		0	Deudas pendientes a proveedores.		0
Efectivo en Bancos, en caja.		5.693,96	Deudas pendientes a acreedores (Seg.Soc, Hacienda)		0
ACTIVO NO CORRIENTE	(Bienes permanecen en la empresa más de 1 año)	547,5	PASIVO NO CORRIENTE	(Deudas a devolver en más de 1 año)	3.413,96

Bienes de inversión (local, mobiliario, vehículo...)	700	Préstamo pendiente de devolución.	3.413,96
Amortización de inversiones (con signo “-“)	-152,50		
TOTAL ACTIVO	6.241,46	TOTAL PASIVO	6.241,46

Análisis del balance patrimonial

Sólida Liquidez Inmediata: El 91% del activo total de la empresa se encuentra de forma líquida en dinero en efectivo (5.693,96 €). Esto demuestra una capacidad de reacción excepcional para afrontar cualquier imprevisto técnico o comercial a corto plazo durante el inicio de la actividad.

Estructura Financiera y Solvencia: El Patrimonio Neto se sitúa en 2.827,50 € debido a que el Capital Social inicial absorbe las pérdidas lógicas del primer año de lanzamiento (-2.104,50 €). A pesar de este resultado negativo derivado de los costes operativos del nuevo alquiler formalizado, la empresa mantiene una base patrimonial sólida y positiva.

Fondo de Maniobra Saludable: La diferencia entre el Activo Corriente (5.693,96 €) y el Pasivo Corriente (0,00 €) es ampliamente positiva. Existe un colchón de seguridad de casi 5.700 € en cuenta corriente para seguir operando con total tranquilidad, sin depender de financiación externa a corto plazo.

Deudas a Corto Plazo Nulas: El pasivo corriente se ha reducido a cero al no existir obligaciones tributarias pendientes por Impuesto de Sociedades al cierre del ejercicio. La única obligación financiera de peso de la sociedad corresponde al préstamo a largo plazo (3.413,96 €), el cual se encuentra plenamente respaldado por los fondos disponibles en el banco.

IMPUESTO SOBRE SOCIEDADES (IS)

<u>IMPUESTO SOBRE SOCIEDADES (I.S)</u>		
TIPO DE IMPUESTO	Impuesto directo: grava la riqueza en su origen, es decir, recae directamente sobre la persona o entidad que obtiene el beneficio, sin posibilidad de trasladarlo a un tercero.	
HECHO IMPONIBLE	La obtención de renta (beneficios) por parte de una persona jurídica (sociedad) durante el período impositivo, con independencia de su procedencia.	
CONTRIBUYENTE SUJETO PASIVO	/	Las personas jurídicas residentes en territorio español: sociedades anónimas, sociedades de responsabilidad limitada, cooperativas, asociaciones, fundaciones, etc. El trabajador autónomo (persona física) queda excluido de este impuesto y tributa por el IRPF. En este proyecto, al operar como autónomo, el IS no es de aplicación.
PERIODO IMPOSITIVO	El año natural (del 1 de enero al 31 de diciembre), salvo que el ejercicio social de la sociedad sea diferente.	
TIPO DE GRAVAMEN	Tipo general: 25% sobre la base imponible. Tipo reducido para empresas de nueva creación: 15% durante el primer período impositivo con base imponible positiva y el siguiente.	
PAGOS A CUENTA O PAGOS FRACCIONADOS IS	Se realizan en los meses de abril, octubre y diciembre. El tipo aplicable en la modalidad 1 (sobre la cuota íntegra del último IS declarado)	18%
DECLARACIÓN (Liquidación)	ANUAL	Se presenta en el plazo de 25 días naturales siguientes a los 6 meses posteriores al cierre del ejercicio, es decir, entre el 1 y el 25 de julio del año siguiente.

IMPUESTO SOBRE EL VALOR AÑADIDO, (IVA).

IMPUESTO SOBRE EL VALOR AÑADIDO (I.V.A) (Común para el empresario individual y para las sociedades)	
TIPO DE IMPUESTO	Impuesto indirecto: no grava la renta del sujeto, sino el consumo. Recae en última instancia sobre el consumidor final, aunque quien lo recauda y lo ingresa en Hacienda es el empresario o profesional que realiza la operación.
HECHO IMPONIBLE	Las entregas de bienes y prestaciones de servicios realizadas por empresarios o profesionales en el ejercicio de su actividad, así como las adquisiciones intracomunitarias de bienes y las importaciones.
SUJETO PASIVO CONTRIBUYENTE	/ El empresario o profesional que realiza las operaciones gravadas. En este proyecto, el promotor autónomo actúa como sujeto pasivo y está obligado a repercutir el IVA en sus facturas y a liquidarlo trimestralmente con la Agencia Tributaria.
EXENCIONES	Están exentas de IVA, entre otras, las actividades sanitarias, educativas, financieras y de seguros, así como la entrega de sellos y la mayor parte de operaciones de vivienda habitual. Las aplicaciones digitales y las suscripciones a servicios digitales no están exentas y tributan al tipo general.
BASE IMPONIBLE	El importe total de la contraprestación de las operaciones sujetas al impuesto, sin incluir el propio IVA. En este proyecto, la base imponible es el precio de las suscripciones Premium (5 €/mes por usuario) y, en su caso, las comisiones por pagos gestionados.
TIPO IMPOSITIVO	Tipo general del 21%, aplicable a los servicios digitales y suscripciones online. Tipo reducido del 10% para determinados bienes y servicios (alimentación, transporte, hostelería). Tipo superreducido del 4% para bienes de primera necesidad (pan, leche, libros, medicamentos). Los servicios digitales de la plataforma tributan al 21%.
REGÍMENES DE TRIBUTACIÓN	Régimen general: el sujeto pasivo repercute el IVA en sus facturas (IVA devengado) y deduce el IVA soportado en sus compras y gastos relacionados con la actividad. La diferencia se ingresa o se compensa trimestralmente. Es el régimen aplicable a este proyecto. Régimen especial simplificado: aplicable a determinadas actividades y autónomos con módulos; no aplica a este proyecto. Régimen especial del recargo de equivalencia: aplicable a comerciantes minoristas; no aplica a este proyecto.
DECLARACIÓN LIQUIDACIÓN	/ Se presenta trimestralmente mediante el Modelo 303, en los siguientes plazos: 1er trimestre (enero-marzo) → del 1 al 20 de abril; 2º trimestre (abril-junio) → del 1 al 20 de julio; 3er trimestre (julio-septiembre) → del 1 al 20 de octubre; 4º trimestre (octubre-diciembre) → del 1 al 30 de enero del año siguiente.
RESUMEN ANUAL	Se presenta mediante el Modelo 390 durante el mes de enero del año siguiente al ejercicio liquidado.

CÁLCULO DEL IMPUESTO SOBRE LA RENTA DE LAS PERSONAS FÍSICAS (IRPF)**PAGOS FRACCIONADOS (I.S)**

PERIODO	MESES	VENTAS (A)	SALIDAS (B)	RENDT. NETO (A – B)	TIPO (C)	CUOTA (A – B) x C	PAGO
1º Trimestre	Ene - Feb - Mar	800	4227	-3427	20%	0	0
2º Trimestre	Abr - May - Jun	1300	1425	-125		0	0
3º Trimestre	Jul - Ago - Sep	1900	1425	475		95	95
4º Trimestre	Oct - Nov - Dic	2100	3261,04	-1161,04		0	0
PAGOS FRACCIONADOS						95	

DECLARACIÓN ANUAL (I.S)**CUOTA DIFERENCIAL**

RENDIMIENTO NETO (A)	TIPO (B)	CUOTA FINAL (A – B) x C
-2104,5	15%	0

CUOTA FINAL	PAGOS FRACCIONADOS	CUOTA DIFERENCIAL I.S	SIGNIFICADO DEL RESULTADO
0	95	-95	El rendimiento neto antes de impuestos del ejercicio es negativo (-2.104,50 €), generando una cuota de Impuesto sobre Sociedades de 0,00 €. Dado que a lo largo del año se han realizado pagos fraccionados por valor de 95,00 € (correspondientes al beneficio puntual obtenido durante el tercer trimestre), la cuota diferencial resulta negativa (-95,00 €). Esto significa que la empresa ha adelantado más dinero del debido debido al desfase temporal de ingresos y gastos, por lo que Hacienda le devolverá esos 95,00 € tras presentar la declaración anual del ejercicio.

10.8. PLAN DE PUESTA EN MARCHA

10.8.1. a) Trámites y plazos para la puesta en marcha.

TRÁMITES DE CONSTITUCIÓN SEGÚN LA FORMA JURÍDICA

Forma jurídica adoptada	Sociedad de Responsabilidad Limitada (S.R.L. / S.L.)	Denominación	Favsy			
Trámite	Descripción / Finalidad		Plazo	Organismo de tramitación	Documento (Modelo oficial)	Coste Estimado (Modelo oficial)
Alta Censal y Declaración de Inicio de Actividad	Comunicar a la Agencia Tributaria el inicio de la actividad económica, identificar el epígrafe del IAE correspondiente (Epígrafe 759.9: Otros servicios de intermediación y gestión digital), y declarar el régimen de IRPF. Es el único trámite constitutivo del autónomo: no requiere notario ni Registro Mercantil. A partir de este momento el promotor adquiere su personalidad jurídica como empresario y queda habilitado para emitir facturas y ejercer su actividad con plenas garantías legales.		Antes del inicio de la actividad.	Agencia Estatal de Administración Tributaria (AEAT) — Delegación de Cuenca o Sede Electrónica (agenciatributaria.es)	Modelo 037 (versión simplificada)	0
Solicitud de Nombre Comercial y Registro de Marca	Registrar el nombre de la aplicación como marca nacional en la clase 42 del Nomenclátor Internacional (servicios de programación informática y plataformas digitales SaaS), protegiendo el nombre comercial del proyecto frente al uso de terceros en todo el territorio nacional. Los efectos de la protección se retrotrae a la fecha de solicitud.		Antes del lanzamiento público de la aplicación.	Oficina Española de Patentes y Marcas (OEPM) — Sede Electrónica (oepm.es)	Formulario electrónico de solicitud de marca nacional (una clase)	155,06 € (primera clase del nomenclátor)

TRÁMITES DE PUESTA EN MARCHA
(Comunes para todas las formas jurídicas)

Forma jurídica adoptada	Sociedad de Responsabilidad Limitada	Denominación	Favsy	
Trámite	Descripción / Finalidad	Organismo de tramitación	Plazo	Documento / Modelo oficial
Alta en el Censo de Empresarios (Declaración Censal)	Notificar el inicio de la actividad económica e identificar las obligaciones tributarias periódicas del autónomo (IVA trimestral e IRPF).	Agencia Estatal de Administración Tributaria (AEAT).	Antes del inicio	Modelo 036
Inscripción de la Empresa en la Seguridad Social	Afiliarse a la Seguridad Social como trabajador por cuenta propia y solicitar la tarifa plana para nuevos autónomos (80 €/mes el primer año, conforme al RDL 13/2022).	Tesorería General de la Seguridad Social (TGSS).	Antes del inicio de la actividad	Modelo TA.6
Alta en el Régimen de Autónomos (RETA)	Afiliarse y darse de alta en la Seguridad Social como trabajador por cuenta propia, determinando la base de cotización y solicitando la aplicación de la cuota reducida para nuevos autónomos	Tesorería General de la Seguridad Social (TGSS).	Antes del inicio de la actividad	Modelo TA.0521/2
Afiliación y Alta de los Trabajadores	Registrar en el sistema de seguridad social	Tesorería General de la Seguridad Social (TGSS).	Previo al inicio	Modelo TA.1
Registro de Contratos de Trabajo	Comunicar formalmente los términos	Servicio Público de Empleo Estatal (SEPE) / Oficina de Empleo de Cuenca.	10 días siguientes	Aplicación Contrato
Comunicación Previa de Actividad al Ayuntamiento	Comunicar el ejercicio de una actividad económica de bajo impacto en el domicilio particular. Al ser actividad puramente digital sin atención presencial de clientes, no se exige licencia de apertura.	Ayuntamiento de Cuenca — Área de Urbanismo y Actividades (presencial o sede electrónica del Ayuntamiento)	Formulario de Declaración Responsable / Comunicación Previa de Actividad	Formulario oficial
Cumplimiento del RGPD y elaboración de la Política de Privacidad	Al tratar datos personales de los usuarios, el promotor está obligado a publicar la política de privacidad, mantener el Registro de Actividades de Tratamiento (RAT) y aplicar las medidas de seguridad adecuadas conforme al Reglamento (UE) 2016/679.	Agencia Española de Protección de Datos (AEPD) — aepd.es (consultas y herramienta de ayuda gratuita)	Antes de la apertura del registro de usuarios en la aplicación	Política de Privacidad + Registro de Actividades de Tratamiento (RAT) — documentos internos
Contratación e Implantación del Plan de Prevención	Organizar la estructura preventiva	Entidad acreditada como Servicio de Prevención Ajeno (SPA).	Antes del inicio	Plan de Prevención de Riesgos Laborales

10.8.2. b) Documentación administrativa-contable.

FACTURA	Nº Factura:	FAP-2025-001	Fecha:	25 de mayo de 2026	Nº Albarán	ALB-0001			
Proveedor:	Favsy	Condiciones de la compraventa							
NIF/CIF:	12345678K	Pago inmediato mediante cheque nominativo al portador.. Acceso inmediato a las funcionalidades Premium tras confirmación del pago. Soporte técnico incluido por correo en menos de 24 h.							
Dirección	C/ Fuensanta 3, 16002 Cuenca								
Ciudad:	Cuenca								
Cliente:	Pepe								
N.I.F/CIF:	12345678N								
Dirección	Calle mercadona nº 2, 3 C								
Ciudad:	Cuenca								
Código	Artículo	Unds	€/ Und	Importe	Descuento	Tipo IVA	Tipo R.E	Subtotal	
					%	Cuántía			
Soporte técnico incluido por correo en menos de 24 h.	Suscripción anual Plan Premium Favsy (acceso durante 12 meses a todas las funcionalidades)	1	60,00 €	60	0	0	21	0	60
SUBTOTAL									60
Gastos Importe antes de impuestos									60
Forma de pago									
Cheque									
I.V.A.					Tipo	4%	10%	21%	
					Base Imp.	—	—	60	I.V.A
									12,60
R.E					Tipo	0,5%	1,4%	5,2%	
					Base				
					Imp.	—	—	—	
					Cuota	—	—	—	R.E
									0
TOTAL FACTURA									72,60

Banco Ibérico

Plaza de España, 30
28032 Madrid

Código cuenta cliente (CCC)

IBAN	Entidad	Oficina	DC	Nº Cuenta
ES72	4356	2105	97	13432674

Euros. 72,60

Páguese, por este cheque, a : Favsy

Euros (en letras) SESENTA Y DOS EUROS Y SESENTA CENTIMOS

Serie n.º N° 00012025

En Cuenca a 25 de MAYO de 2026

Fdo. Cristian Guerrero Terciado

11. Metodología de Programación

11.1. Tipo de Metodología

Se ha seguido un enfoque incremental e iterativo, similar a Scrum, dividiendo el desarrollo en sprints semanales. Cada sprint tiene un conjunto de funcionalidades objetivo (backlog del sprint) y termina con una versión ejecutable del software que puede ser probada.

11.2. Fases de Trabajo y Diagrama de Gantt

El proyecto se ha dividido en las siguientes fases:

Fase	Duración estimada
1. Análisis de requisitos y diseño de la BBDD	1 semana
2. Diseño de la arquitectura del sistema	1 semana
3. Implementación del backend (API + modelos + rutas)	3 semanas
4. Implementación del cliente de escritorio MAUI	3 semanas
5. Desarrollo de la mini app móvil	1 semana
6. Pruebas de integración y corrección de errores	1 semana
7. Redacción de la documentación	1 semana

11.3. Organización del Código

Se ha aplicado el patrón MVVM en el cliente y una arquitectura por capas en el backend. El control de versiones se gestiona con Git, con ramas separadas para cada nueva funcionalidad (feature branches) y fusiones a la rama principal tras pasar las pruebas.

11.4. Pruebas Realizadas

- Pruebas unitarias de los modelos Pydantic: validación de campos obligatorios, lógica de compensación y modalidad.
- Pruebas de integración de la API con test_API.py: se verifican todos los endpoints con casos de éxito y de error.
- Pruebas de aceptación concurrente de favores: dos peticiones simultáneas al mismo endpoint para verificar que solo una tiene éxito.
- Pruebas de envío simultáneo de mensajes en el chat.

- Pruebas de validación de roles: intentos de acceso a endpoints de administrador sin permisos.

12. Conclusiones

El desarrollo de este proyecto ha permitido aplicar de forma integrada y práctica las competencias adquiridas a lo largo del ciclo formativo de Desarrollo de Aplicaciones Multiplataforma, demostrando que es posible construir un sistema software completo, cohesionado y funcional de forma autónoma.

Desde el punto de vista técnico, se ha conseguido diseñar e implementar un ecosistema de tres capas perfectamente interconectadas. El backend, construido con FastAPI y Python, ha demostrado ser una elección acertada gracias a su modelo asíncrono nativo basado en `async/await`, que permite gestionar múltiples peticiones concurrentes sin bloquear el servidor. La integración con MongoDB mediante el cliente Motor ha resultado natural y eficiente, especialmente para estructuras de datos anidadas como los mensajes de chat o los arrays de miembros de un grupo, donde un modelo relacional habría exigido múltiples JOINS costosos. La autenticación mediante JWT con firmado HS256 y el almacenamiento de contraseñas con `bcrypt` garantizan que el sistema cumple con los estándares actuales de seguridad en aplicaciones web.

El cliente de escritorio desarrollado con .NET MAUI ha supuesto un reto técnico considerable, especialmente en lo relativo al patrón MVVM y al sistema de navegación mediante AppShell. La separación estricta entre vista, VistaModelo y servicio ha permitido mantener el código organizado y testable, y el uso de `ObservableCollection` junto con `INotifyPropertyChanged` ha hecho que la interfaz responda de forma reactiva a cualquier cambio en los datos sin necesidad de recargar pantallas completas. El sistema de temas claro y oscuro y la internacionalización en español e inglés, implementados intercambiando diccionarios de recursos en tiempo de ejecución, han añadido un nivel de pulido que eleva el producto por encima de un simple prototipo académico.

La mini aplicación móvil desarrollada en Kotlin para Android ha sido, posiblemente, el componente más enriquecedor desde el punto de vista del aprendizaje. Trabajar con el ecosistema nativo de Android ha implicado dominar conceptos específicos de la plataforma que no se pueden abstraer fácilmente: el ciclo de vida de `Activities` y `Fragments`, la navegación con `NavGraph` y `NavController`, la comunicación reactiva mediante `LiveData` y `ViewModel`, y la gestión eficiente de listas con `RecyclerView` y `DiffUtil`. La implementación de dos `NavGraphs` separados para el flujo de autenticación y el flujo principal, con destrucción explícita de la pila de actividades al hacer login y logout mediante flags de `Intent`, refleja un conocimiento sólido de cómo Android gestiona la memoria y la navegación. Además, la integración de geolocalización con `FusedLocationProviderClient` y geocodificación inversa con el `Geocoder` nativo ha añadido una funcionalidad de valor real para el usuario final.

Desde el punto de vista funcional, el sistema cubre todos los casos de uso definidos en los objetivos: publicación y gestión de favores con ciclo de vida completo, mensajería

privada y grupal, grupos temáticos con roles diferenciados, sistema de valoraciones, notificaciones automáticas y panel de administración. La gestión de la concurrencia en operaciones críticas, como la aceptación simultánea de un favor por dos usuarios o el envío paralelo de mensajes en el chat, se ha resuelto de forma elegante aprovechando la atomicidad a nivel de documento que garantiza MongoDB, sin necesidad de implementar mecanismos de bloqueo explícitos en el servidor.

Desde el punto de vista empresarial, el plan de negocio elaborado demuestra que la plataforma tiene una propuesta de valor diferencial real en el mercado de la economía colaborativa: combina favores gratuitos, recíprocos y remunerados en un mismo ecosistema, ofrece acceso tanto desde escritorio como desde móvil, e integra chat en tiempo real y grupos temáticos administrables, características que ninguno de los competidores analizados reúne de forma conjunta. El modelo freemium con un umbral de rentabilidad de tan solo 25 suscriptores Premium confirma la viabilidad económica del proyecto incluso en sus fases más tempranas.

Desde el punto de vista personal, este proyecto ha supuesto el mayor reto de autogestión y toma de decisiones de arquitectura afrontado durante la formación. Haber tenido que elegir entre distintas tecnologías, justificar esas elecciones, resolver problemas de concurrencia, diseñar una base de datos NoSQL eficiente y mantener coherencia entre tres componentes que evolucionan en paralelo ha desarrollado una madurez como desarrollador que va más allá de los conocimientos técnicos concretos. El resultado es un producto del que el autor se siente orgulloso no solo como ejercicio académico, sino como una herramienta con potencial de impacto social real en comunidades de vecinos, asociaciones y entornos educativos.

13. Trabajo Futuro y Posibles Mejoras

Las siguientes funcionalidades se proponen como líneas de mejora y ampliación futura del proyecto:

- Chat en tiempo real con WebSockets: sustituir el sistema de polling actual por conexiones WebSocket para recibir mensajes instantáneamente sin necesidad de refrescar.
- Geolocalización de favores: integrar mapas interactivos con marcadores que muestren la ubicación aproximada de los favores presenciales.
- Sistema de pagos integrado: implementar Stripe o PayPal para gestionar de forma segura los favores con compensación económica.
- Notificaciones push nativas: integrar Firebase Cloud Messaging (FCM) para notificaciones push en Android e iOS.
- Versión web (SPA): desarrollar un cliente web con Blazor WebAssembly o React que consume el mismo backend.
- Panel de administrador global: interfaz dedicada para supervisar la actividad de la plataforma, gestionar usuarios y resolver disputas.

- Sistema de valoraciones bidireccional: actualmente solo se valora al usuario; en el futuro se podría valorar también el favor en sí.
- Internacionalización: soporte para múltiples idiomas en la interfaz de usuario.

14. Glosario de Términos

Término	Definición
API REST	Interfaz de programación de aplicaciones que sigue el estilo arquitectónico REST, usando HTTP y JSON para la comunicación entre cliente y servidor.
JWT (JSON Web Token)	Estándar abierto (RFC 7519) para transmitir información entre partes de forma segura como un objeto JSON firmado digitalmente.
MongoDB	Sistema de base de datos NoSQL orientado a documentos que almacena datos en formato BSON (Binary JSON).
MVVM	Patrón de diseño Model-View-ViewModel que separa la lógica de negocio de la interfaz gráfica.
.NET MAUI	Multi-platform App UI, framework de Microsoft para desarrollar aplicaciones nativas de escritorio y móvil desde un único código base en C#.
FastAPI	Framework web moderno de Python para construir APIs con validación automática, documentación OpenAPI y soporte asíncrono nativo.
Pydantic	Librería de Python para validación de datos y gestión de configuración mediante anotaciones de tipo.
Bcrypt	Función de hash de contraseñas diseñada para ser computacionalmente costosa y resistente a ataques de fuerza bruta.
Motor	Driver asíncrono de Python para MongoDB, compatible con asyncio y FastAPI.
Shell (MAUI)	Componente de .NET MAUI que proporciona la estructura de navegación principal de la aplicación mediante URI.
Data Binding	Mecanismo que sincroniza automáticamente los datos entre la vista (XAML) y el ViewModel.
Endpoint	URL concreta de la API que acepta peticiones HTTP de un tipo específico (GET, POST, PUT, DELETE, PATCH).

15. Bibliografía

- FastAPI Official Documentation. (2024). FastAPI — Modern Web Framework for Building APIs with Python. <https://fastapi.tiangolo.com>
- MongoDB, Inc. (2024). MongoDB Manual. <https://www.mongodb.com/docs/manual>
- Microsoft. (2024). .NET MAUI Documentation. <https://learn.microsoft.com/en-us/dotnet/maui>
- Pydantic. (2024). Pydantic V2 Documentation. <https://docs.pydantic.dev/latest>
- Python Software Foundation. (2024). Python 3.11 Documentation. <https://docs.python.org/3.11>
- python-jose. (2024). Python JOSE library. <https://python-jose.readthedocs.io>
- Android Developers. (2024). *EncryptedSharedPreferences — Security library for Android*. <https://developer.android.com/reference/androidx/security/crypto/EncryptedSharedPreferences>

16. Enlace al Repositorio

El código fuente completo del proyecto (backend y cliente de escritorio) está disponible en el siguiente repositorio de [CristianGuet](#)

17. ANEXO I CURRÍCULUM VITAE EUROPASS

A continuación se incluye el Currículum Vitae del promotor del proyecto en formato Europass.



Cristian Guerrero Terciado

Fecha de nacimiento: 01/08/2006 | **Nacionalidad:** Española | **Género:** Masculino | **Dirección** España
(Domicilio)

● **EDUCACIÓN Y FORMACIÓN**

22/08/2024 - Actual - TARANCON (CUENCA), ESPAÑA

DAM- CIFP N°1

- desarrollo de aplicaciones multiplataforma (aplicaciones web, móviles y de escritorio)
- programación y bases de datos (Java, SQL, APIs, gestión de datos y lógica de negocio)
- tecnologías informáticas y sistemas (redes, sistemas operativos, seguridad y entornos de desarrollo)
- trabajo en equipo y metodologías de desarrollo (análisis, diseño, documentación y mantenimiento de software)
- conocimiento del sector tecnológico y transformación digital en empresas
- resolución de problemas informáticos y adaptación a nuevas tecnologías
- uso de herramientas de desarrollo profesional (Git, IDEs, control de versiones y despliegue de aplicaciones)

Nivel en el MEC: 1

● **CAPACIDADES**

Trabajo en equipo | Responsabilidad en el trabajo | Organización y puntualidad

18. ANEXO II NOMINA

Empresa:Favsy
Domicilio:C/ Fuensanta 3, 16002
Cuenca
CIF:123456

NIF:20979078N
CCC Seguridad Social:123456

Trabajador: Profesional de Oficio de 1ª
Nº Afiliación Seguridad Social:1234567

Categoría profesional:
Profesional de Oficio de 1ª
De mayo

Grupo de cotización: 8

Periodo de liquidación: del 1
I. DEVENGOS

Al 31
IMPORTE

De 2026
TOTALES

Total días:30

1. Percepciones salariales

Salario base.....979,19 €

Complementos salariales.....114,52 €

.....273,43 €

.....

.....

Horas extraordinarias

Voluntarias.....0

Fuerza mayor.....0

Horas complementarias (contratos a tiempo parcial).....0

Gratificaciones extraordinarias.....0

Salario en especie.....0

2. Percepciones no salariales

Indemnizaciones o suplidos.....0

.....0

Prestaciones e indemnizaciones de la Seguridad Social.....0

Indemnizaciones por traslados, suspensiones o despidos.....0

.....0

Otras percepciones no salariales.....0

.....0

(A) TOTAL DEVENGADO: 1.367,14 €

II. DEDUCCIONES

1. Aportaciones del trabajador a las cotizaciones de la Seguridad Social y conceptos de recaudación conjunta

Contingencias.....4,70 %.....64,26 €

comunes.....

.....

Desempleo.....1,60 %.....21,87 €

Formación.....0,10 %.....1,37 €

profesional.....

Horas extraordinarias por fuerza mayor...0 %.....0,00 €

Otras horas.....0 %.....0,00 €

extraordinarias.....

Total aportaciones:.....87,50 €

2. Impuesto sobre la Renta de las Personas Físicas.....27,34 €

3. Anticipos.....0

4. Valor de los productos recibidos en especie.....0

5. Otras deducciones.....0

(B) TOTAL A DEDUCIR: 114,84 €

LÍQUIDO TOTAL A PERCIBIR (A - B) 1.252,30 €

Firma y sello de la empresa

RECIBÍ

DETERMINACIÓN DE LAS BASES DE COTIZACIÓN A LA SEGURIDAD SOCIAL Y CONCEPTOS DE RECAUDACIÓN CONJUNTA Y DE LA BASE SUJETA A RETENCIÓN DEL IRPF Y APORTACIÓN DE LA EMPRESA

1. Base de cotización por contingencias comunes

Remuneración mensual.....1.093,71 €

Prorrata de pagas extraordinarias.....273,43 €

TOTAL: 1.367,14 €

2. Base de cotización por contingencias profesionales y conceptos de recaudación conjunta

Concepto

BASE

TIPO

APORTACION EMPRESA

AT y EP.....I.T (0,63%) + I.M.S (0,36%).....1.367,14 €.....1,50 %.....20,51 €

Desempleo.....1.367,14 €.....5,50 %.....75,19 €

Formación profesional.....1.367,14 €.....0,60 %.....8,20 €

FOGASA.....1.367,14 €.....0,20 %.....2,73 €

3. Base de cotización adicional por horas extraordinarias.....0.....0 %.....

4. Base de cotización adicional por horas extraordinarias por fuerza mayor.....0.....0 %.....

5. Base sujeta a retención del IRPF.....1.367,14 €.....

TOTAL.....106,63 €